

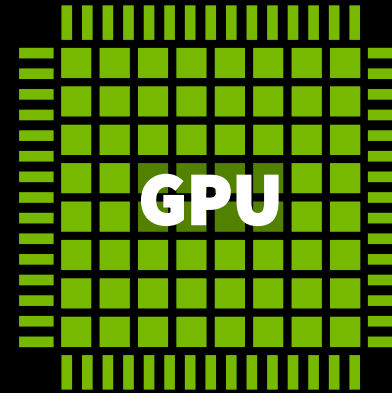
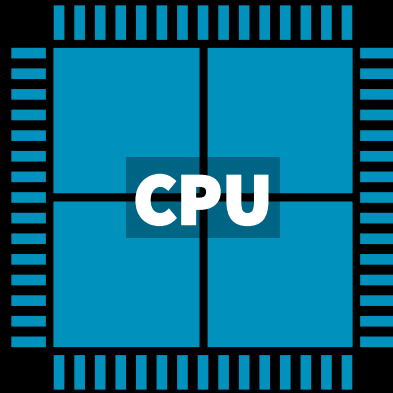
THE CUDA C++ DEVELOPER'S TOOLBOX

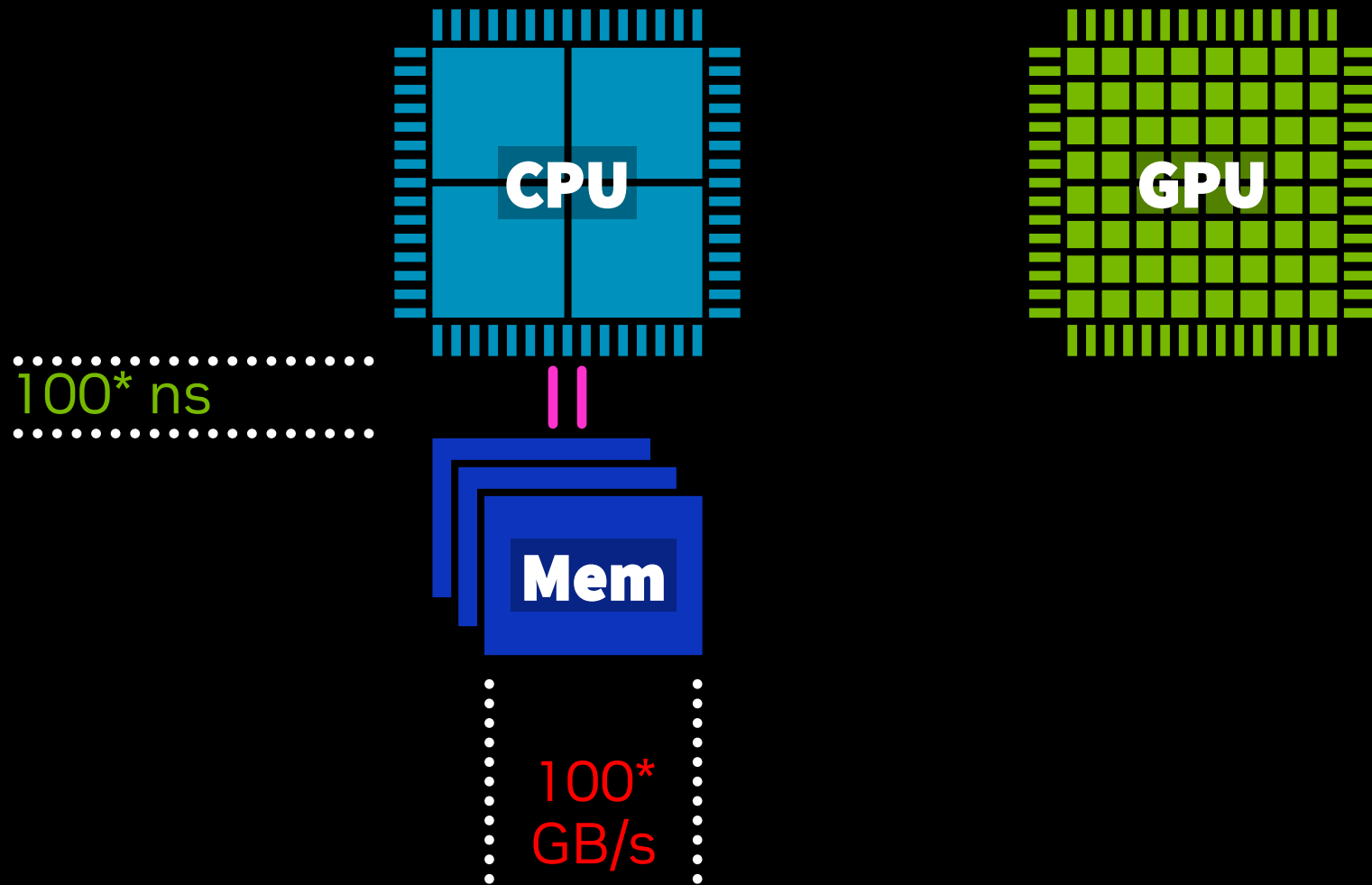
Bryce Adelstein Lelbach

Principal Architect

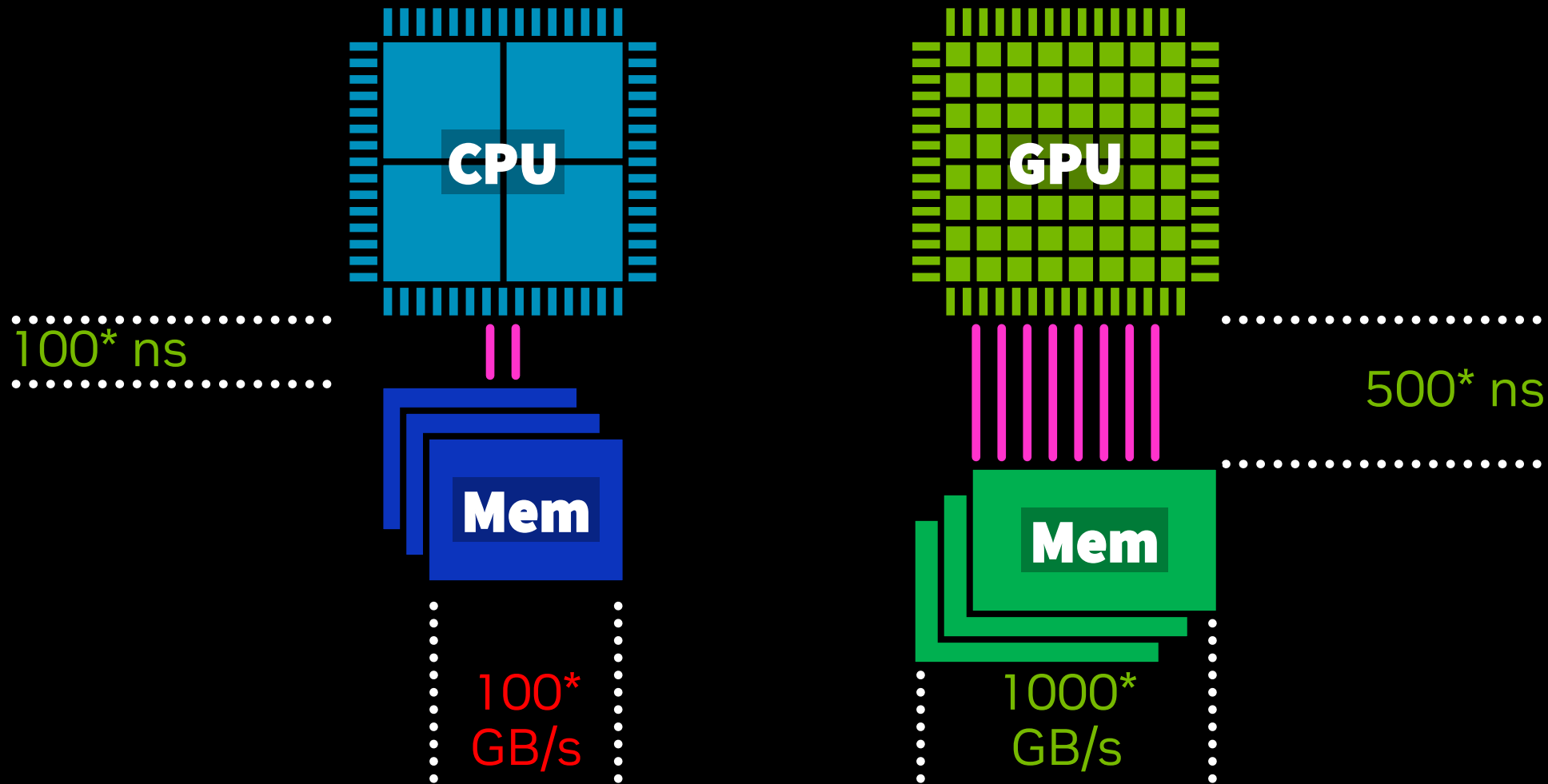
✉ brycelelbach@gmail.com [@blelbach](https://twitter.com/blelbach)







Numbers are made up and
for expository purposes only.

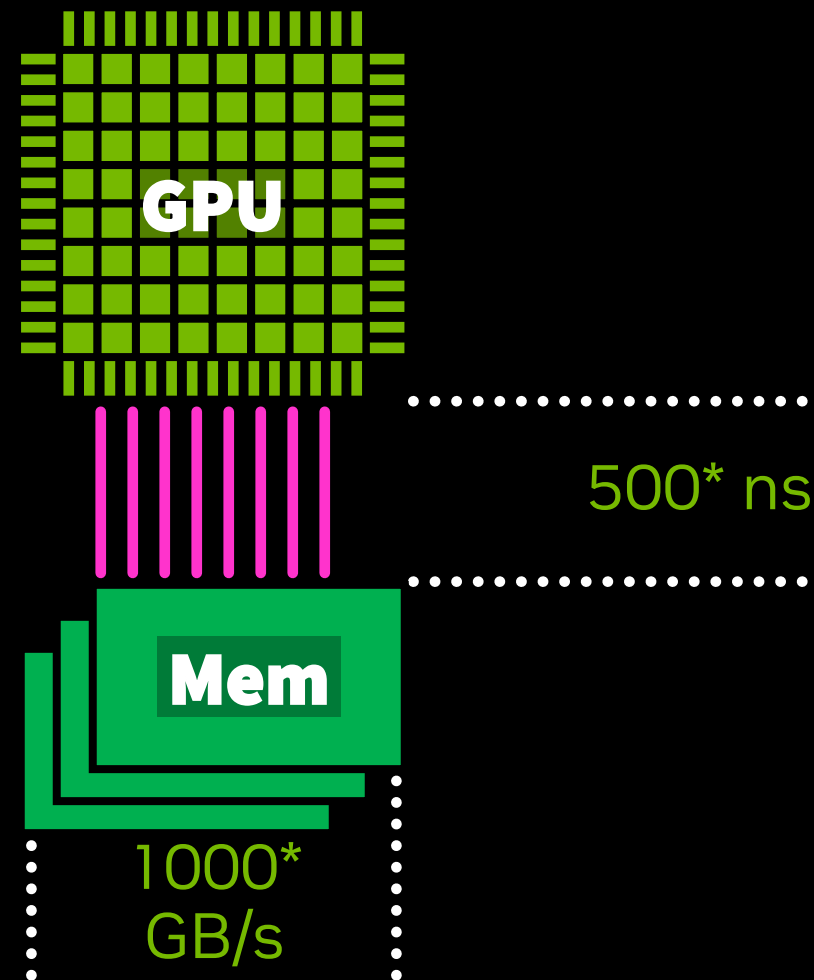


Numbers are made up and
for expository purposes only.

You need a sufficient workload to benefit from GPUs.

$O(N)$ compute => MBs of data.

More than $O(N)$ compute => less data.



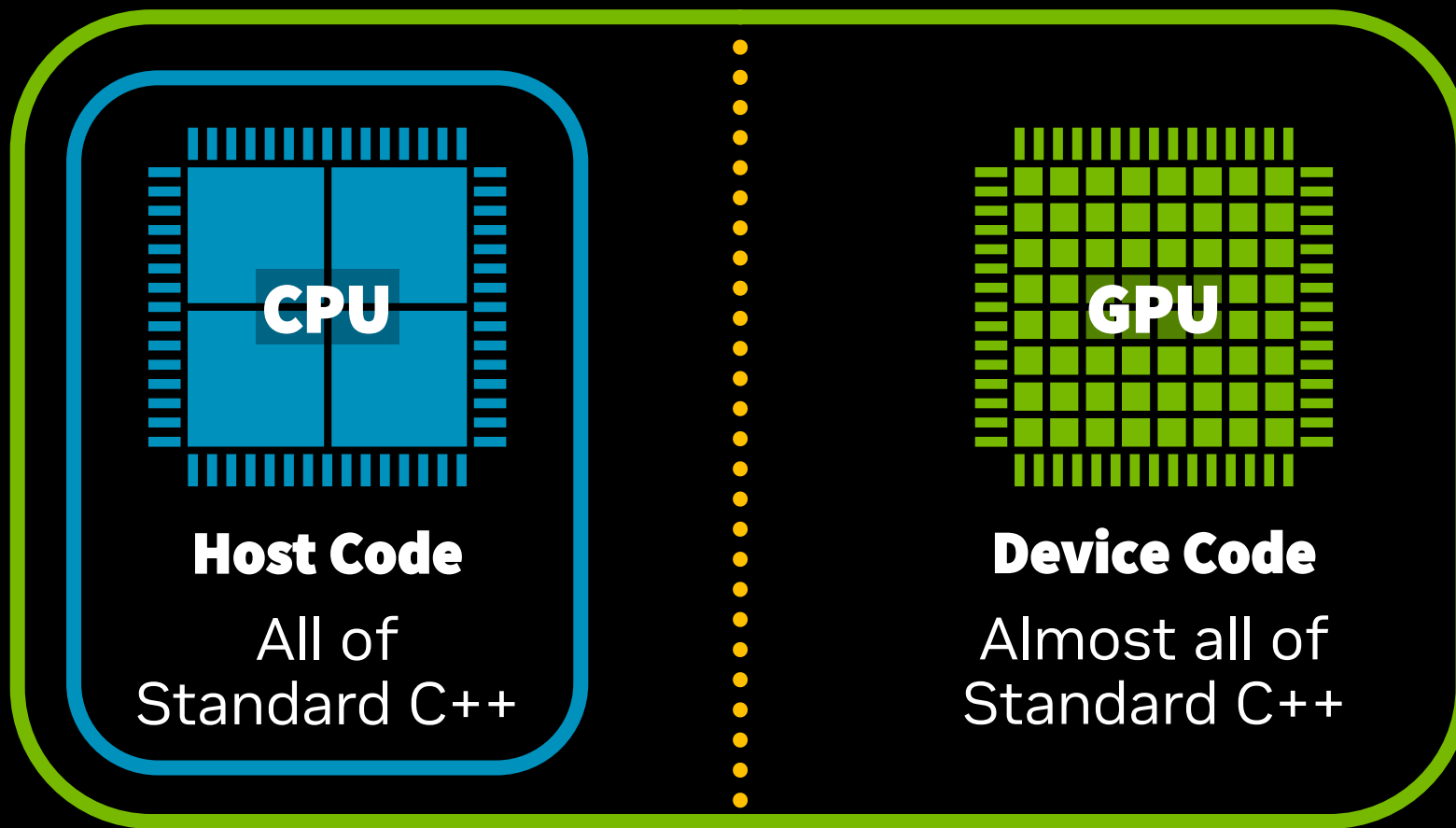
Numbers are made up and
for expository purposes only.

How do we program a GPU in C++?

How do we program a GPU in C++?

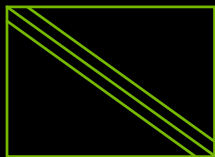
CUDA C++ and Accelerated Libraries

CUDA C++ is an extension of Standard C++ for writing programs that simultaneously run on CPUs (hosts) & GPUs (devices).

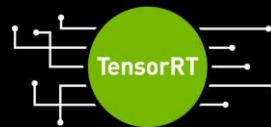




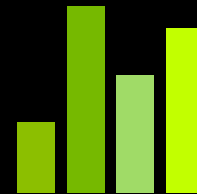
cuBLAS



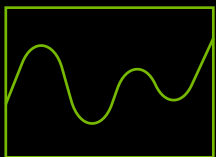
cuSPARSE



**cuFile
(GDS)**



NVBench



cuFFT

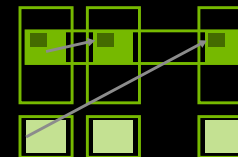


cuDSS

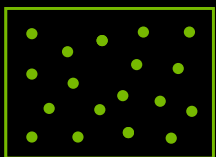
**CV
CUDA**

cuLitho

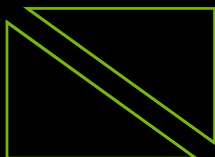
**Cooperative
Groups**



NVSHMEM



cuRAND

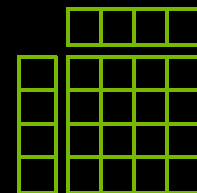


cuSOLVER

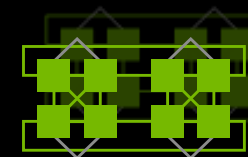
RAPIDS



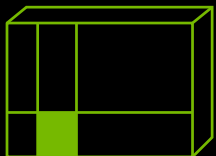
CUDA-Q



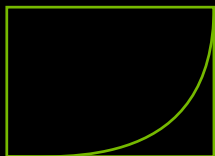
CUTLASS



NCCL



cuTENSOR



CUDA Math



**CUDA
Runtime**



Ambient Temperature = 20°C
Heat Transfer Coefficient (k) = 0.5

Current
Temperature

42°



Next
Temperature

31°

24°



22°

50°



35°

Next Temp = Current Temp + Heat Transfer Coefficient * (Ambient Temp – Current Temp)

```
int steps = 3;  
float k = 0.5;  
float ambient_temp = 20;  
std::vector<float> cups{42, 24, 50};
```

```
int steps = 3;  
float k = 0.5;  
float ambient_temp = 20;  
std::vector<float> cups{42, 24, 50};
```

```
auto op = [=] (float t) {  
    float diff = ambient_temp - t;  
    return t + k * diff;  
};
```

```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    ...
}
```

```

int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

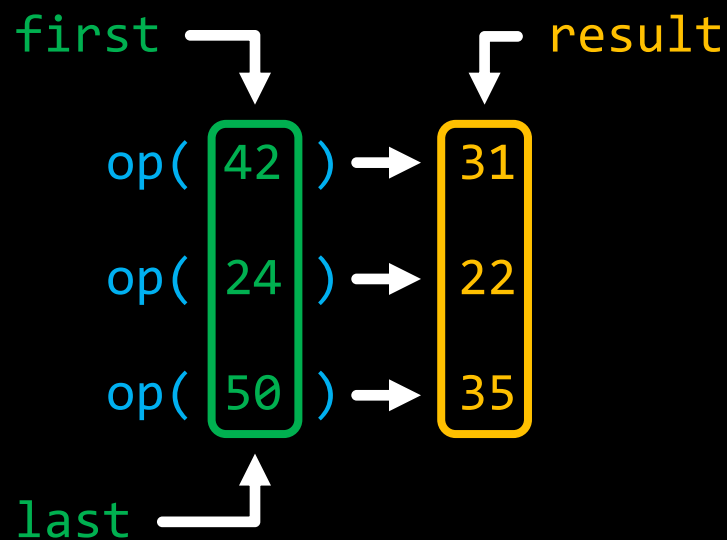
for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}

```

```

std::transform(first, last,
               result, op);

```





```
t + k * diff
```



```
g++ main.cpp -o a.out
```



```
vm1a.f32
```

Host code
executable
by CPU

**CUDA
C++**

`t + k * diff`

NVCC

`nvcc main.cpp -o a.out`

Host code
executable
by CPU

`vm1a.f32`

`fma.rn.f32`

Device code
executable
by GPU

```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}
```

```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}
```

```

int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}

```

```
void a();
```

```
__host__
void b();
```

Only executable
by the CPU.
Includes all
unannotated
functions.

```

int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}

```

```

void a();

__host__
void b();

```

Only executable
by the CPU.
Includes all
unannotated
functions.

```

__device__
void c();

```

Only executable
by the GPU.

```

int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}

```

```

void a();

__host__
void b();

```

Only executable
by the CPU.
Includes all
unannotated
functions.

```

__device__
void c();

```

Only executable
by the GPU.

```

__host__
__device__
void d();

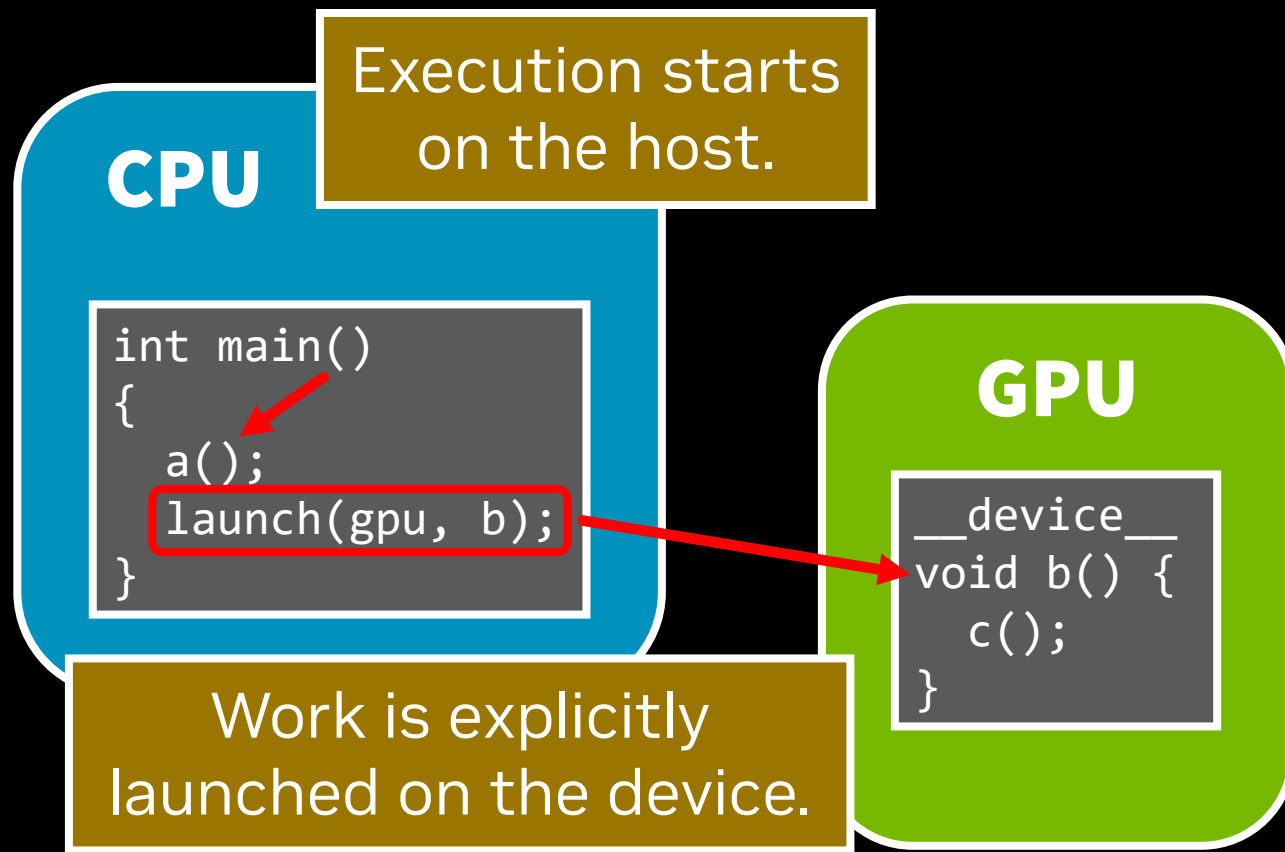
```

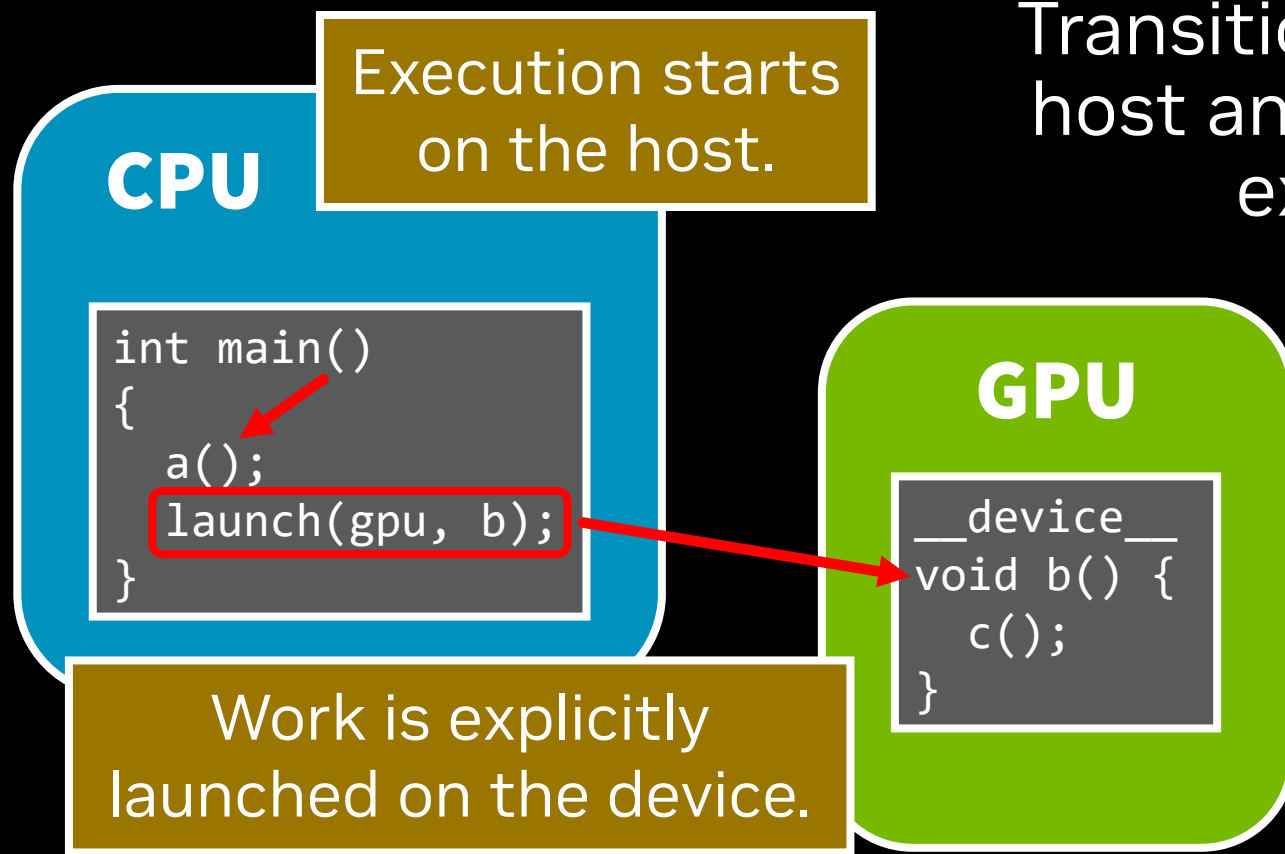
Executable by
CPU and GPU.

Execution starts
on the host.

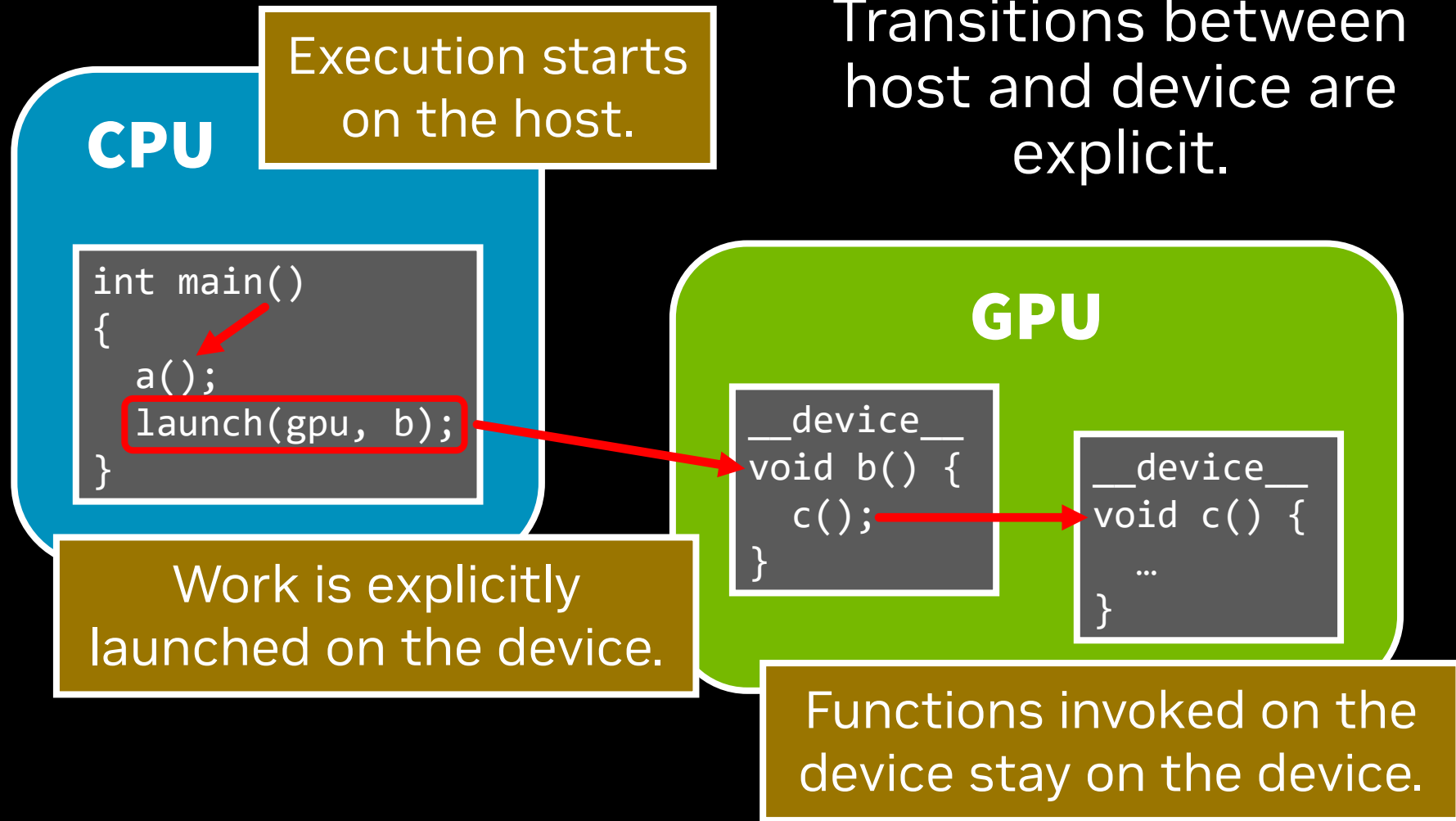
CPU

```
int main()  
{  
    a();  
    ...  
}
```





Transitions between host and device are explicit.



```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    std::transform(cups.begin(), cups.end(),
                  cups.begin(), op);
}
```

```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
std::vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    thrust::transform(thrust::cuda::par, cups.begin(), cups.end(),
                      cups.begin(), op);
}
```

```
int steps = 3;
float k = 0.5;
float ambient_temp = 20;
thrust::universal_vector<float> cups{42, 24, 50};

auto op = [=] __host__ __device__ (float t) {
    float diff = ambient_temp - t;
    return t + k * diff;
};

for (int step : std::views::iota(0, steps))
{
    std::println("{} {}", step, cups);
    thrust::transform(thrust::cuda::par, cups.begin(), cups.end(),
                     cups.begin(), op);
}
```



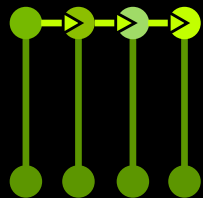
The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



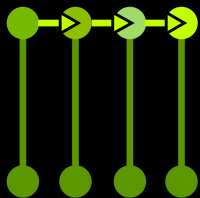
Standard Algorithms

- `thrust::transform_reduce`
- `thrust::inclusive_scan`
- `thrust::sort`
- `thrust::copy`
- ...



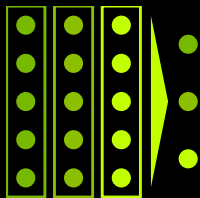
The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



Standard Algorithms

- `thrust::transform_reduce`
- `thrust::inclusive_scan`
- `thrust::sort`
- `thrust::copy`
- ...



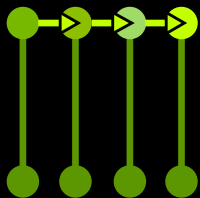
Extended Algorithms

- `thrust::reduce_by_key`
- `thrust::sort_by_key`
- `thrust::tabulate`
- `thrust::gather`
- ...



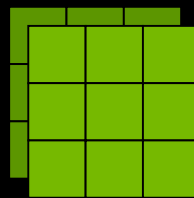
The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



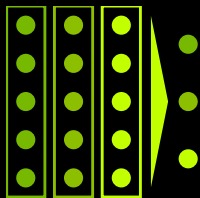
Standard Algorithms

- `thrust::transform_reduce`
- `thrust::inclusive_scan`
- `thrust::sort`
- `thrust::copy`
- ...



Containers

- `thrust::device_vector`
- `thrust::host_vector`
- `thrust::universal_vector`
- `thrust::allocate_unique`
- ...



Extended Algorithms

- `thrust::reduce_by_key`
- `thrust::sort_by_key`
- `thrust::tabulate`
- `thrust::gather`
- ...

```
std::vector<T> h0(1 << 30);  
std::vector<T> h1(1 << 30);
```

```
h0 = h1;
```

std::vector's contents might only be accessible in *host code*.

Construction and assignment is *serial*.

```
std::vector<T> h0(1 << 30);  
std::vector<T> h1(1 << 30);
```

```
h0 = h1;
```

```
thrust::universal_vector<T> u0(1 << 30);  
thrust::universal_vector<T> u1(1 << 30);
```

```
u0 = u1;
```

std::vector's contents might only be accessible in *host* code.

Construction and assignment is *serial*.

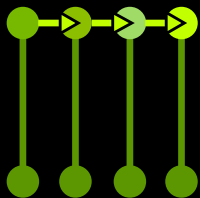
thrust::universal_vector's contents are accessible in host and device code.

Construction and assignment is parallel.



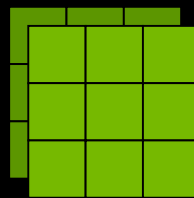
The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



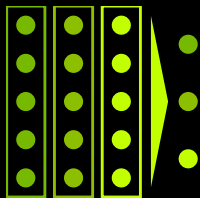
Standard Algorithms

- `thrust::transform_reduce`
- `thrust::inclusive_scan`
- `thrust::sort`
- `thrust::copy`
- ...



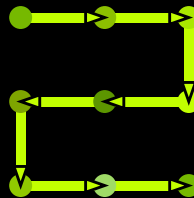
Containers

- `thrust::device_vector`
- `thrust::host_vector`
- `thrust::universal_vector`
- `thrust::allocate_unique`
- ...



Extended Algorithms

- `thrust::reduce_by_key`
- `thrust::sort_by_key`
- `thrust::tabulate`
- `thrust::gather`
- ...

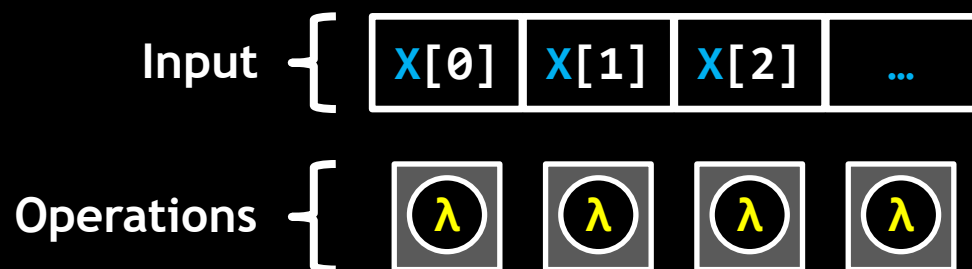


Iterators

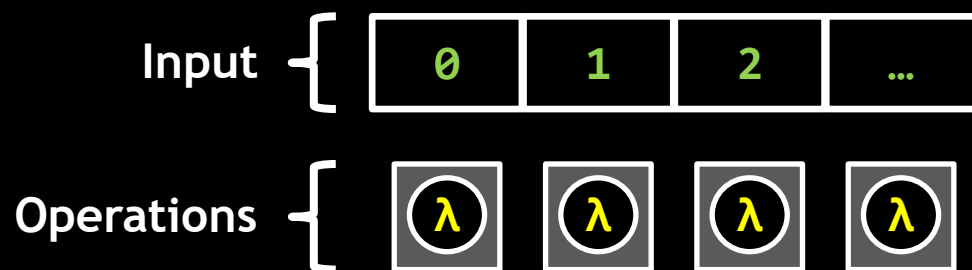
- `thrust::counting_iterator`
- `thrust::constant_iterator`
- `thrust::transform_iterator`
- `thrust::zip_iterator`
- ...

```
thrust::universal_vector X(N);
```

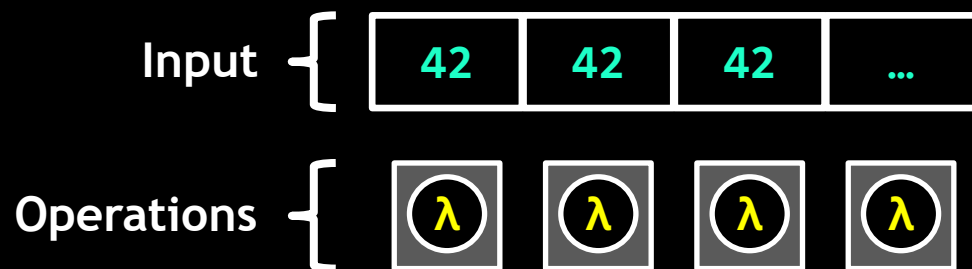
```
thrust::for_each_n(  
    thrust::cuda::par, X.begin(), N,  
    [...] __host__ __device__ (auto& obj) { ... }));
```



```
auto i = thrust::make_counting_iterator(0);  
  
thrust::for_each_n(  
    thrust::cuda::par, i, N,  
    [...] __host__ __device__ (auto idx) { ... }));
```



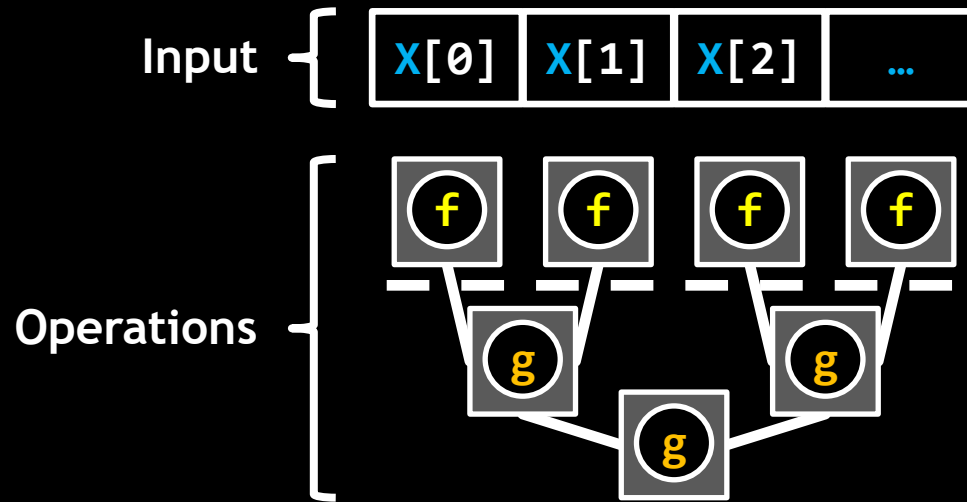
```
auto i = thrust::make_constant_iterator(42);  
  
thrust::for_each_n(  
    thrust::cuda::par, i, N,  
    [...] __host__ __device__ (auto idx) { ... }));
```



```
thrust::universal_vector X(N), tmp(N);
```

```
thrust::transform(thrust::cuda::par,  
    X.begin(), X.end(), tmp.begin(), f);
```

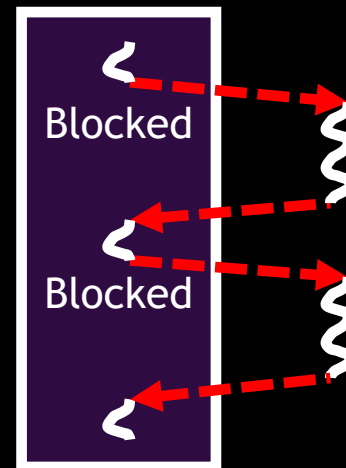
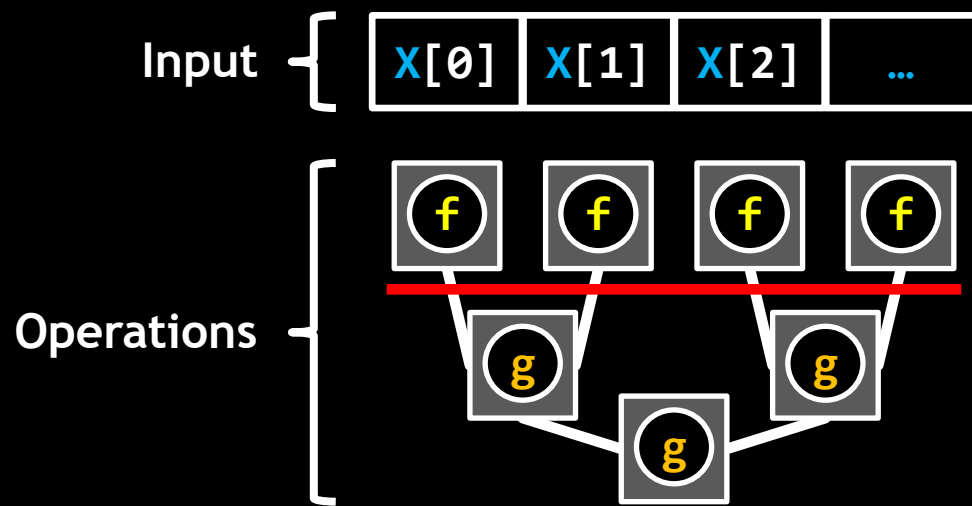
```
auto r = thrust::reduce(thrust::cuda::par,  
    tmp.begin(), tmp.end(), T{}, g);
```




```
thrust::universal_vector X(N), tmp(N);
```

```
thrust::transform(thrust::cuda::par,  
    X.begin(), X.end(), tmp.begin(), f);
```

```
auto r = thrust::reduce(thrust::cuda::par,  
    tmp.begin(), tmp.end(), T{}, g);
```

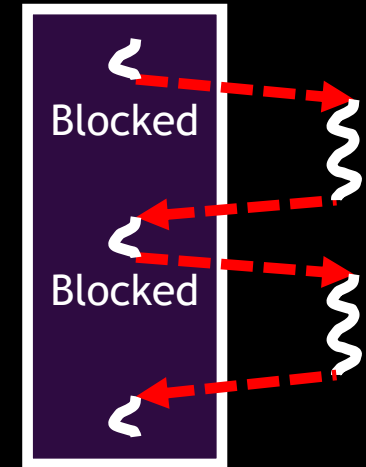
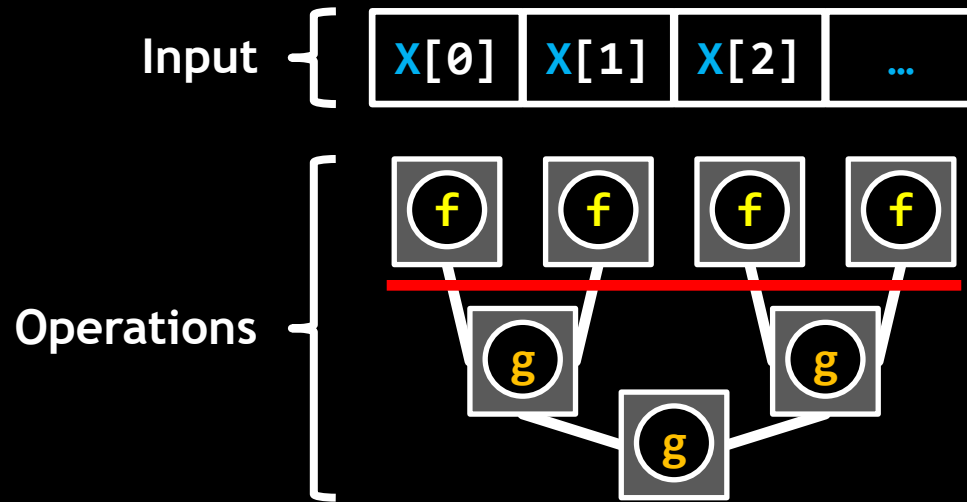


```
thrust::universal_vector X(N), tmp(N);
```

```
thrust::transform(thrust::cuda::par,  
    X.begin(), X.end(), tmp.begin(), f);
```

```
auto r = thrust::reduce(thrust::cuda::par,  
    tmp.begin(), tmp.end(), T{}, g);
```

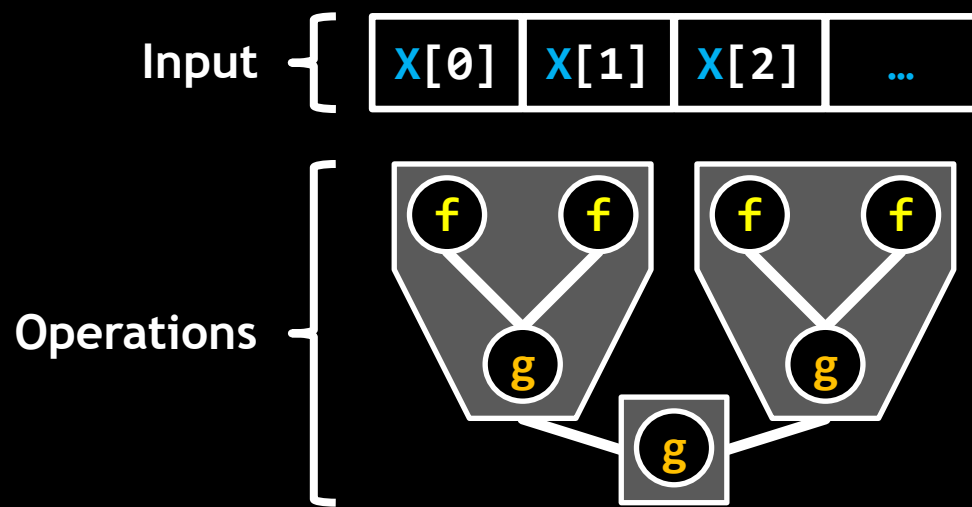
Materializes $O(N)$
temporary
storage in
precious device
memory.



```
thrust::universal_vector X(N);
```

```
auto tmp = thrust::make_transform_iterator(  
    X.begin(), f);
```

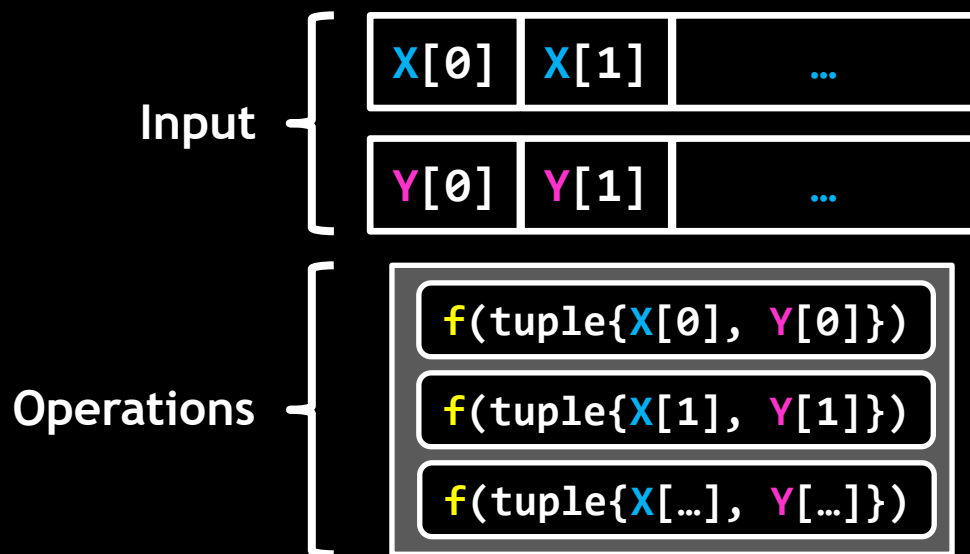
```
auto r = thrust::reduce(thrust::cuda::par,  
    tmp, tmp + N, T{}, g);
```



```
thrust::universal_vector X(N), Y(N);
```

```
auto XY = thrust::make_zip_iterator(  
    thrust::cuda::par, X.begin(), Y.begin());
```

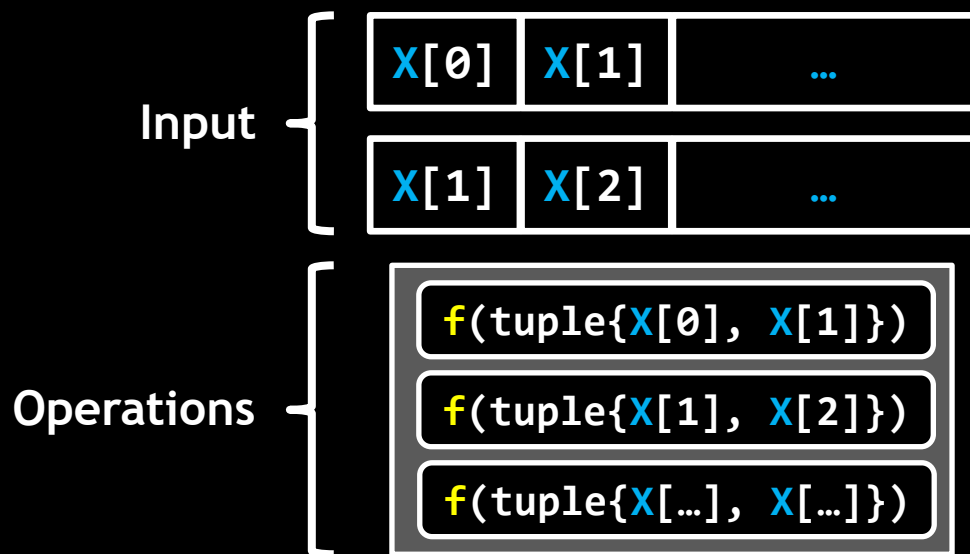
```
thrust::for_each_n(  
    thrust::cuda::par, XY, N, f);
```



```
thrust::universal_vector X(N);

auto adj = thrust::make_zip_iterator(
    thrust::cuda::par, X.begin(), X.begin() + 1);

thrust::for_each_n(
    thrust::cuda::par, adj, N - 1, f);
```



5	1		5	-	1		4
2	9		2	-	9		-7
9	6	→	9	-	6	→	3 → 7
7	8		7	-	8		-1
A	B		A	-	B		max(abs)

```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

5

2

9

7

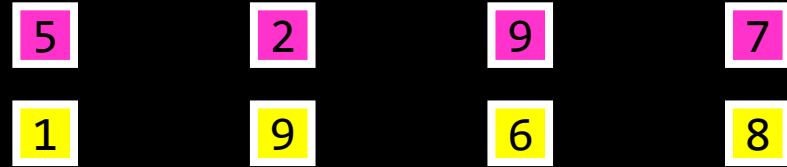
1

9

6

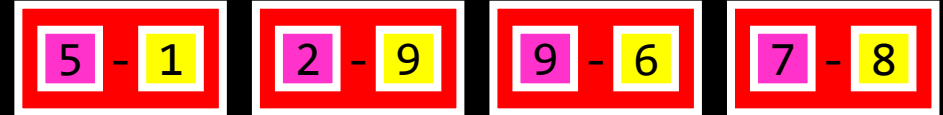
8

```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```



```
thrust::universal_vector<int> diffs(A.size());
```

```
thrust::transform(thrust::cuda::par,  
    A.begin(), A.end(), B.begin(), diffs.begin(),  
    [] __host__ __device__ (int a, int b)  
    { return abs(a - b); });
```



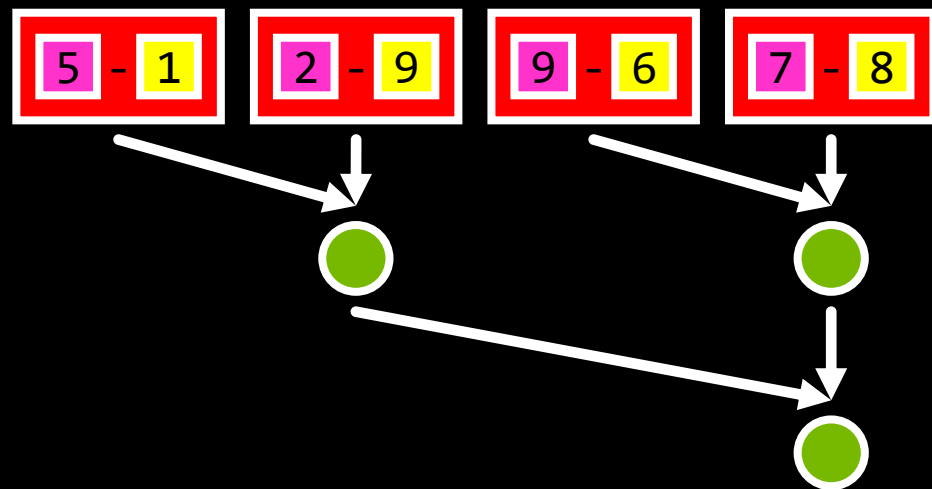

```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```



```
thrust::universal_vector<int> diffs(A.size());
```

```
thrust::transform(thrust::cuda::par,  
    A.begin(), A.end(), B.begin(), diffs.begin(),  
    [] __host__ __device__ (int a, int b)  
    { return abs(a - b); });
```

```
auto max_diff = thrust::reduce(  
    thrust::cuda::par,  
    diffs.begin(), diffs.end(),  
    0,  
    cuda::maximum{});
```

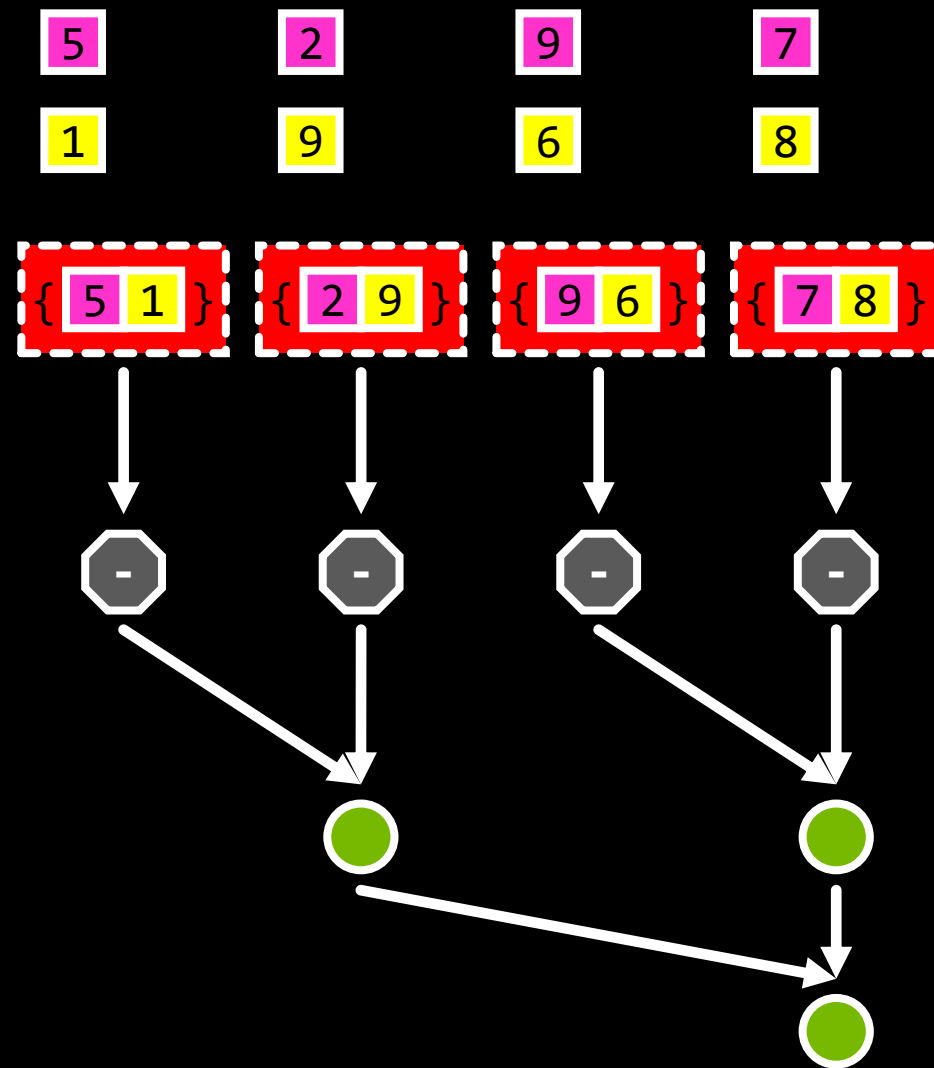


```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

```
auto AB = thrust::make_zip_iterator(  
    A.begin(), B.begin());
```

```
auto diffs = thrust::make_transform_iterator(  
    AB.begin(),  
    [] __host__ __device__  
    (cuda::std::tuple<int, int> ab) {  
        auto [a, b] = ab;  
        return abs(a - b)  
    });
```

```
auto max_diff = thrust::reduce(  
    thrust::cuda::par,  
    diffs, diffs + A.size(),  
    0,  
    cuda::maximum{});
```

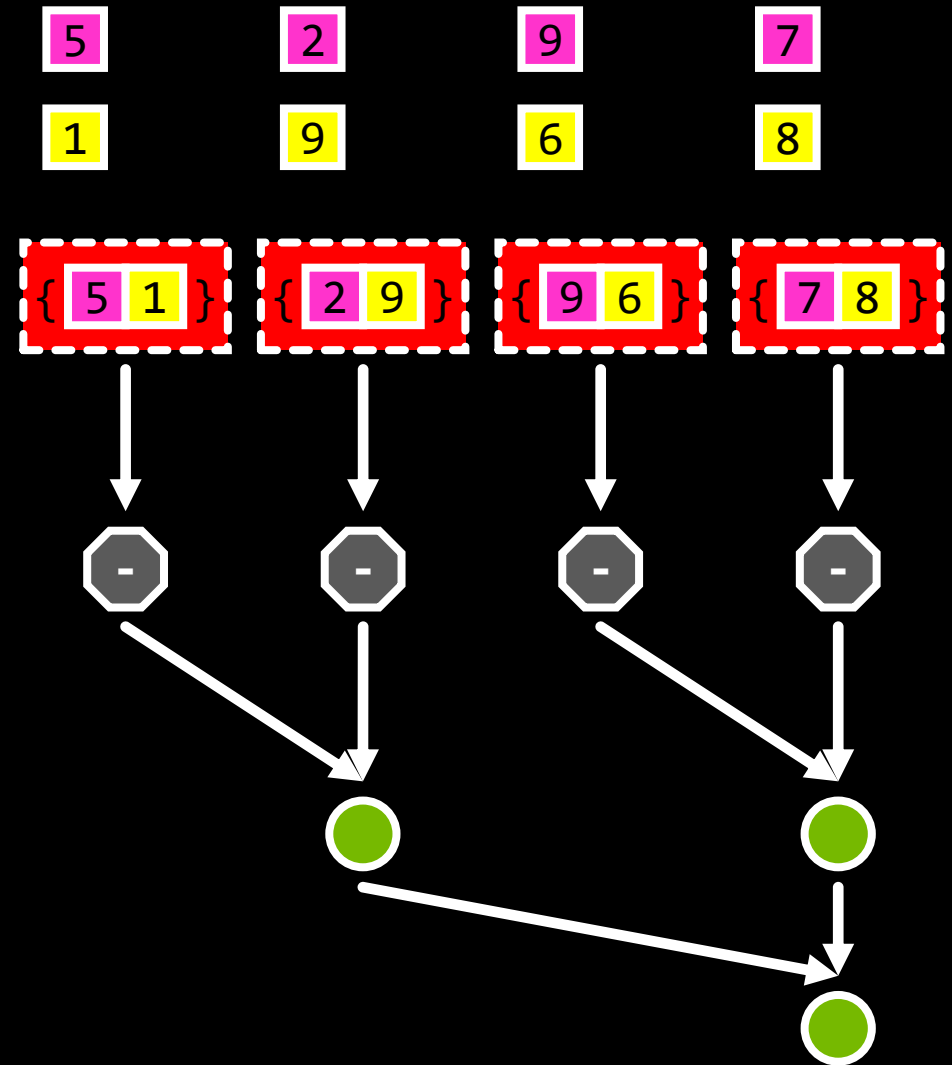


```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

```
auto AB = thrust::make_zip_iterator(  
    A.begin(), B.begin());
```

```
auto diffs = thrust::make_transform_iterator(  
    AB.begin(),  
    [] __host__ __device__  
    (cuda::std::tuple<int, int> ab) {  
        auto [a, b] = ab;  
        return abs(a - b)  
    });
```

```
auto max_diff = thrust::reduce(  
    thrust::cuda::par,  
    diffs, diffs + A.size(),  
    0,  
    cuda::maximum{});
```

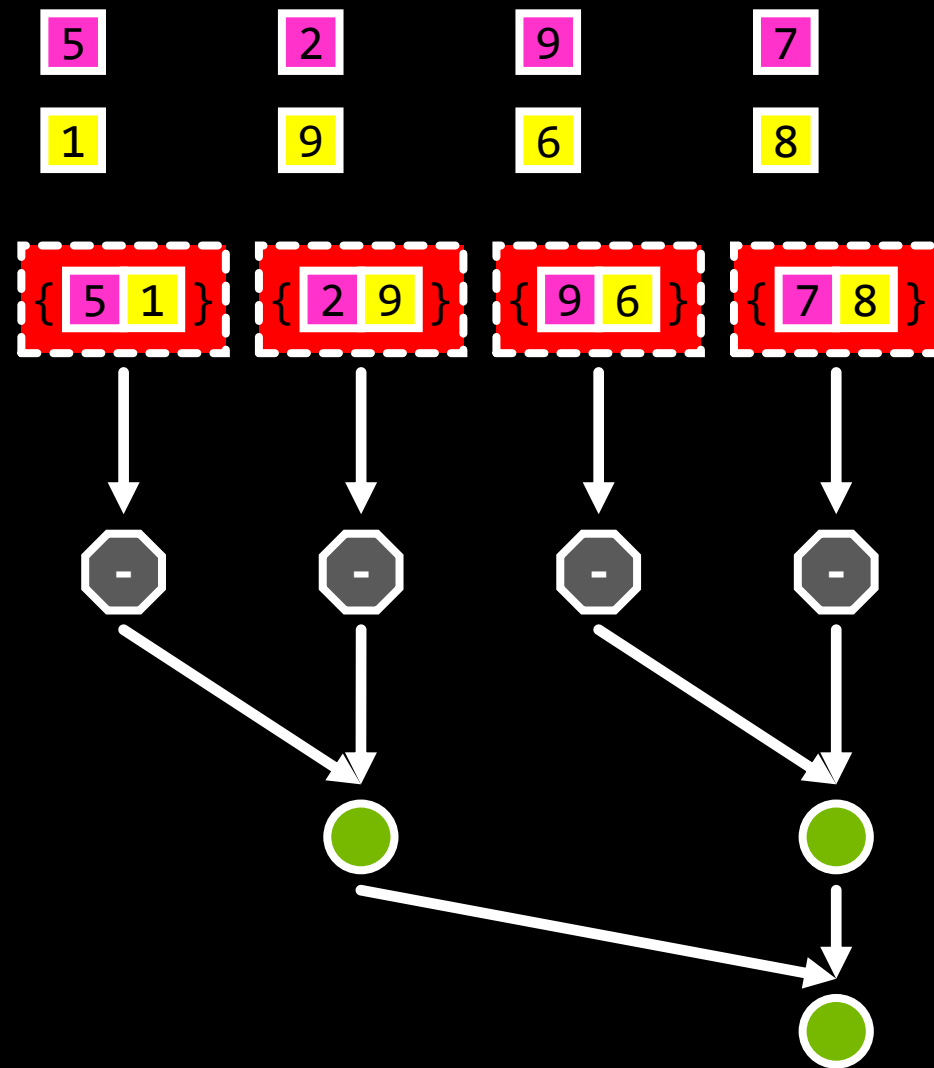


```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

```
auto AB = thrust::make_zip_iterator(  
    A.begin(), B.begin());
```

```
auto diffs = thrust::make_transform_iterator(  
    AB.begin(),  
    [] __host__ __device__  
    (cuda::std::tuple<int, int> ab) {  
        auto [a, b] = ab;  
        return abs(a - b)  
    });
```

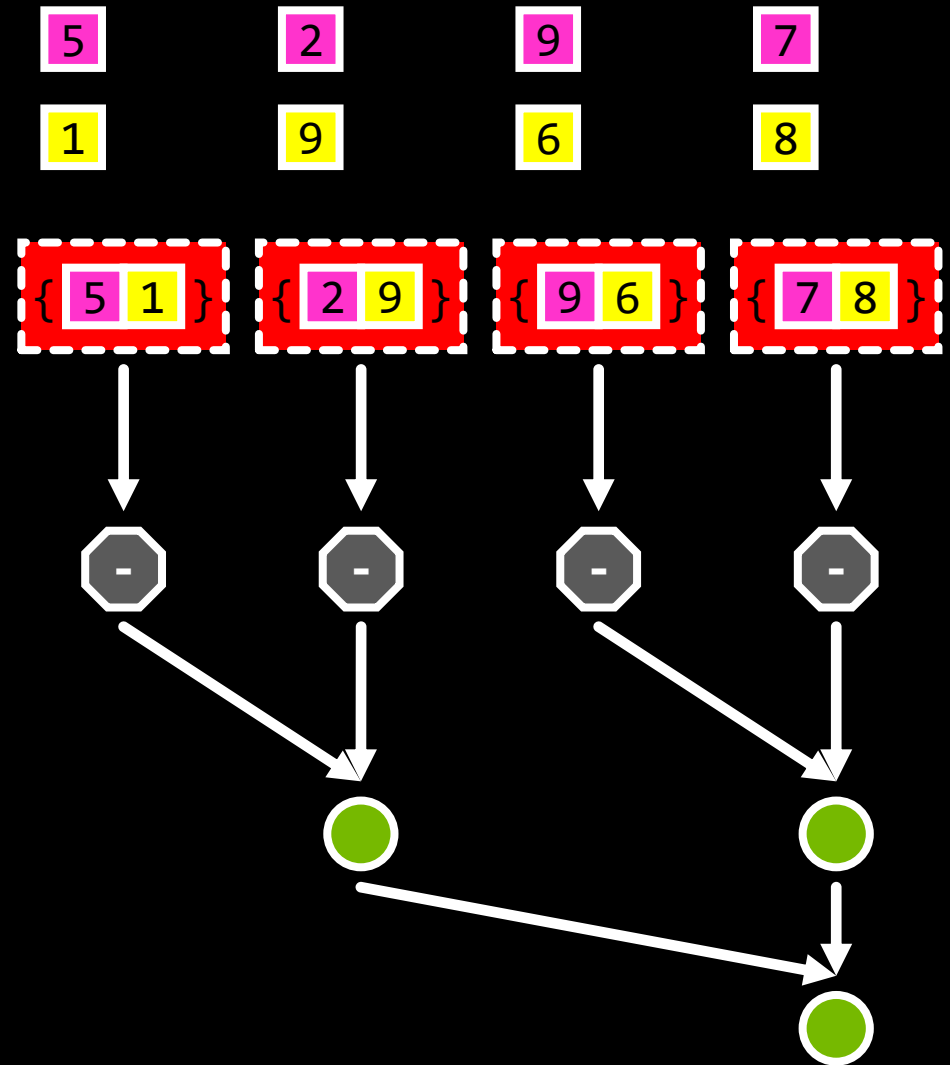
```
auto max_diff = thrust::reduce(  
    thrust::cuda::par,  
    diffs, diffs + A.size(),  
    0,  
    cuda::maximum{});
```



```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

```
auto AB = thrust::make_zip_iterator(  
    A.begin(), B.begin());
```

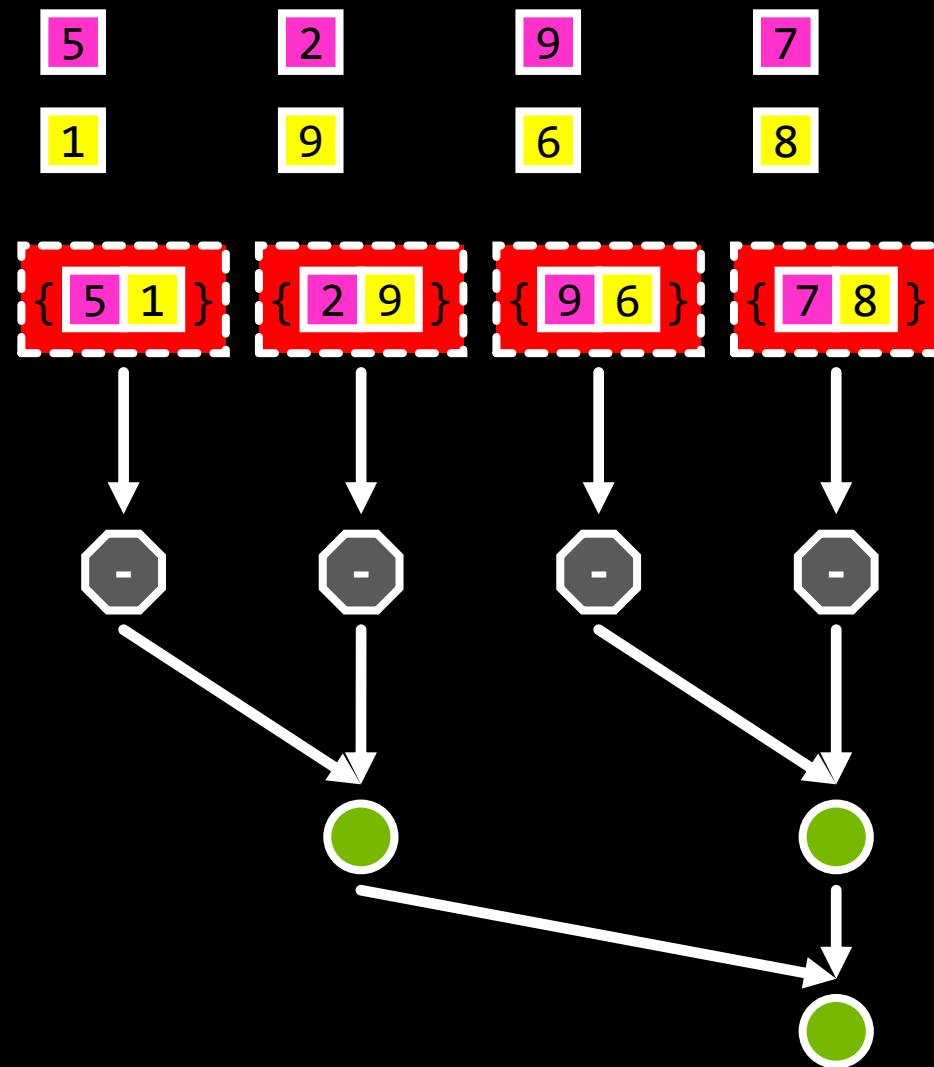
```
auto max_diff = thrust::transform_reduce(  
    thrust::cuda::par,  
    AB, AB + A.size(),  
    [] __host__ __device__  
    (cuda::std::tuple<int, int> ab) {  
        auto [a, b] = ab;  
        return abs(a - b)  
    },  
    0,  
    cuda::maximum{});
```

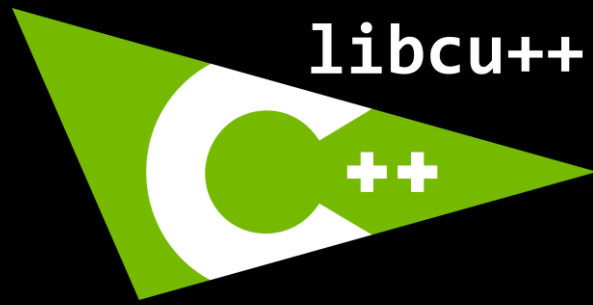


```
thrust::universal_vector<int> A(...);  
thrust::universal_vector<int> B(...);
```

```
auto AB = thrust::make_zip_iterator(  
    A.begin(), B.begin());
```

```
auto max_diff = thrust::transform_reduce(  
    thrust::cuda::par,  
    AB, AB + A.size(),  
    [] __host__ __device__  
    (cuda::std::tuple<int, int> ab) {  
        auto [a, b] = ab;  
        return abs(a - b)  
    },  
    0,  
    cuda::maximum{});
```



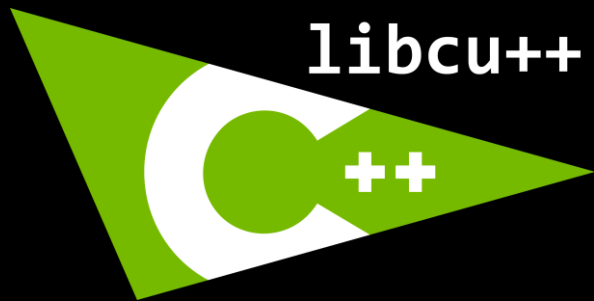


libcu++

The CUDA C++ Foundational Library

<https://nvidia.github.io/cccl/libcudacxx>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel



libcu++

The CUDA C++ Foundational Library

<https://nvidia.github.io/cccl/libcudacxx>

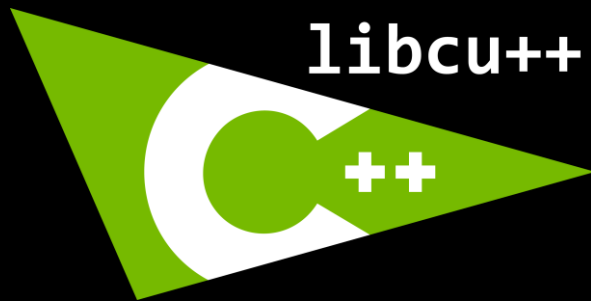
Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel

Host Compiler's Standard Library (GCC, LLVM, MSVC, etc)

```
#include <...>  
std::
```

Standard C++, `__host__` only.

Complete, strictly conforming to Standard C++.



The CUDA C++ Foundational Library

<https://nvidia.github.io/cccl/libcudacxx>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel

Host Compiler's Standard Library (GCC, LLVM, MSVC, etc)

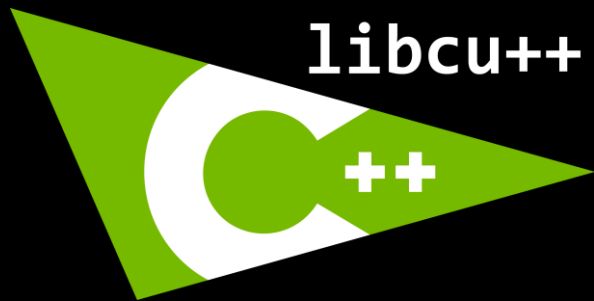
```
#include <...>  
std::
```

Standard C++, `__host__` only.
Complete, strictly conforming to Standard C++.

```
#include <cuda/std/...>  
cuda::std::
```

CUDA C++, `__host__` `__device__`.
Subset, strictly conforming to Standard C++.

libcu++



The CUDA C++ Foundational Library

<https://nvidia.github.io/cccl/libcudacxx>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel

Host Compiler's Standard Library (GCC, LLVM, MSVC, etc)

```
#include <...>
std::
```

Standard C++, `__host__` only.
Complete, strictly conforming to Standard C++.

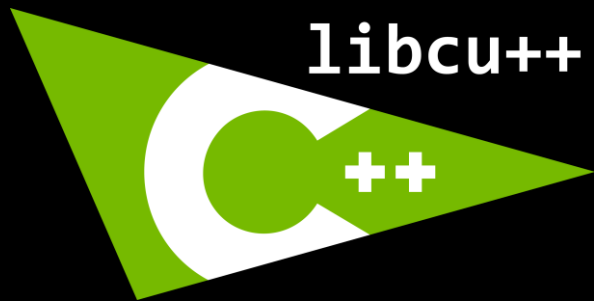
```
#include <cuda/std/...>
cuda::std::
```

CUDA C++, `__host__` `__device__`.
Subset, strictly conforming to Standard C++.

```
#include <cuda/...>
cuda::
```

CUDA C++, `__host__` and/or `__device__`.
Modern C++ API for CUDA & Standard C++ extensions.

libcu++



libcu++

The CUDA C++ Foundational Library

<https://nvidia.github.io/cccl/libcudacxx>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel

Host Compiler's Standard Library (GCC, LLVM, MSVC, etc)

```
#include <...>  
std::
```

Standard C++, `__host__` only.
Complete, strictly conforming to Standard C++.

```
#include <cuda/std/...>  
cuda::std::
```

CUDA C++, `__host__` `__device__`.
Subset, strictly conforming to Standard C++.

```
#include <cuda/...>  
cuda::
```

CUDA C++, `__host__` and/or `__device__`.
Modern C++ API for CUDA & Standard C++ extensions.

```
#include <cuda/experimental/...>  
cuda::experimental:: (cudax::)
```

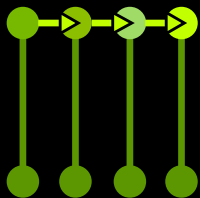
CUDA C++, `__host__` and/or `__device__`.
Beta features.

libcu++



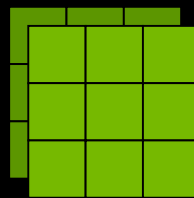
The C++ Parallel Algorithms Library

<https://nvidia.github.io/cccl/thrust>



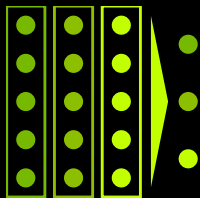
Standard Algorithms

- `thrust::transform_reduce`
- `thrust::inclusive_scan`
- `thrust::sort`
- `thrust::copy`
- ...



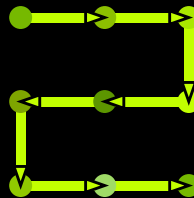
Containers

- `thrust::device_vector`
- `thrust::host_vector`
- `thrust::universal_vector`
- `thrust::allocate_unique`
- ...



Extended Algorithms

- `thrust::reduce_by_key`
- `thrust::sort_by_key`
- `thrust::tabulate`
- `thrust::gather`
- ...



Iterators

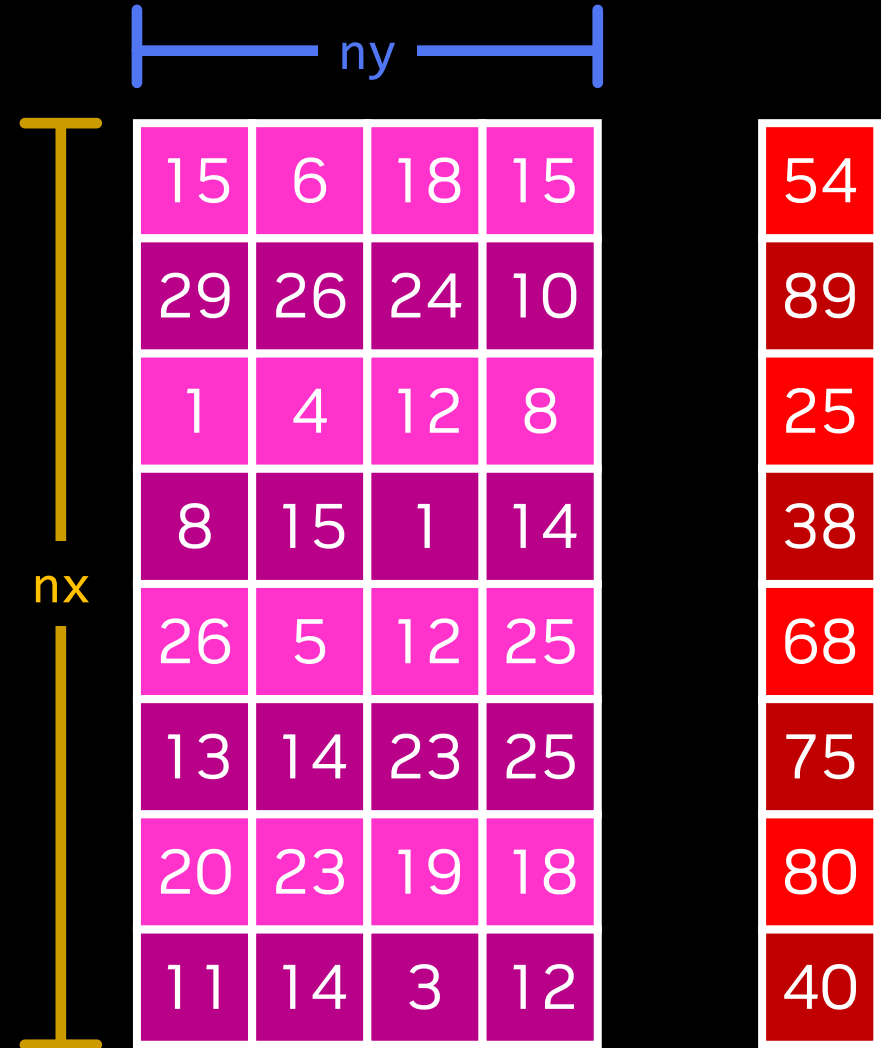
- `thrust::counting_iterator`
- `thrust::constant_iterator`
- `thrust::transform_iterator`
- `thrust::zip_iterator`
- ...

```
int nx(...), ny(...);  
thrust::universal_vector<float> M(nx * ny);
```

15	6	18	15	54
29	26	24	10	89
1	4	12	8	25
8	15	1	14	38
26	5	12	25	68
13	14	23	25	75
20	23	19	18	80
11	14	3	12	40

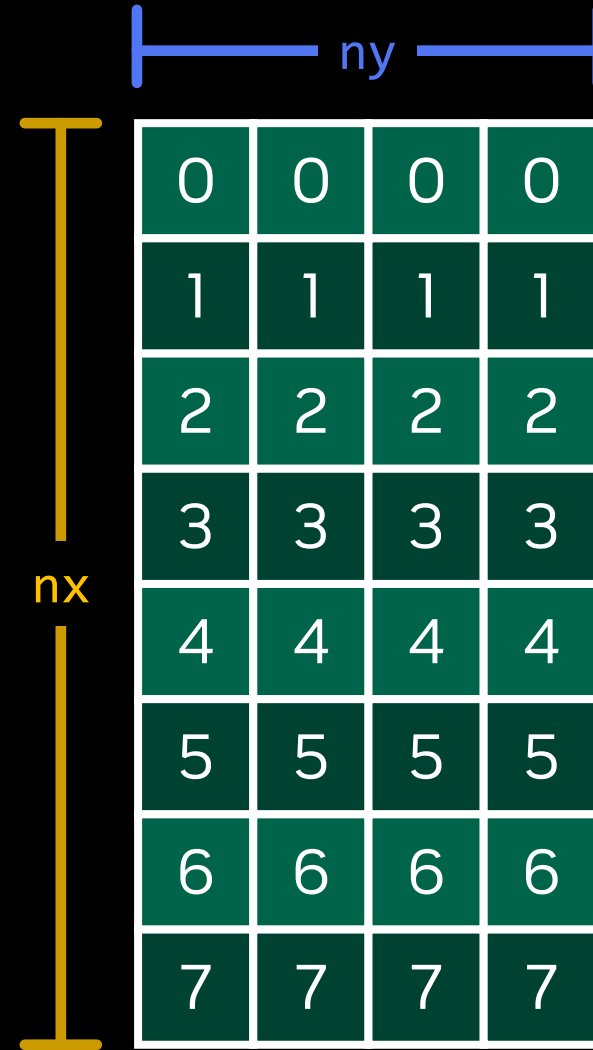
```
int nx(...), ny(...);  
thrust::universal_vector<float> M(nx * ny);
```

Each set of consecutive keys that compare equal defines a group.



```
int nx(...), ny(...);  
thrust::universal_vector<float> M(nx * ny);
```

Each set of consecutive keys that compare equal defines a group.



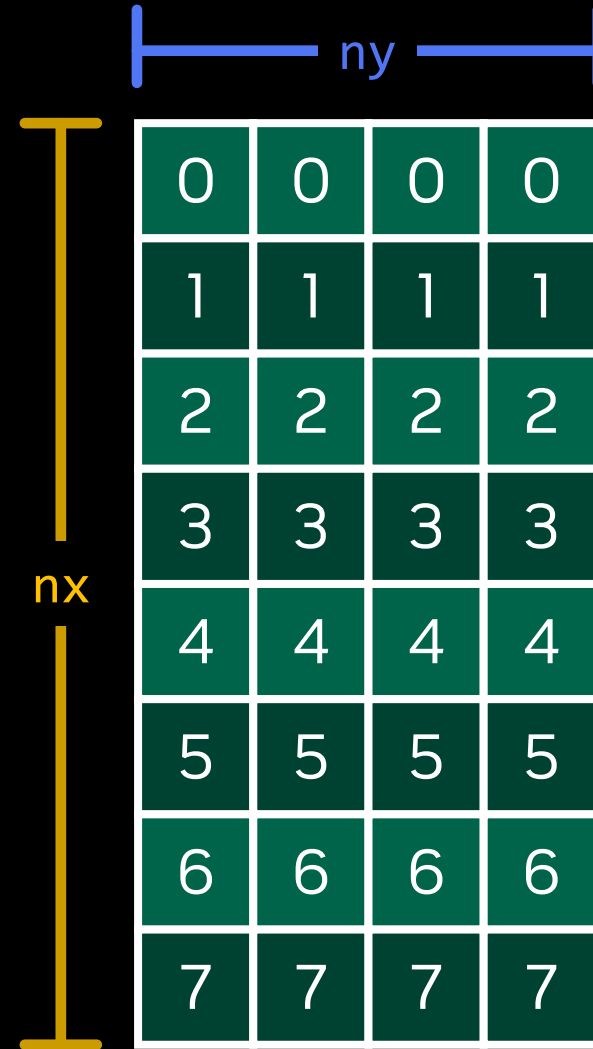
```

int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

```

Each set of consecutive keys that compare equal defines a group.



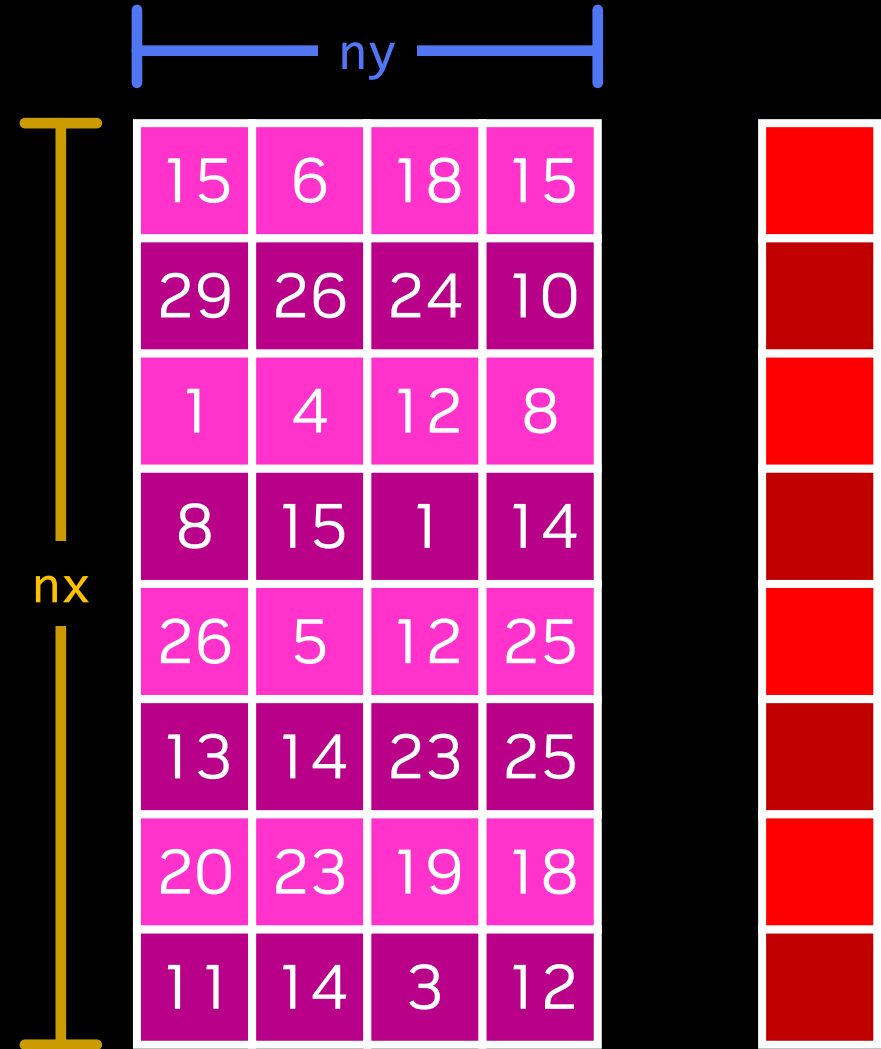

```

int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

```



```

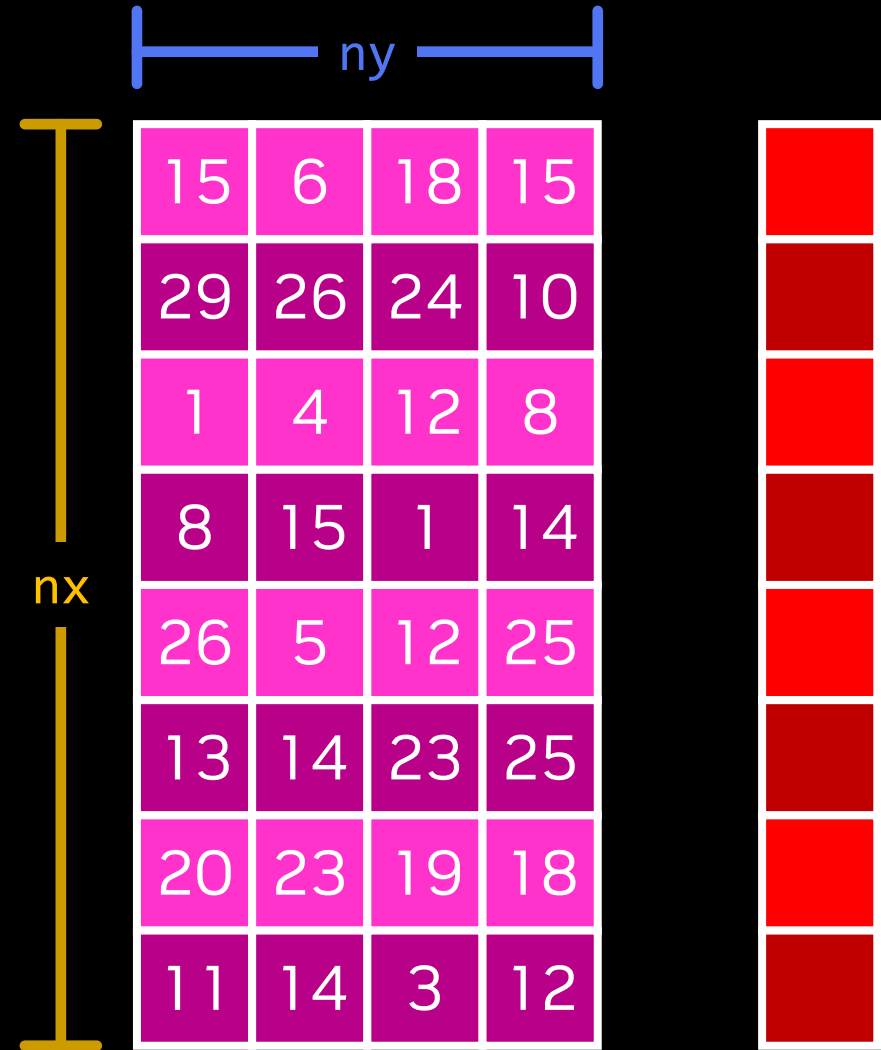
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

thrust::reduce_by_key(thrust::cuda::par,
    ...);

```



```

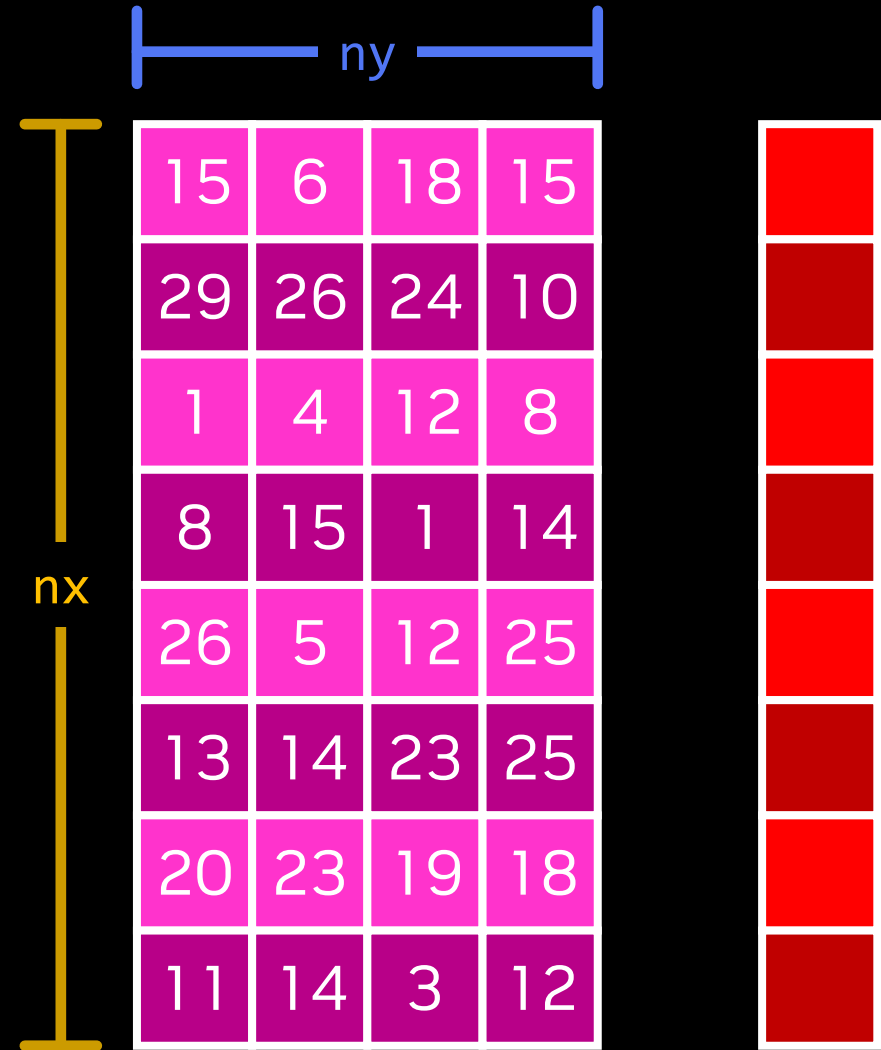
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

thrust::reduce_by_key(thrust::cuda::par,
    // Keys input.
    ...);

```



```

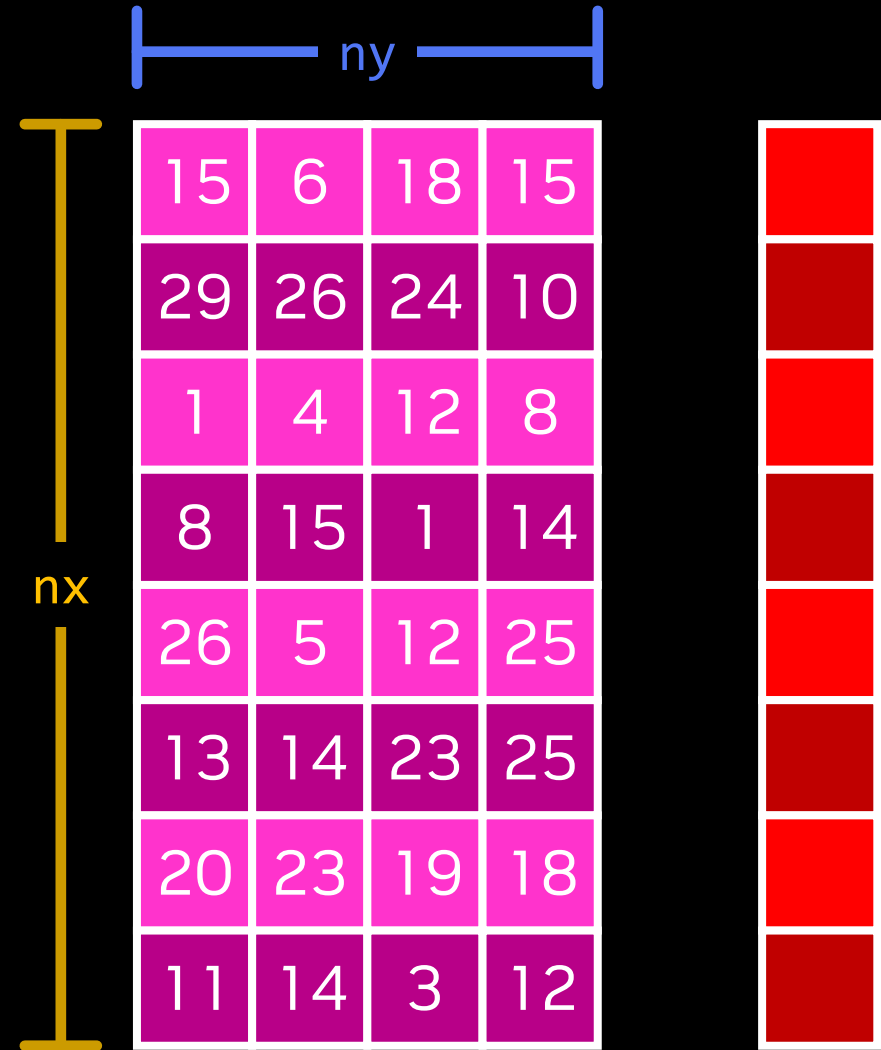
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    // Values input.
    ...);

```

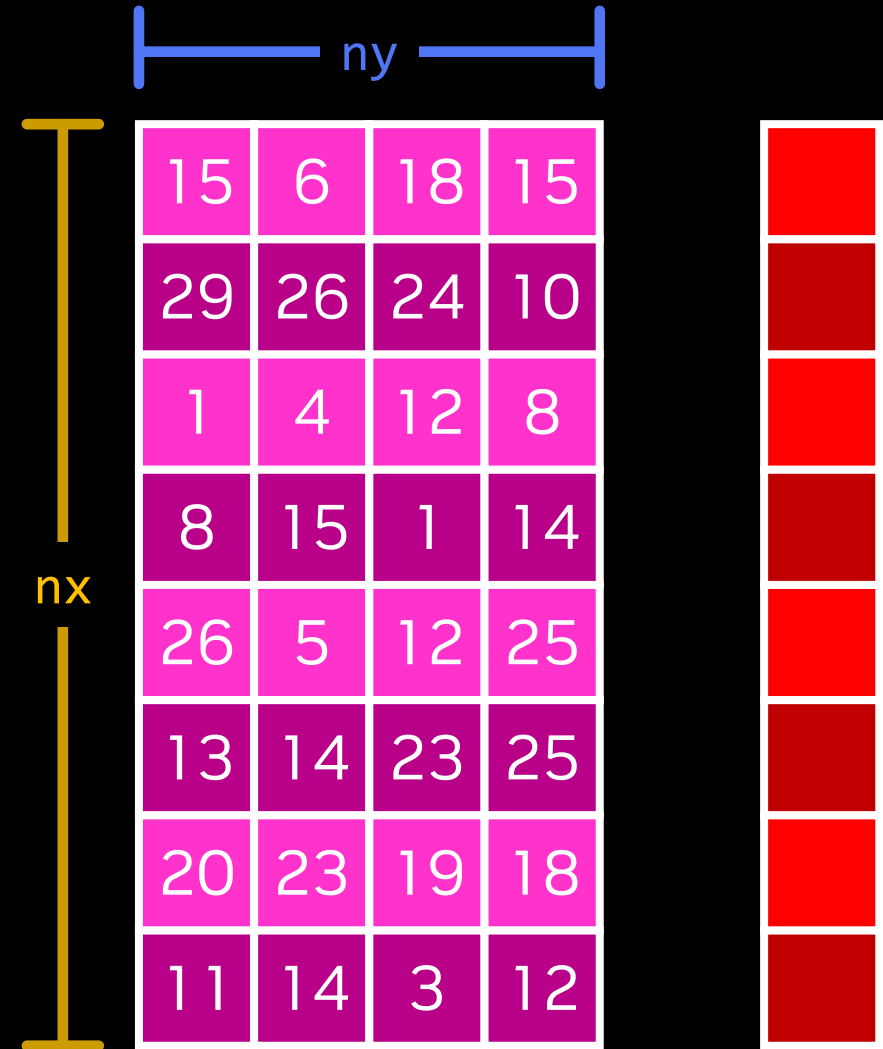


```
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);
```

```
auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
```

The underlying algorithm is applied to the values in each group.

```
thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    ...);
```



```

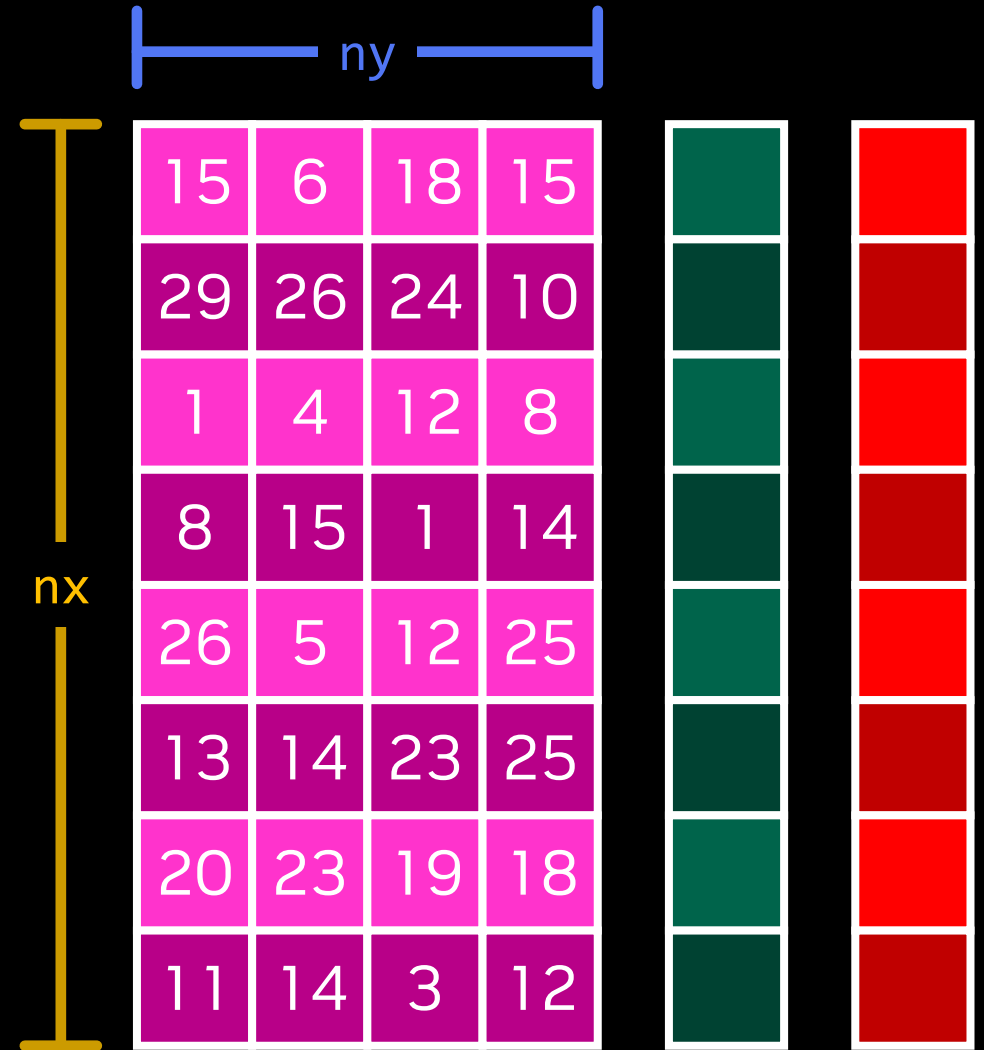
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    // Keys output.
    ...);

```



```

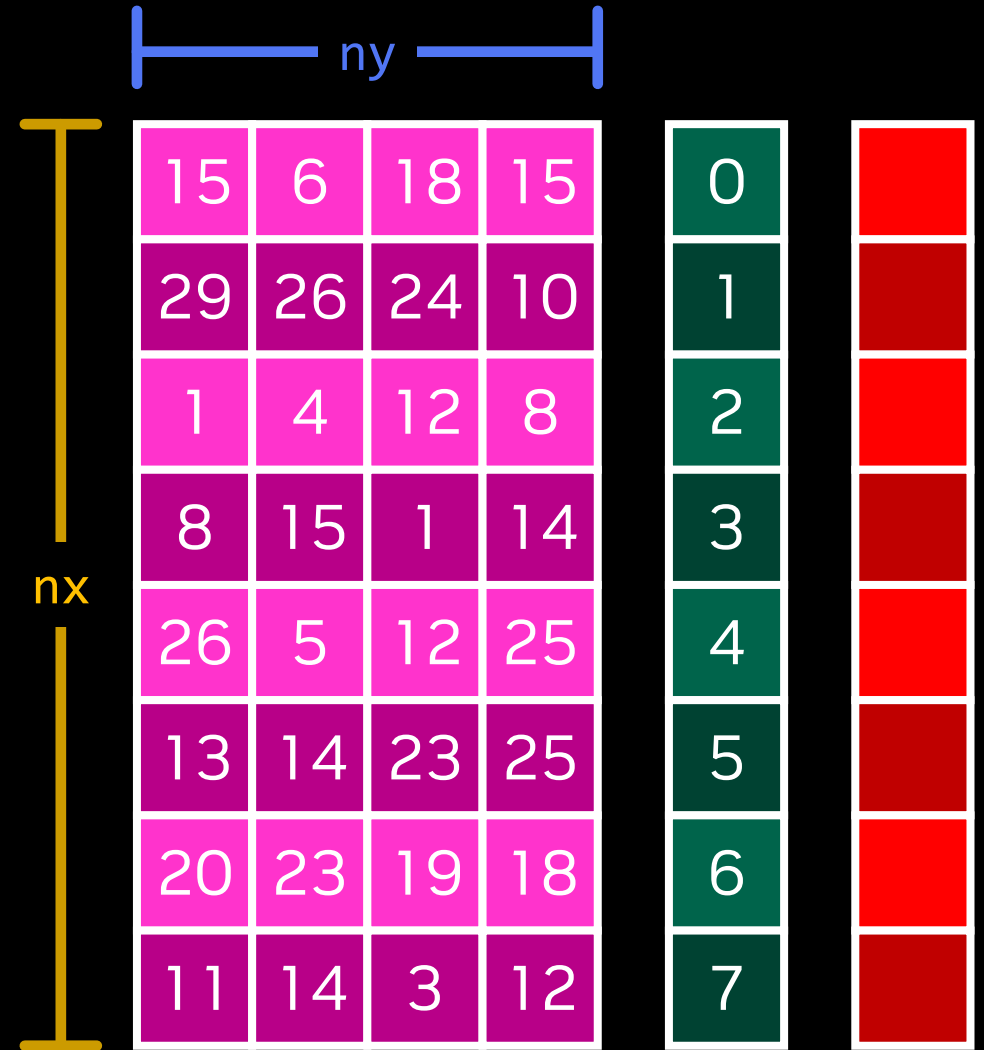
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx;

thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    // Keys output.
    ...);

```

The first key in each group is copied to the key output range.



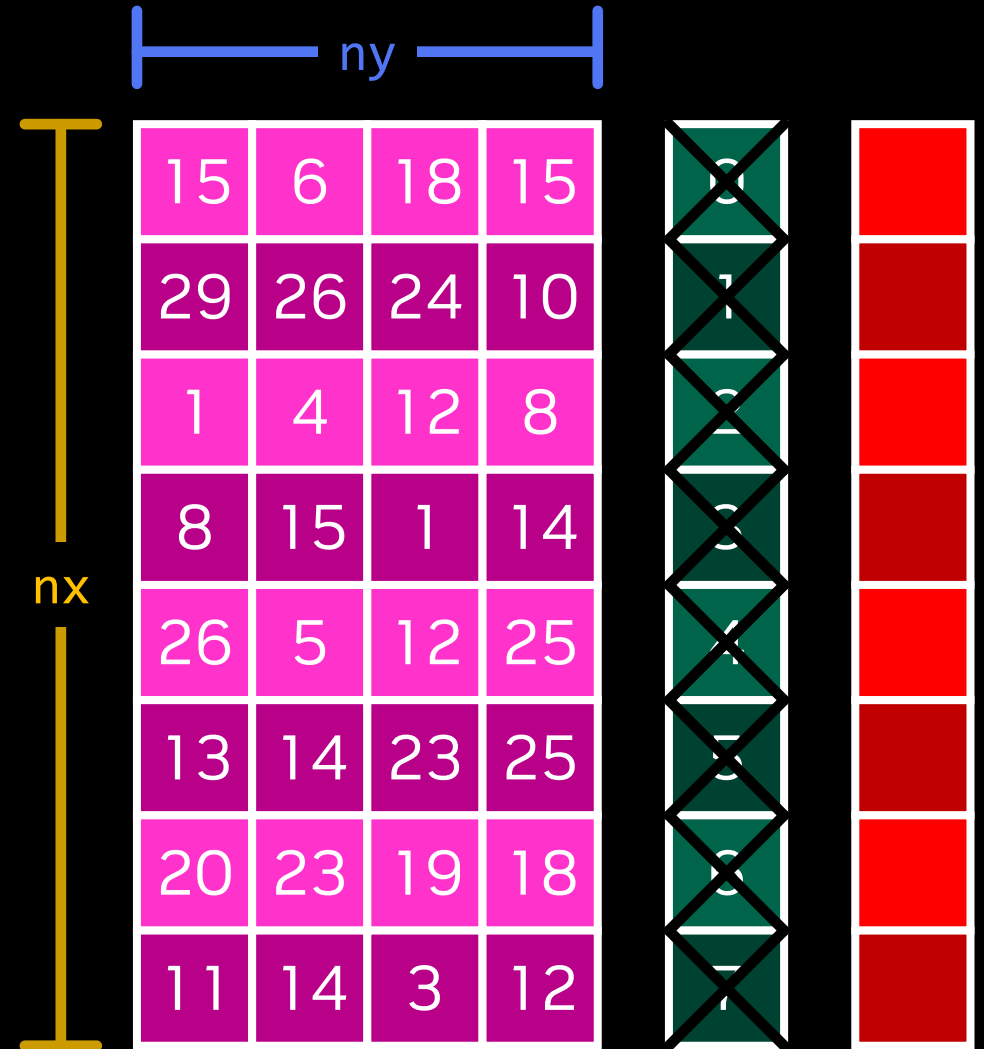
```
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);
```

```
auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
```

```
auto row_ids_end = row_ids_begin + nx;
thrust::reduce_by_key(row_ids_begin, row_ids_end,
```

The first key in each group is copied to the key output range.

```
thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    thrust::make_discard_iterator(),
    ...);
```




```

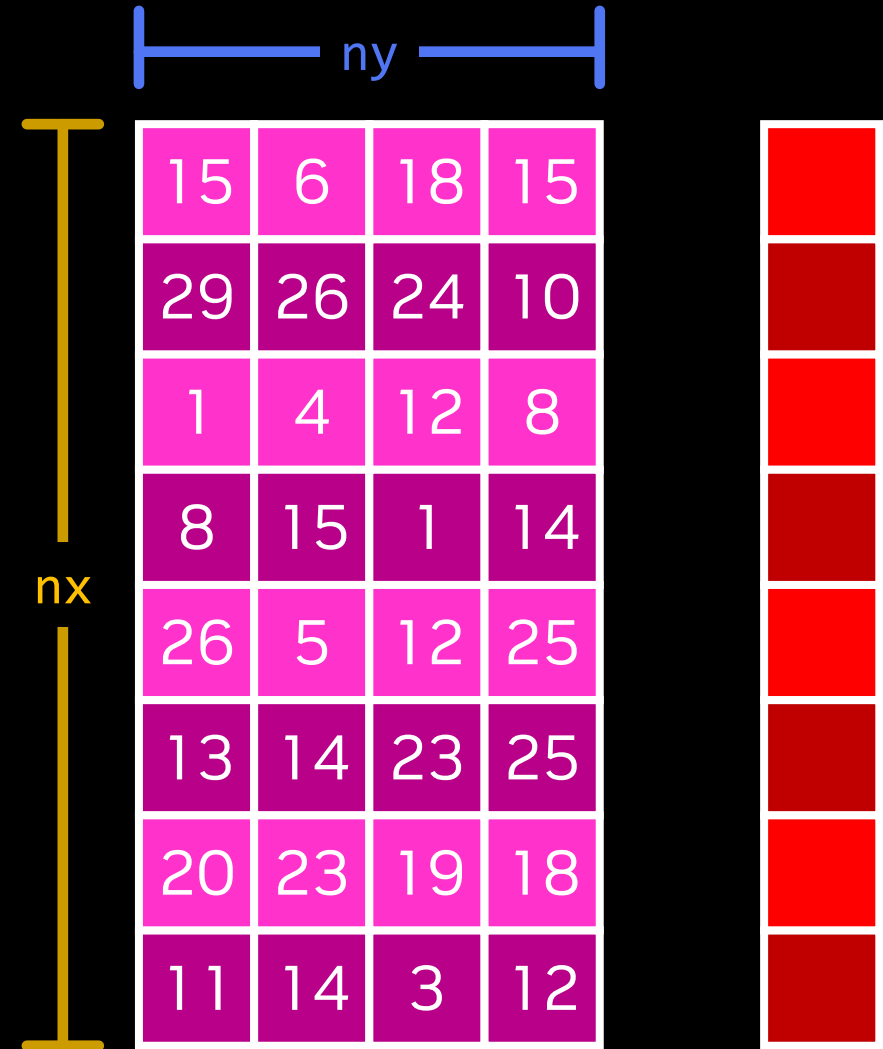
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    thrust::make_discard_iterator(),
    // Values output.);

```



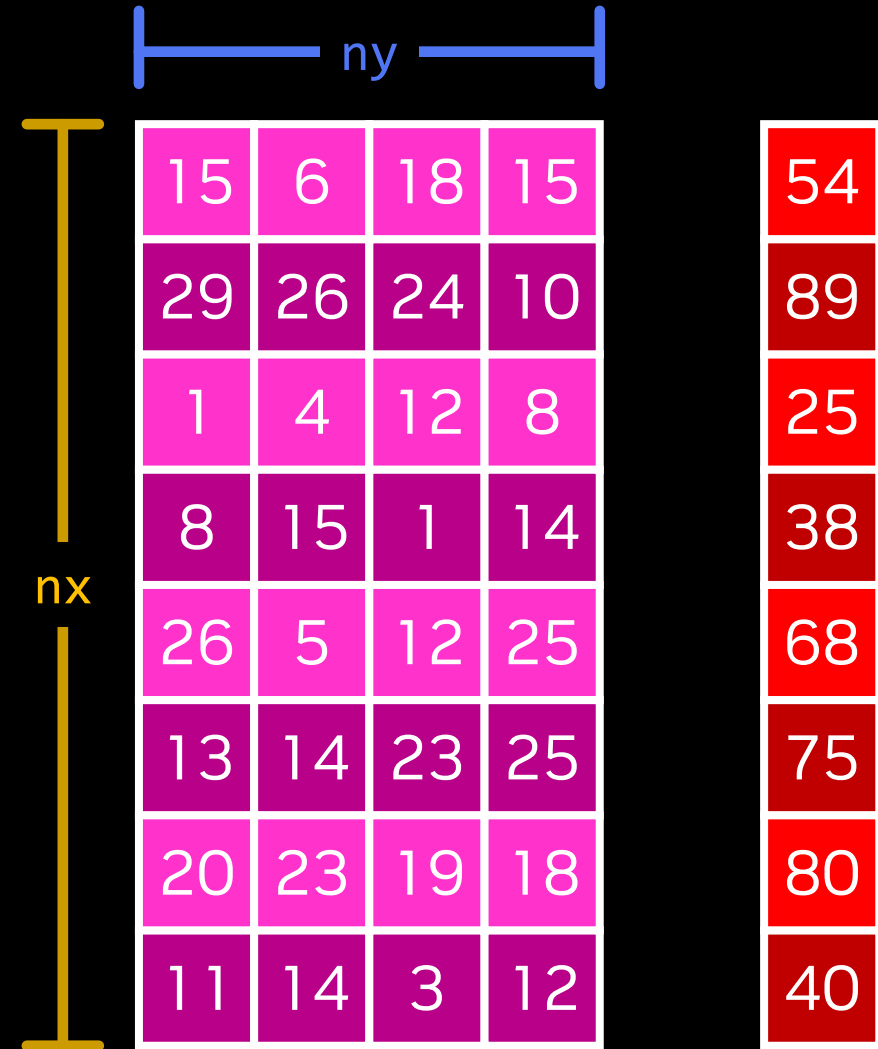
```
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);
```

```
auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
```

```
auto thrust
```

The algorithm result for each group
is written to the values output.

```
thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    thrust::make_discard_iterator(),
    sums.begin());
```



```

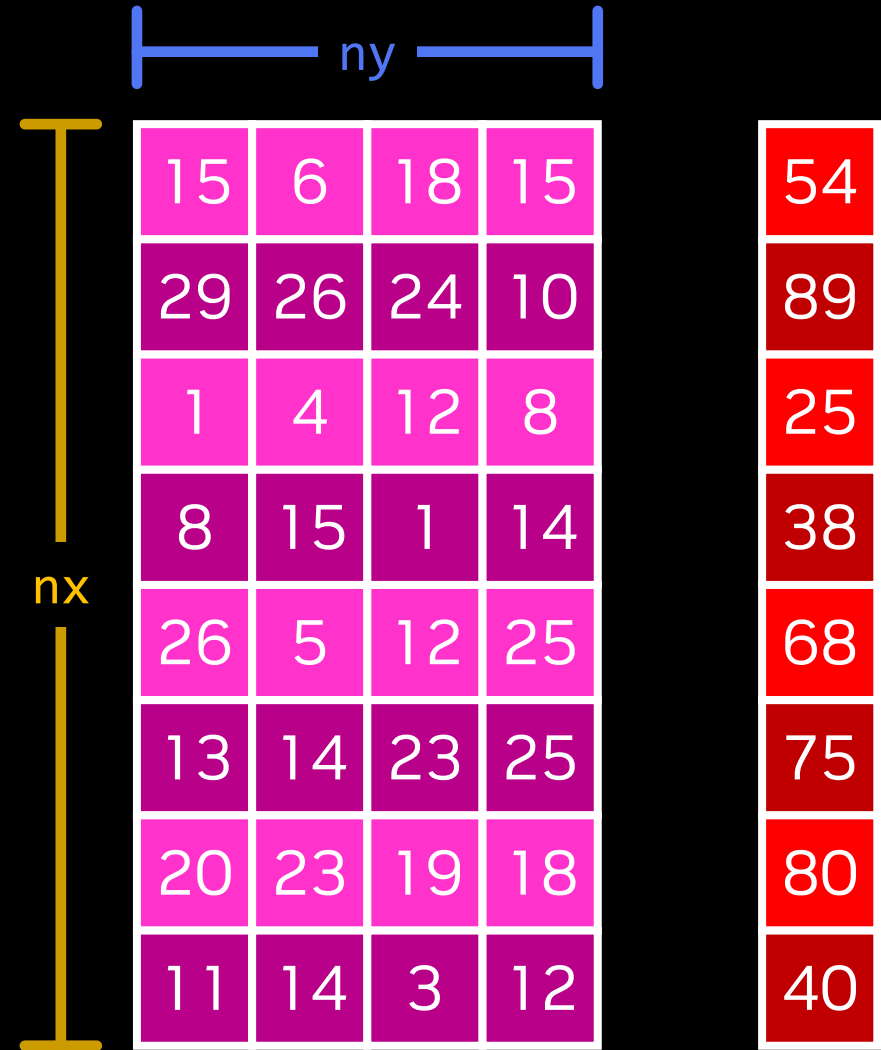
int nx(...), ny(...);
thrust::universal_vector<float> M(nx * ny);

auto row_ids_begin =
    thrust::make_transform_iterator(
        thrust::make_counting_iterator(0),
        [=] __host__ __device__ (int x) {
            return x / ny;
        });
auto row_ids_end = row_ids_begin + nx * ny;

thrust::universal_vector<float> sums(nx);

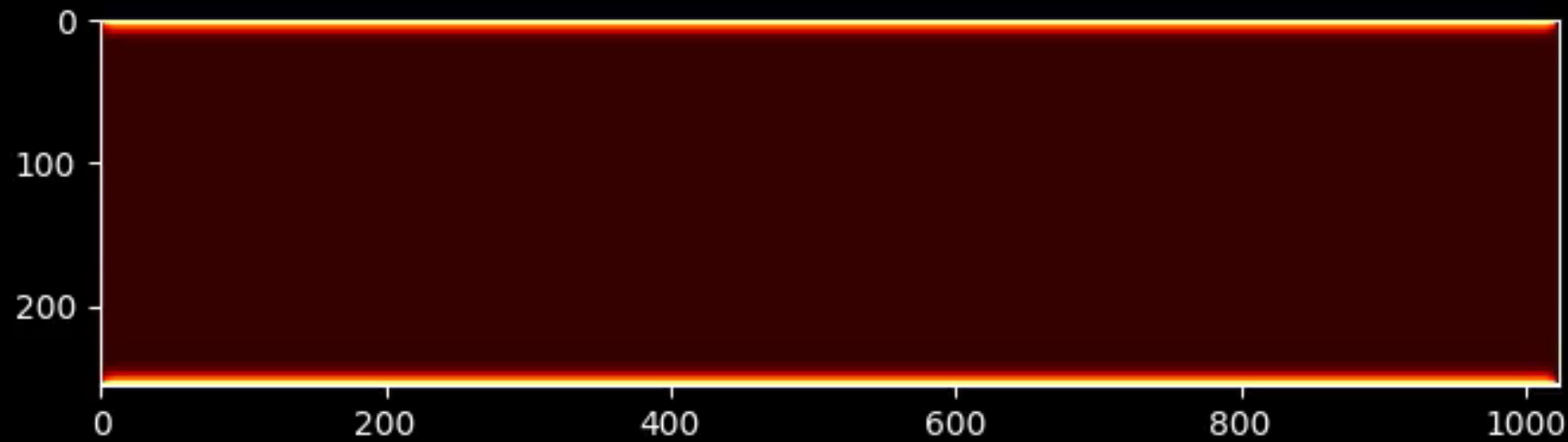
thrust::reduce_by_key(thrust::cuda::par,
    row_ids_begin, row_ids_end,
    M.begin(),
    thrust::make_discard_iterator(),
    sums.begin());

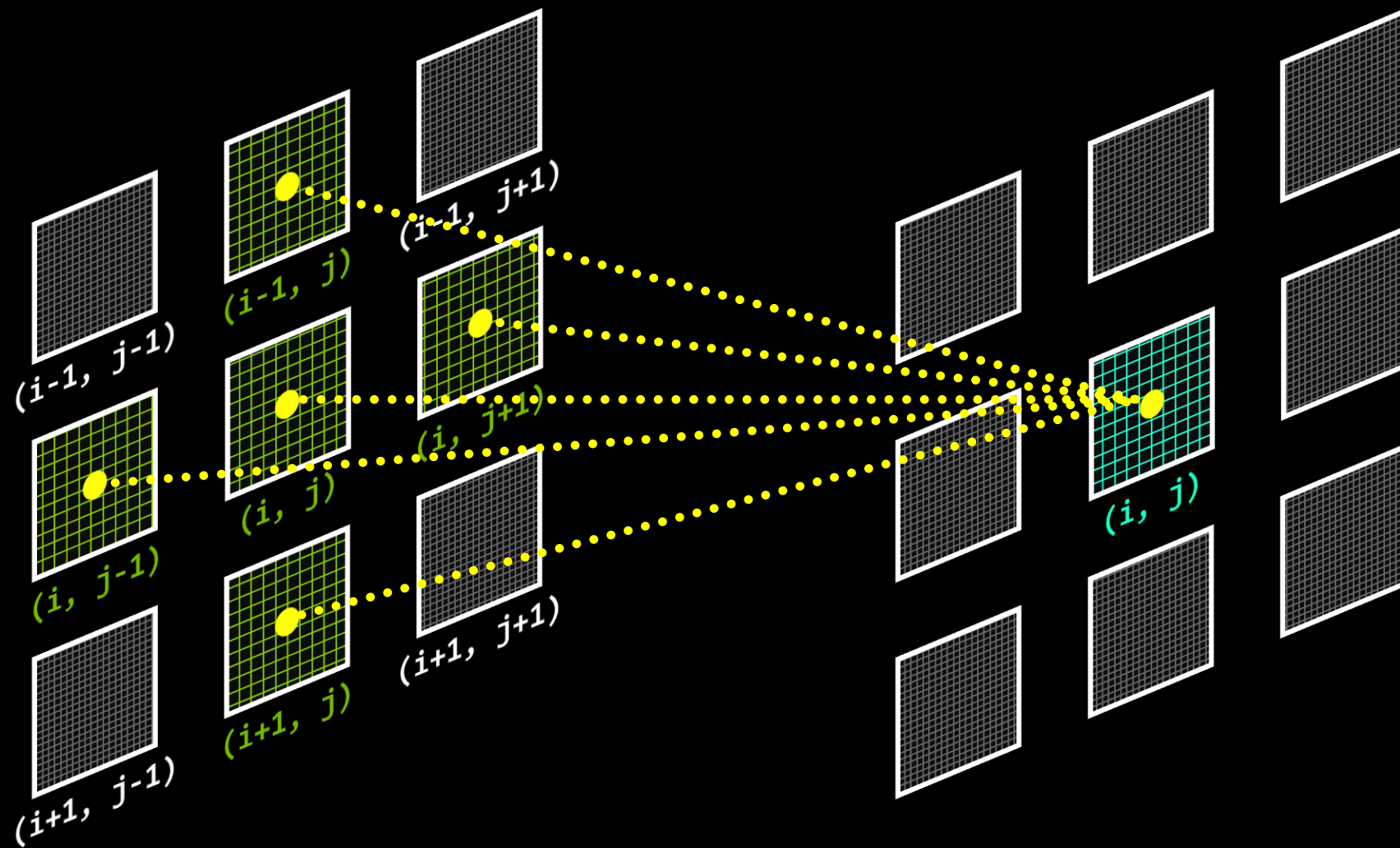
```



By Key Algorithms

- `reduce_by_key`
- `(inclusive|exclusive)_scan_by_key`
- `sort_by_key`
- `unique_by_key`
- `merge_by_key`
- `set_(union|difference|intersection)_by_key`





```
void heat_equation(auto policy, int nx, int ny,  
                  const thrust::universal_vector<float>& U_data,  
                  thrust::universal_vector<float>& V_data);
```

```
void heat_equation(auto policy, int nx, int ny,  
                  const thrust::universal_vector<float>& U_data,  
                  thrust::universal_vector<float>& V_data) {  
    ...  
  
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(), ...);  
}
```



```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    ...

    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [?, nx, ny] __host__ __device__ (int xy) {
        ...
    });
}

```

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
        ...
    });
}

```

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
            int x = xy / ny;
            int y = xy % ny;

            ...
        });
}

```

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
            int x = xy / ny;
            int y = xy % ny;

            if (x > 0 && y > 0 && x < nx - 1 && y < ny - 1) {
                ...
            } else ...
        });
}

```

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
            int x = xy / ny;
            int y = xy % ny;

            if (x > 0 && y > 0 && x < nx - 1 && y < ny - 1) {
                auto d2tdx2 = U[x*ny + y - 1] - U[x*ny + y] * 2 + U[x*ny + y + 1];
                auto d2tdy2 = U[(x - 1)*ny + y] - U[x*ny + y] * 2 + U[(x + 1)*ny + y];

                V[x*ny + y] = U[x*ny + y] + 0.2f * (d2tdx2 + d2tdy2);
            } else ...
        });
}

```

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
            int x = xy / ny;
            int y = xy % ny;

            if (x > 0 && y > 0 && x < nx - 1 && y < ny - 1) {
                auto d2tdx2 = U[x*ny + y - 1] - U[x*ny + y] * 2 + U[x*ny + y + 1];
                auto d2tdy2 = U[(x - 1)*ny + y] - U[x*ny + y] * 2 + U[(x + 1)*ny + y];

                V[x*ny + y] = U[x*ny + y] + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                V[x*ny + y] = U[x*ny + y];
            }
        });
}

```

mdspan: A non-owning handle to multidimensional data

```
template <class T, class Extents, class LayoutPolicy = ..., class AccessorPolicy = ...>
class std::mdspan;
```

```
mdspan A(data, N, M); // Row-major: C/NumPy.
mdspan A(data, layout_right::mapping(dims(N, M)));
```

```
A(i, j) == data(i * M + j)
A.stride(0) == M
A.stride(1) == 1
```

Location	Element
0	a_{00}
1	a_{01}
2	a_{10}
3	a_{11}

$$\begin{bmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{bmatrix}$$

```
// Column-major: Fortran/MATLAB.
mdspan B(data, layout_left::mapping(dims(N, M)));
```

```
B(i, j) == data(i + j * N)
B.stride(0) == 1
B.stride(1) == N
```

Location	Element
0	a_{00}
1	a_{10}
2	a_{01}
3	a_{11}

Learn More: [S51755](#) C++ Standard Parallelism (2023)

```

void heat_equation(auto policy, int nx, int ny,
                  const thrust::universal_vector<float>& U_data,
                  thrust::universal_vector<float>& V_data) {
    const float* U = thrust::raw_pointer_cast(U_data.data());
    const float* V = thrust::raw_pointer_cast(V_data.data());
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V, nx, ny] __host__ __device__ (int xy) {
            int x = xy / ny;
            int y = xy % ny;

            if (x > 0 && y > 0 && x < nx - 1 && y < ny - 1) {
                auto d2tdx2 = U[x*ny + y - 1] - U[x*ny + y] * 2 + U[x*ny + y + 1];
                auto d2tdy2 = U[(x - 1)*ny + y] - U[x*ny + y] * 2 + U[(x + 1)*ny + y];

                V[x*ny + y] = U[x*ny + y] + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                V[x*ny + y] = U[x*ny + y];
            }
        });
}

```



```

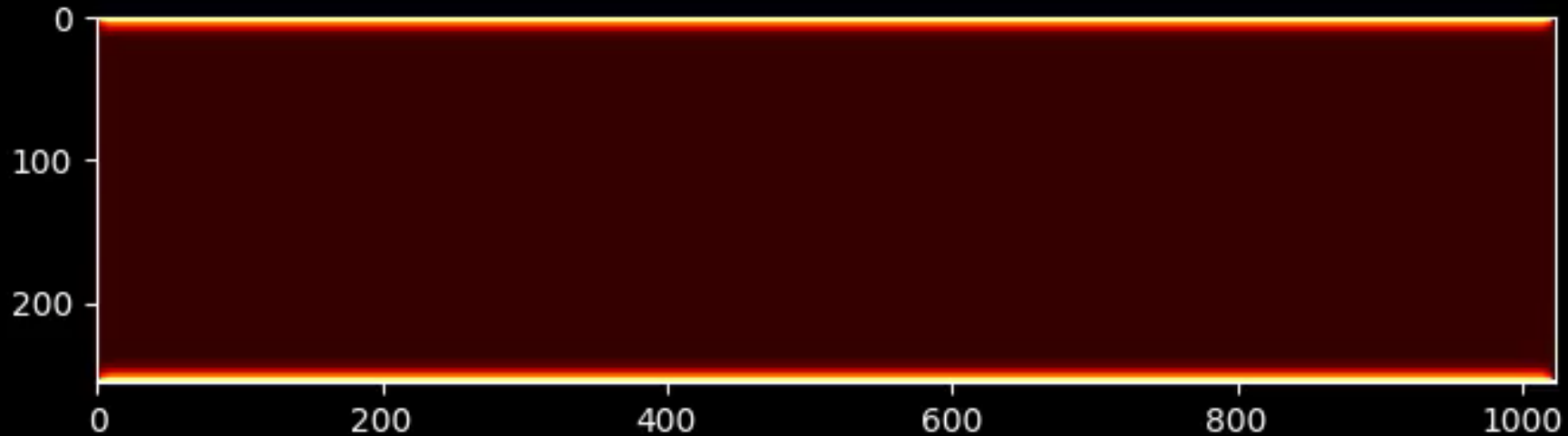
void heat_equation(auto policy,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V] __host__ __device__ (int xy) {
            int x = xy / U.extent(1);
            int y = xy % U.extent(1);

            if (x > 0 && y > 0 && x < U.extent(0) - 1 && y < U.extent(1) - 1) {
                auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
                auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

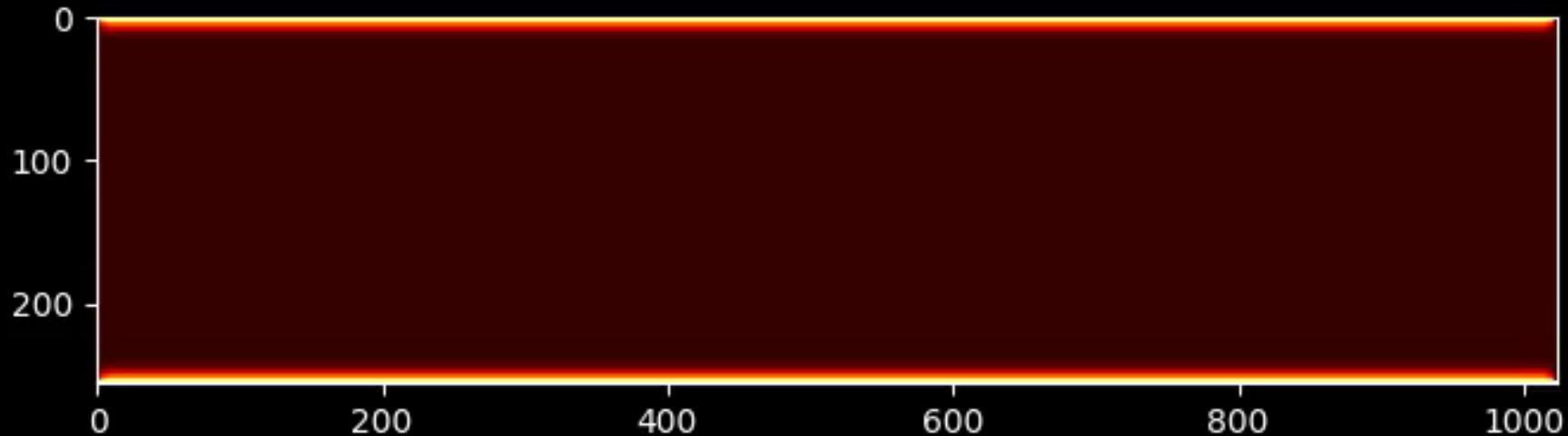
                V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                V(x, y) = U(x, y);
            }
        });
}

```

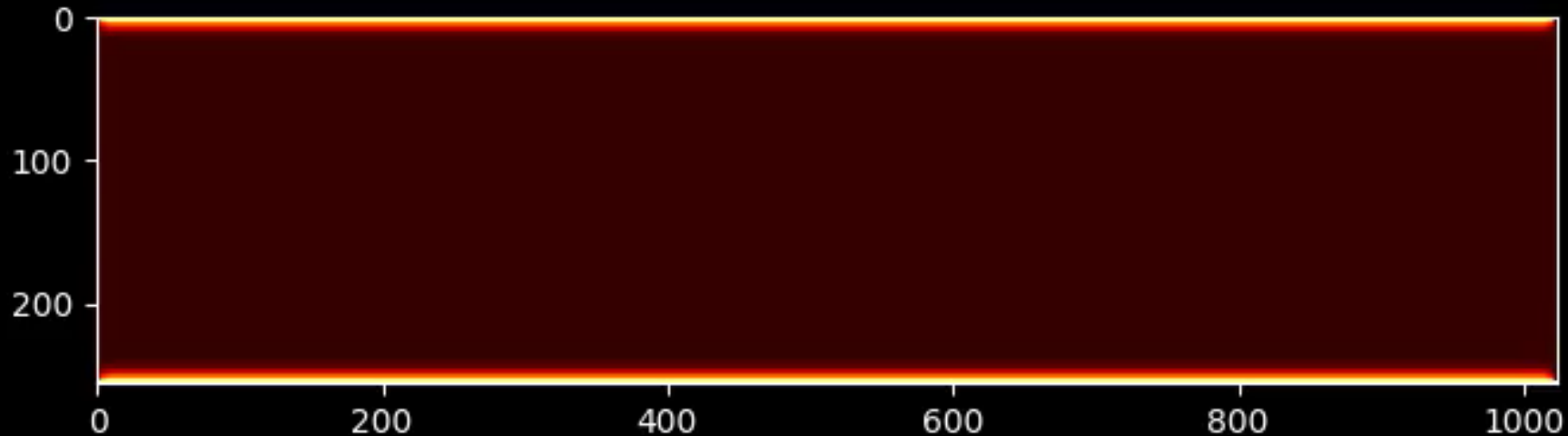
```
void initialize_oven(auto policy, cuda::std::mdspan<float, cuda::std::dims<2>> U);
```



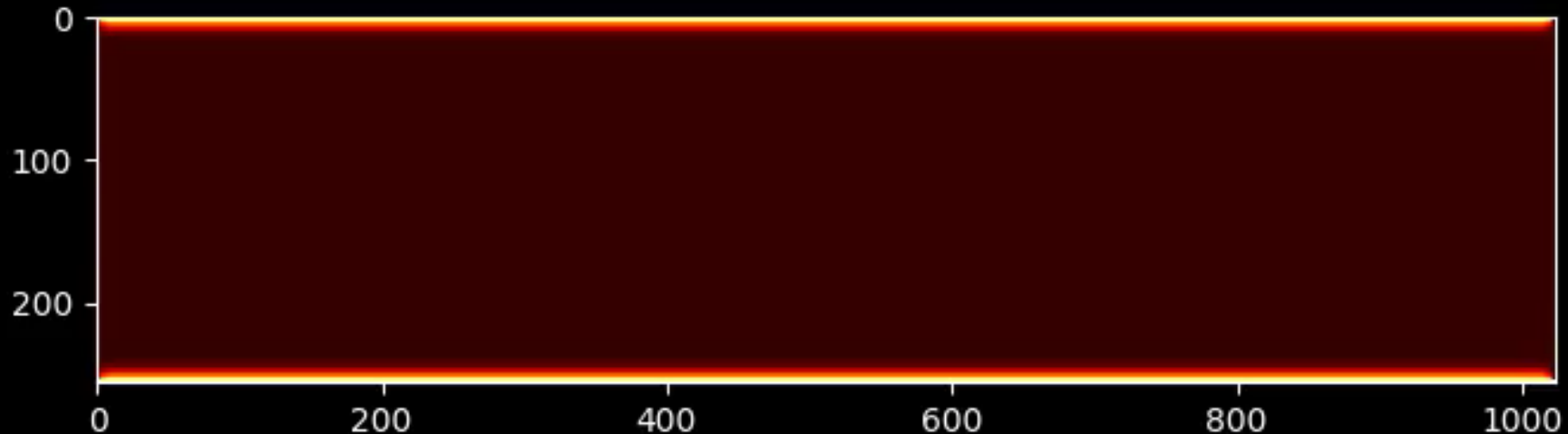
```
void initialize_oven(auto policy, cuda::std::mdspan<float, cuda::std::dims<2>> U) {  
    auto nx = U.extent(0);  
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);  
  
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);  
}
```



```
void initialize_oven(auto policy, cuda::std::mdspan<float, cuda::std::dims<2>> U) {  
    auto nx = U.extent(0);  
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);  
    auto mid = cuda::std::submdspan(U, cuda::std::tuple(1, nx-2), cuda::std::full_extent);  
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);  
}
```



```
void initialize_oven(auto policy, cuda::std::mdspan<float, cuda::std::dims<2>> U) {  
    auto nx = U.extent(0);  
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);  
    auto mid = cuda::std::submdspan(U, cuda::std::tuple(1, nx-2), cuda::std::full_extent);  
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);  
  
    thrust::fill_n(policy, top.data_handle(), top.size(), 90.0);  
  
    thrust::fill_n(policy, bot.data_handle(), bot.size(), 90.0);  
}
```

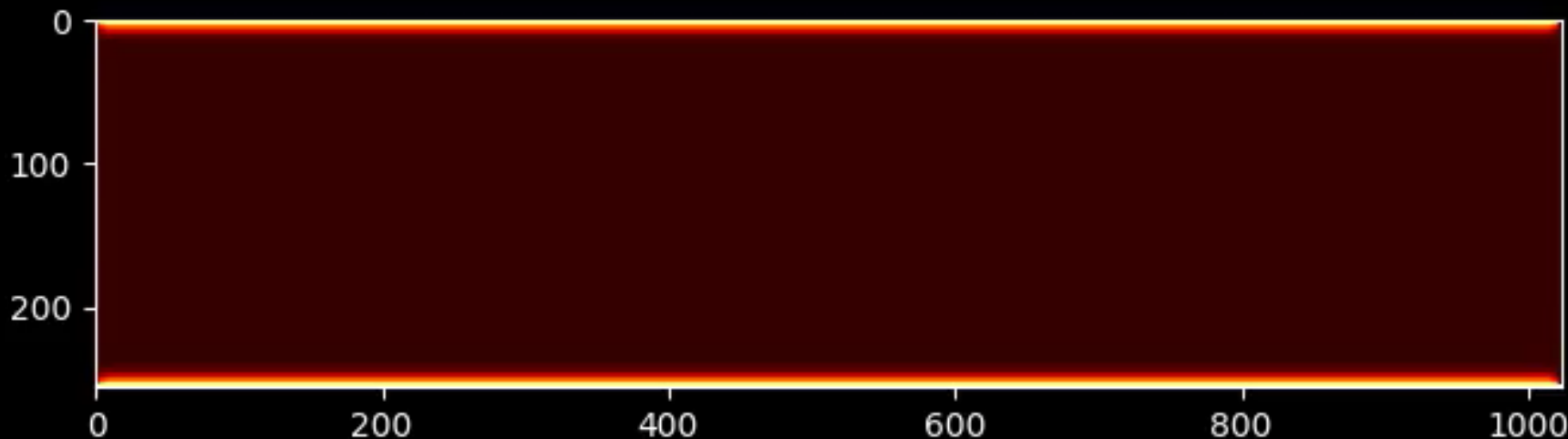


```

void initialize_oven(auto policy, cuda::std::mdspan<float, cuda::std::dims<2>> U) {
    auto nx = U.extent(0);
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);
    auto mid = cuda::std::submdspan(U, cuda::std::tuple(1, nx-2), cuda::std::full_extent);
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);

    thrust::fill_n(policy, top.data_handle(), top.size(), 90.0);
    thrust::fill_n(policy, mid.data_handle(), mid.size(), 15.0);
    thrust::fill_n(policy, bot.data_handle(), bot.size(), 90.0);
}

```



```
int nx(...), ny(...), write_steps(...), compute_steps(...);
```

```
int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);
```



```
int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);
```

```
int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    ...
}
```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        ...
    }
}

```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par, U, V);
        ...
    }
}

```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par, U, V);
        U.swap(V);
    }
}

```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par, U, V);
        U.swap(V);
    }
}

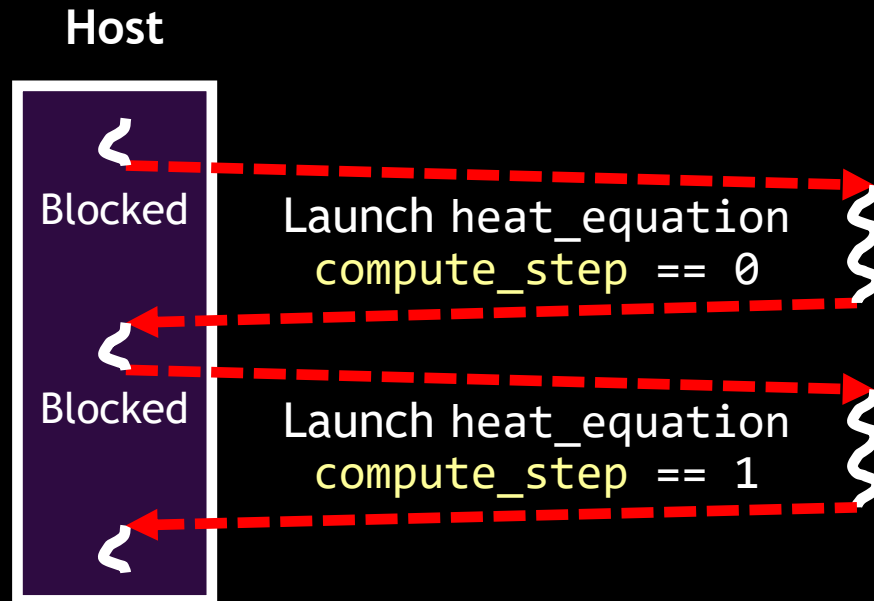
```

```

heat_equation(..., U_data.data(), V_data.data());
heat_equation(..., V_data.data(), U_data.data());
heat_equation(..., U_data.data(), V_data.data());

```

```
compute_step = 0;  
heat_equation(thrust::cuda::par, U, V);  
U.swap(V);  
compute_step = 1;  
heat_equation(thrust::cuda::par, U, V);  
U.swap(V);
```



```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par, U, V);
        U.swap(V);
    }
}

```



```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

initialize_oven(thrust::cuda::par_nosync, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par_nosync, U, V);
        U.swap(V);
    }
}

```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(thrust::raw_pointer_cast(U_data.data()), nx, ny);
cuda::std::mdspan V(thrust::raw_pointer_cast(V_data.data()), nx, ny);

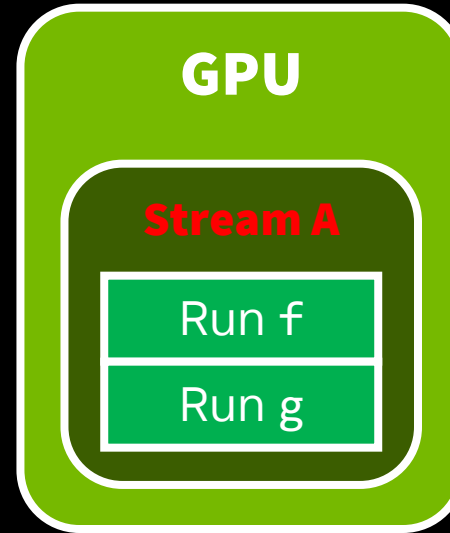
initialize_oven(thrust::cuda::par_nosync, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    cudaDeviceSynchronize();
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par_nosync, U, V);
        U.swap(V);
    }
}

```

```
auto pol(thrust::cuda::par_nosync);  
  
cuda::stream A;  
  
thrust::for_each(pol.on(A), ..., f);  
thrust::for_each(pol.on(A), ..., g);
```



A stream is a chain of device work that is executed in order.

```
auto pol(thrust::cuda::par_nosync);  
  
cudax::stream A;  
  
thrust::for_each(pol.on(A), ..., f);  
thrust::for_each(pol.on(A), ..., g);  
A.sync();
```

GPU

A stream is a chain of device work that is executed in order.

They can record events that host code can sync with.

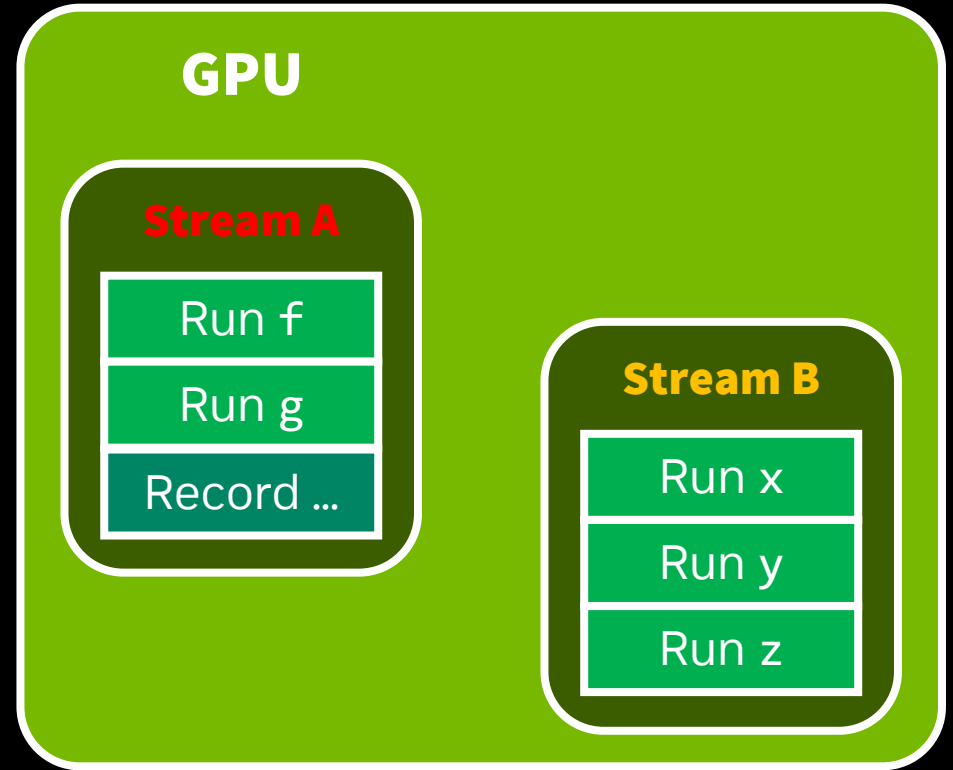
```
auto pol(thrust::cuda::par_nosync);
```

```
cuda::stream A;
```

```
thrust::for_each(pol.on(A), ..., f);  
thrust::for_each(pol.on(A), ..., g);  
A.sync();
```

```
cuda::stream B;
```

```
thrust::for_each(pol.on(B), ..., x);  
thrust::for_each(pol.on(B), ..., y);  
thrust::for_each(pol.on(B), ..., z);
```



```

auto pol(thrust::cuda::par_nosync);

cuda::stream A;

thrust::for_each(pol.on(A), ..., f);
thrust::for_each(pol.on(A), ..., g);
A.sync();

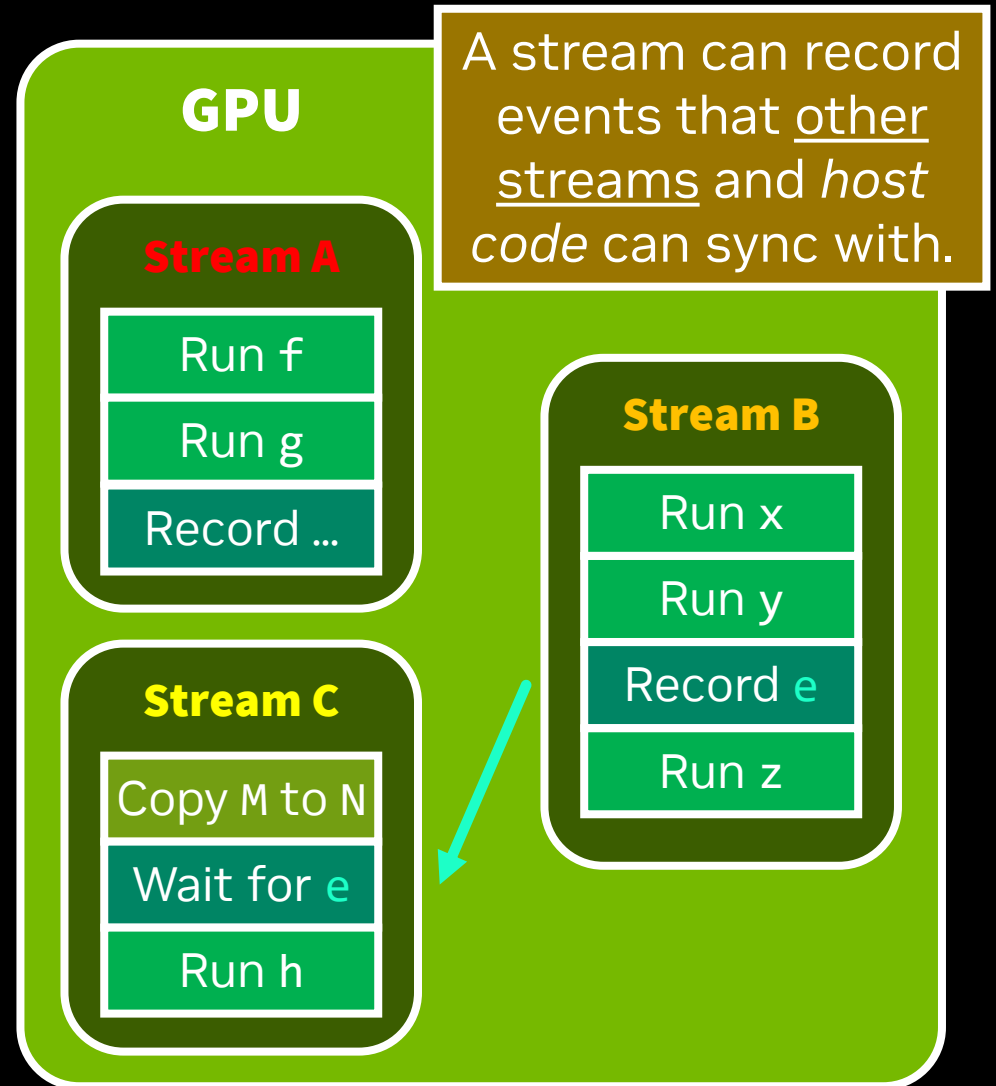
cuda::stream B;

thrust::for_each(pol.on(B), ..., x);
thrust::for_each(pol.on(B), ..., y);
auto e = B.record_event();
thrust::for_each(pol.on(B), ..., z);

cuda::stream C;

thrust::copy(pol.on(C), M, ..., N);
C.wait(e);
thrust::for_each(pol.on(C), ... h);

```



```
auto pol(thrust::cuda::par_nosync);

cuda::stream A(cuda::devices[0]);

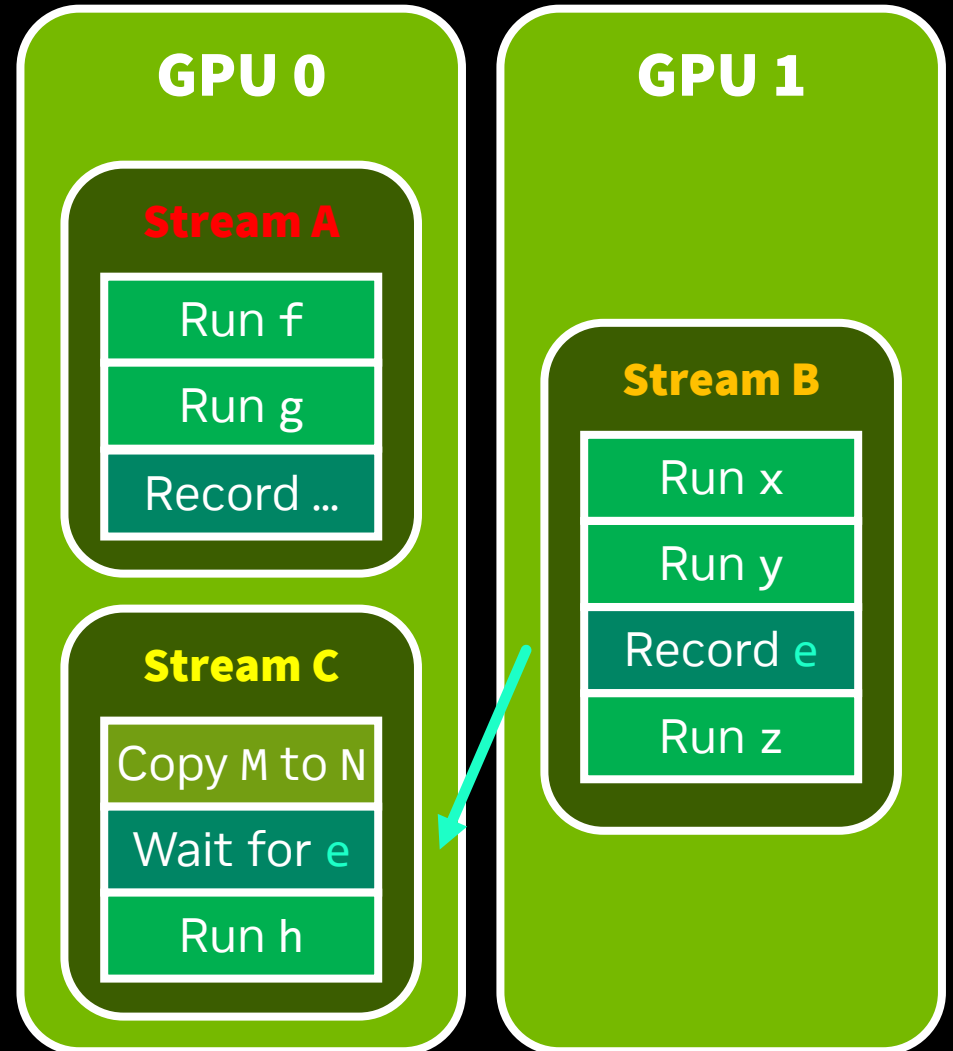
thrust::for_each(pol.on(A), ..., f);
thrust::for_each(pol.on(A), ..., g);
A.sync();

cuda::stream B(cuda::devices[1]);

thrust::for_each(pol.on(B), ..., x);
thrust::for_each(pol.on(B), ..., y);
auto e = B.record_event();
thrust::for_each(pol.on(B), ..., z);

cuda::stream C(cuda::devices[0]);

thrust::copy(pol.on(C), M, ..., N);
C.wait(e);
thrust::for_each(pol.on(C), ... h);
```



```

int nx(...), ny(...), write_steps(...), compute_steps(...);

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(...), V(...);

initialize_oven(thrust::cuda::par_nosync, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    cudaDeviceSynchronize();
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(thrust::cuda::par_nosync, U, V);
        U.swap(V);
    }
}

```



```

int nx(...), ny(...), write_steps(...), compute_steps(...);

cuda::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(...), V(...);

initialize_oven(policy, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    stream.sync();
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(policy, U, V);
        U.swap(V);
    }
}

```

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

Some Thrust algorithms return a result that depends on the computation. These interfaces will always block.

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

```
auto it2 = thrust::inclusive_scan(policy, A.begin(), A.end(), B.begin(), f2);  
// Also returns the end of the output, but needs temporary storage.
```

Some Thrust algorithms return a result that depends on the computation. These interfaces will always block.

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

```
auto it2 = thrust::inclusive_scan(policy, A.begin(), A.end(), B.begin(), f2);  
// Also returns the end of the output, but needs temporary storage.
```

```
auto it3 = thrust::copy_if(policy, A.begin(), A.end(), B.begin(), f3);  
// Returns the end of the output, but it depends on the computation.
```

Some Thrust algorithms return a result that depends on the computation. These interfaces will always block.

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

```
auto it2 = thrust::inclusive_scan(policy, A.begin(), A.end(), B.begin(), f2);  
// Also returns the end of the output, but needs temporary storage.
```

```
auto it3 = thrust::copy_if(policy, A.begin(), A.end(), B.begin(), f3);  
// Returns the end of the output, but it depends on the computation.
```

```
auto r4 = thrust::reduce(policy, A.begin(), A.end(), T{}, f4);  
auto r5 = thrust::find_if(policy, A.begin(), A.end(), f5);  
// Returns the result of the computation itself.
```

Some Thrust algorithms return a result that depends on the computation. These interfaces will always block.

```
auto policy = thrust::cuda::par_nosync;  
thrust::universal_vector A(...), B(...);
```

```
thrust::for_each_n(  
    policy, A.begin(), A.size(), f0);  
// Doesn't return anything.
```

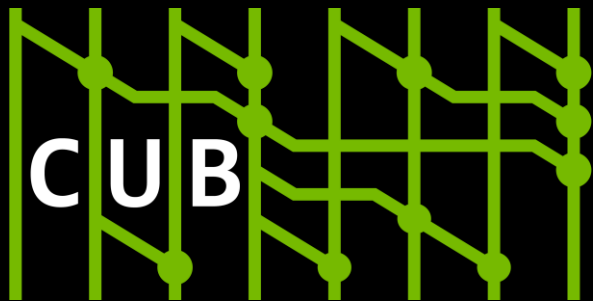
```
auto it1 = thrust::transform(policy,  
    A.begin(), A.end(), B.begin(), f1);  
// Returns an iterator to the end of the output (B).  
// Doesn't depend on the computation; it's just B.begin() + A.size().
```

```
auto it2 = thrust::inclusive_scan(policy, A.begin(), A.end(), B.begin(), f2);  
// Also returns the end of the output, but needs temporary storage.
```

```
auto it3 = thrust::copy_if(policy, A.begin(), A.end(), B.begin(), f3);  
// Returns the end of the output, but it depends on the computation.
```

```
thrust::universal_vector<T> out(1);  
thrust::reduce_into(policy, A.begin(), A.end(), out.begin() T{}, f4);
```

Some Thrust algorithms return a result that depends on the computation. These interfaces will always block.



The CUDA C++ Algorithm Authoring Toolkit

<https://nvidia.github.io/cccl/cub>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel



Device-Wide Kernels

- `cub::DeviceSegmentedReduce`
- `cub::DeviceHistogram`
- `cub::DeviceRadixSort`
- `cub::DeviceSelect`
- ...



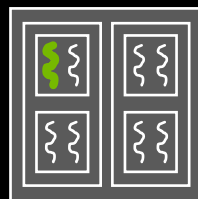
Block-Wide Cooperative Primitives

- `cub::BlockReduce`
- `cub::BlockExchange`
- `cub::BlockRunLengthDecode`
- `cub::BlockLoad/Store`
- ...



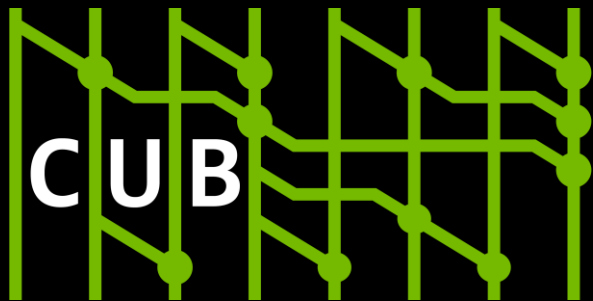
Warp-Wide Cooperative Primitives

- `cub::WarpReduce`
- `cub::WarpExchange`
- `cub::WarpMergeSort`
- `cub::WarpLoad/Store`
- ...



Per-Thread Primitives

- `cub::ThreadReduce`
- `cub::ThreadScan`
- `cub::MergePathSearch`
- `cub::ThreadLoad/Store`
- ...



The CUDA C++ Algorithm Authoring Toolkit

<https://nvidia.github.io/cccl/cub>

Learn More: [S72575](#) How You Should Write a CUDA C++ Kernel



Device-Wide Kernels

- `cub::DeviceSegmentedReduce`
- `cub::DeviceHistogram`
- `cub::DeviceRadixSort`
- `cub::DeviceSelect`
- ...



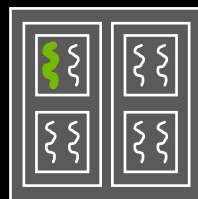
Block-Wide Cooperative Primitives

- `cub::BlockReduce`
- `cub::BlockExchange`
- `cub::BlockRunLengthDecode`
- `cub::BlockLoad/Store`
- ...



Warp-Wide Cooperative Primitives

- `cub::WarpReduce`
- `cub::WarpExchange`
- `cub::WarpMergeSort`
- `cub::WarpLoad/Store`
- ...



Per-Thread Primitives

- `cub::ThreadReduce`
- `cub::ThreadScan`
- `cub::MergePathSearch`
- `cub::ThreadLoad/Store`
- ...

```
thrust::universal_vector X(...);
```

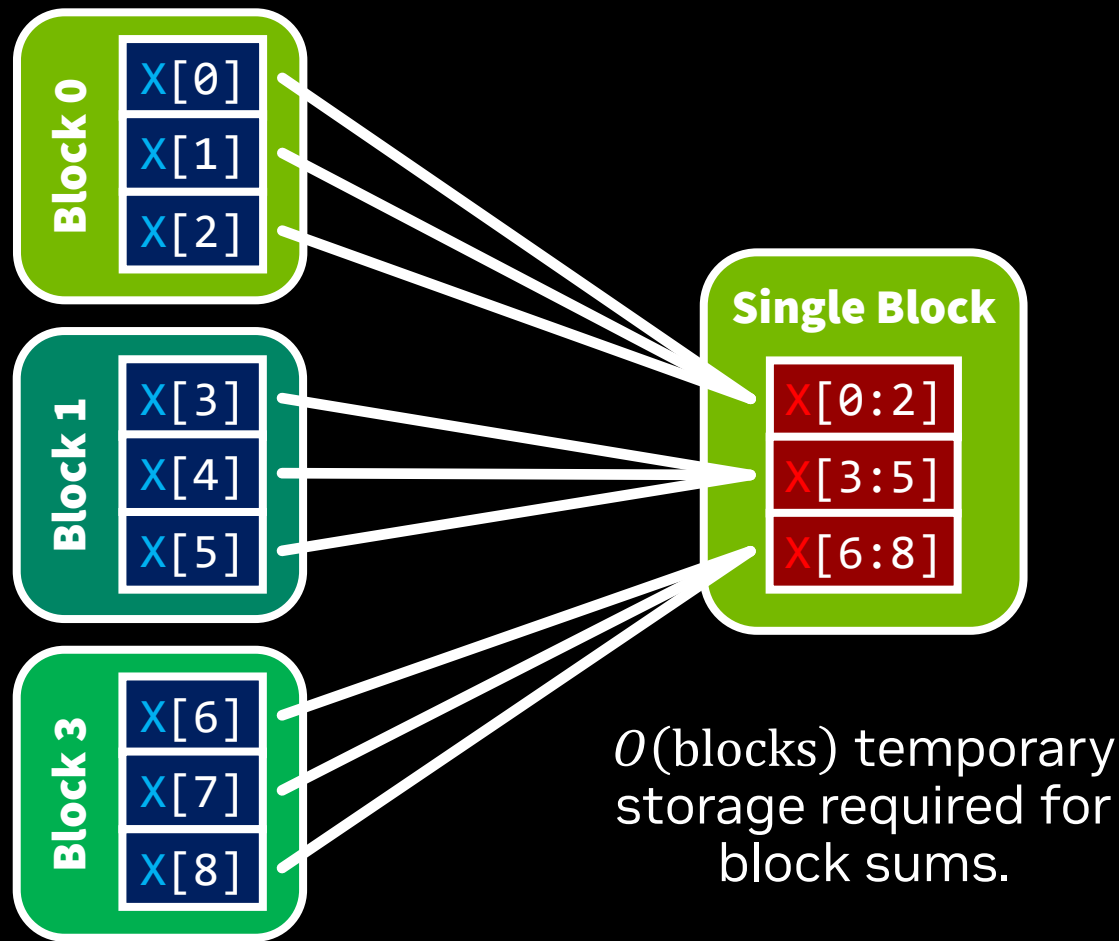
```
auto res = thrust::reduce(  
    thrust::cuda::par,  
    X.begin(), X.end(),  
    T{}, cuda::maximum{});
```

1st Kernel
 $O(\text{input}) \rightarrow O(\text{blocks})$

2nd Kernel
 $O(\text{blocks}) \rightarrow 1$

```
thrust::universal_vector X(...);
```

```
auto res = thrust::reduce(  
    thrust::cuda::par,  
    X.begin(), X.end(),  
    T{}, cuda::maximum{});
```



```
thrust::universal_vector X(...);  
thrust::universal_vector res(1);
```

```
thrust::universal_vector X(...);
```

```
auto res = thrust::reduce(  
    thrust::cuda::par,  
    X.begin(), X.end(),  
    T{}, cuda::maximum{});
```

```
thrust::universal_vector X(...);

auto res = thrust::reduce(
    thrust::cuda::par,
    X.begin(), X.end(),
    T{}, cuda::maximum{});
```

```
thrust::universal_vector X(...);
thrust::universal_vector res(1);

// Determine temporary storage size.
// Doesn't launch anything.
int tmp_size = 0;
cub::DeviceReduce::Reduce(
    nullptr,
    tmp_size,
    X.begin(), res.begin(), X.size(),
    cuda::maximum{}, T{});
```

```
thrust::universal_vector X(...);

auto res = thrust::reduce(
    thrust::cuda::par,
    X.begin(), X.end(),
    T{}, cuda::maximum{});
```

```
thrust::universal_vector X(...);
thrust::universal_vector res(1);

// Determine temporary storage size.
// Doesn't launch anything.
int tmp_size = 0;
cub::DeviceReduce::Reduce(
    nullptr,
    tmp_size,
    X.begin(), res.begin(), X.size(),
    cuda::maximum{}, T{});
```

```
thrust::universal_vector X(...);

auto res = thrust::reduce(
    thrust::cuda::par,
    X.begin(), X.end(),
    T{}, cuda::maximum{});
```

```
thrust::universal_vector X(...);
thrust::universal_vector res(1);

// Determine temporary storage size.
// Doesn't launch anything.
int tmp_size = 0;
cub::DeviceReduce::Reduce(
    nullptr,
    tmp_size,
    X.begin(), res.begin(), X.size(),
    cuda::maximum{}, T{});

thrust::device_vector<cuda::std::byte>
    tmp(tmp_size);
```

```
thrust::universal_vector X(...);

auto res = thrust::reduce(
    thrust::cuda::par,
    X.begin(), X.end(),
    T{}, cuda::maximum{});
```

```
thrust::universal_vector X(...);
thrust::universal_vector res(1);

// Determine temporary storage size.
// Doesn't launch anything.
int tmp_size = 0;
cub::DeviceReduce::Reduce(
    nullptr,
    tmp_size,
    X.begin(), res.begin(), X.size(),
    cuda::maximum{}, T{});

thrust::device_vector<cuda::std::byte>
    tmp(tmp_size);

// Launch kernel.
cub::DeviceReduce::Reduce(
    thrust::raw_pointer_cast(tmp.data()),
    tmp_size,
    X.begin(), res.begin(), X.size(),
    cuda::maximum{}, T{});
```



```

void heat_equation(auto policy,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    thrust::for_each_n(policy, thrust::make_counting_iterator(0), U.size(),
        [U, V] __host__ __device__ (int xy) {
            int x = xy / U.extent(1);
            int y = xy % U.extent(1);

            if (x > 0 && y > 0 && x < U.extent(0) - 1 && y < U.extent(1) - 1) {
                auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
                auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

                V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                V(x, y) = U(x, y);
            }
        });
}

```

```

void heat_equation(cuda::stream_ref stream,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    cub::DeviceFor::ForEachN(thrust::make_counting_iterator(0), U.size(),
    [U, V] __host__ __device__ (int xy) {
        int x = xy / U.extent(1);
        int y = xy % U.extent(1);

        if (x > 0 && y > 0 && x < U.extent(0) - 1 && y < U.extent(1) - 1) {
            auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
            auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

            V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
        } else {
            V(x, y) = U(x, y);
        }
    }, stream);
}

```

```

void heat_equation(cuda::stream_ref stream,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    cub::DeviceFor::ForEachN(thrust::make_counting_iterator(0), U.size(),
    [U, V] __host__ __device__ (int xy) {
        int x = xy / U.extent(1);
        int y = xy % U.extent(1);

        if (x > 0 && y > 0 && x < U.extent(0) - 1 && y < U.extent(1) - 1) {
            auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
            auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

            V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
        } else {
            V(x, y) = U(x, y);
        }
    }, stream);
}

```

```

void heat_equation(cuda::stream_ref stream,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    cub::DeviceFor::ForEachN(thrust::make_counting_iterator(0), U.size(),
    [U, V] __host__ __device__ (int xy) {
        int x = xy / U.extent(1);
        int y = xy % U.extent(1);

        if (x > 0 && y > 0 && x < U.extent(0) - 1 && y < U.extent(1) - 1) {
            auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
            auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

            V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
        } else {
            V(x, y) = U(x, y);
        }
    }, stream);
}

```

```

void heat_equation(cuda::stream_ref stream,
                  cuda::std::mdspan<float const, cuda::std::dims<2>> U,
                  cuda::std::mdspan<float, cuda::std::dims<2>> V) {
    cub::DeviceFor::ForEachInLayout(U.layout(),
    [U, V] __host__ __device__ (int idx, int x, int y) {
        if (x > 0 && y > 0 && x < I.extent(0) - 1 && y < U.extent(1) - 1) {
            auto d2tdx2 = U(x, y - 1) - U(x, y) * 2 + U(x, y + 1);
            auto d2tdy2 = U(x - 1, y) - U(x, y) * 2 + U(x + 1, y);

            V(x, y) = U(x, y) + 0.2f * (d2tdx2 + d2tdy2);
        } else {
            V(x, y) = U(x, y);
        }
    }, stream);
}

```

```

void initialize_oven(auto policy,
                    cuda::std::mdspan<float, cuda::std::dims<2>> U) {
    auto nx = U.extent(0);
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);
    auto mid = cuda::std::submdspan(U, cuda::std::tuple(1, nx-2), cuda::std::full_extent);
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);

    thrust::fill_n(policy, top.data_handle(), top.size(), 90.0);
    thrust::fill_n(policy, mid.data_handle(), mid.size(), 15.0);
    thrust::fill_n(policy, bot.data_handle(), bot.size(), 90.0);
}

```

```

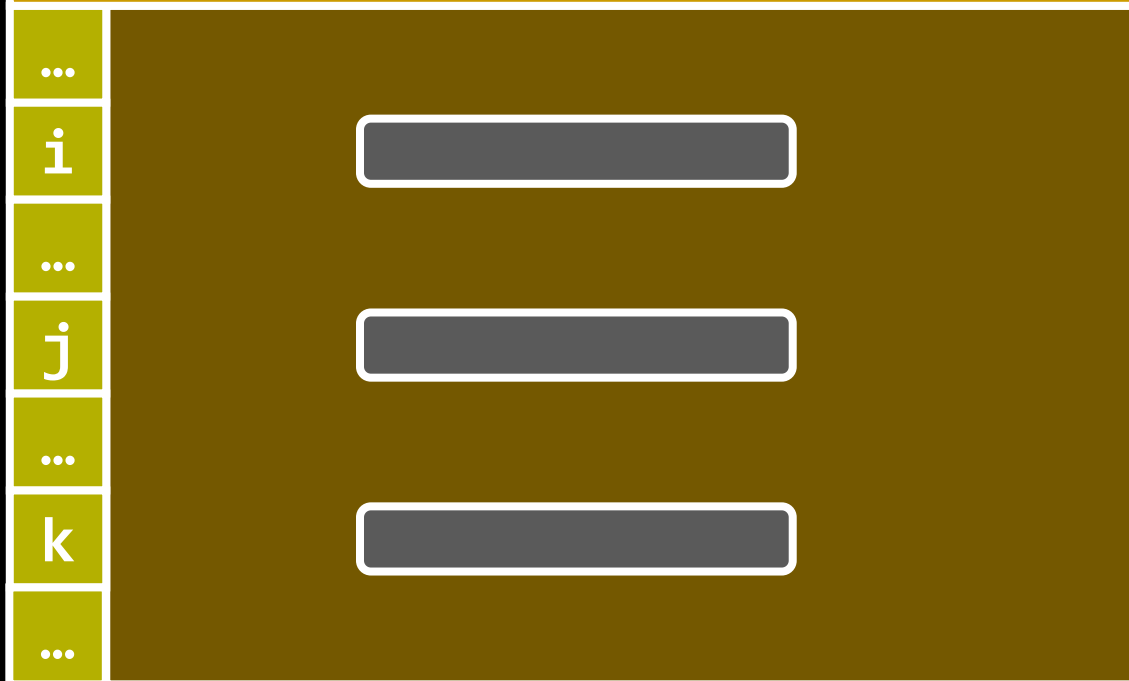
void initialize_oven(cuda::stream_ref stream,
                   cuda::std::mdspan<float, cuda::std::dims<2>> U) {
    auto nx = U.extent(0);
    auto top = cuda::std::submdspan(U, 0, cuda::std::full_extent);
    auto mid = cuda::std::submdspan(U, cuda::std::tuple(1, nx-2), cuda::std::full_extent);
    auto bot = cuda::std::submdspan(U, nx-1, cuda::std::full_extent);

    auto set_top = [top] __host__ __device__ (int idx, int x, int y) { top[x, y] = 90.0; };
    auto set_mid = [mid] __host__ __device__ (int idx, int x, int y) { mid[x, y] = 15.0; };
    auto set_bot = [bot] __host__ __device__ (int idx, int x, int y) { bot[x, y] = 90.0; };

    cub::DeviceFor::ForEachInLayout(top.layout(), set_top, stream);
    cub::DeviceFor::ForEachInLayout(mid.layout(), set_mid, stream);
    cub::DeviceFor::ForEachInLayout(bot.layout(), set_bot, stream);
}

```

thrust::universal_vector



Host Memory

Device Memory

`thrust::universal_vector`

...

i

...

j

...

k

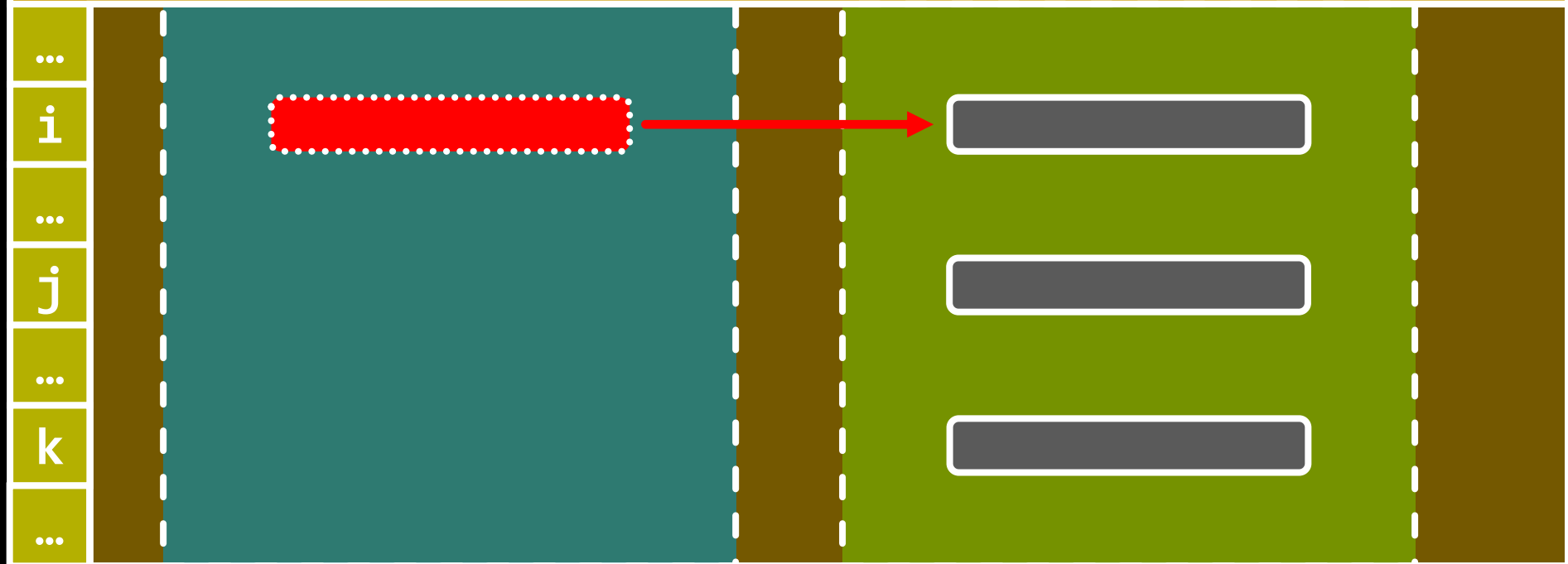
...



Host Memory

Device Memory

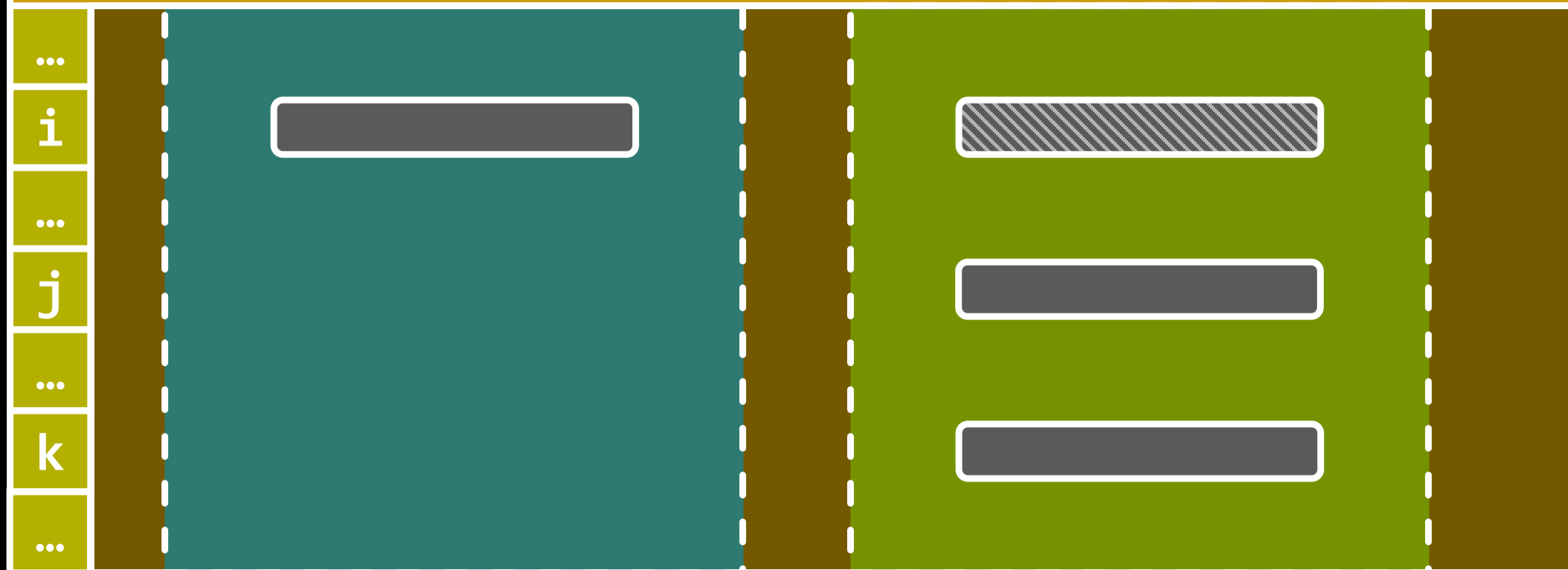
`thrust::universal_vector`



Host Memory

Device Memory

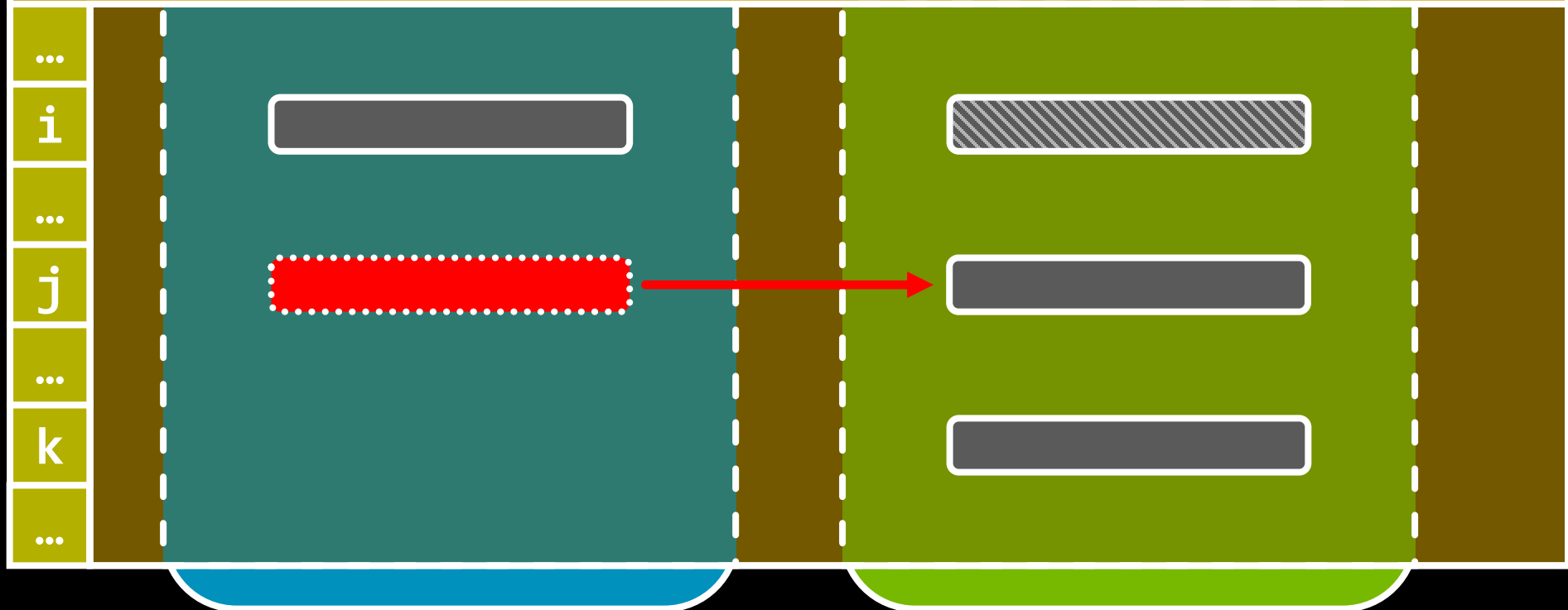
`thrust::universal_vector`



Host Memory

Device Memory

`thrust::universal_vector`



Host Memory

Device Memory

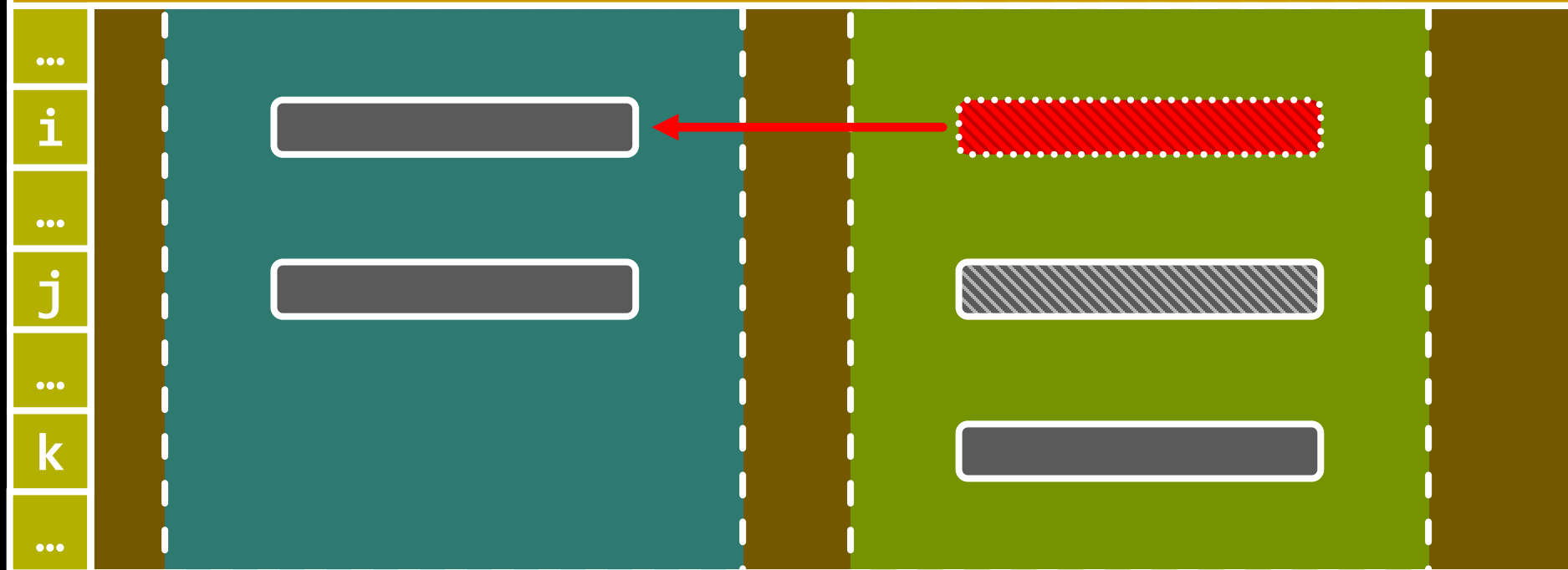
`thrust::universal_vector`



Host Memory

Device Memory

`thrust::universal_vector`



Host Memory

Device Memory

`thrust::universal_vector`



Host Memory

Device Memory

`thrust::universal_vector`

...

i

...

j

...

k

...

With Unified Memory,
transfers are implicit
and *piecemeal*.

Thrust Container	Accessible in...	Transfers are...
<code>universal_vector</code>	Host and device	Implicit upon access

Thrust Container	Accessible in...	Transfers are...
<code>universal_vector</code>	Host and device	Implicit upon access
<code>host_vector</code>	Host only	
<code>device_vector</code>	Device only	

Thrust Container	Accessible in...	Transfers are...
<code>universal_vector</code>	Host and device	Implicit upon access
<code>host_vector</code>	Host only	Explicit via <code>thrust::copy</code>
<code>device_vector</code>	Device only	

```
int nx(...), ny(...), write_steps(...), compute_steps(...);

cuda::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(...), V(...);

initialize_oven(stream, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    stream.wait();
    save_to_file(U);

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(stream, U, V);
        U.swap(V);
    }
}
```

```
int nx(...), ny(...), write_steps(...), compute_steps(...);

cuda::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
cuda::std::mdspan U(...), V(...);

initialize_oven(stream, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    stream.wait();
    save_to_file(U); // Implicit device to host transfer.

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(stream, U, V);
        U.swap(V);
    }
}
```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

cudax::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
thrust::host_vector<float> buf_data(nx * ny);
cuda::std::mdspan U(...), V(...), buf(...);

initialize_oven(stream, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    thrust::copy_n(policy, U.data(), U.size(), buf); // Device to host.
    stream.wait();

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(stream, U, V);
        U.swap(V);
    }

    save_to_file(buf); // No transfer.
}

```

```

int nx(...), ny(...), write_steps(...), compute_steps(...);

cudax::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
thrust::host_vector<float> buf_data(nx * ny);
cuda::std::mdspan U(...), V(...), buf(...);

initialize_oven(stream, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    thrust::copy_n(policy, U.data(), U.size(), buf); // Device to host.
    stream.wait();

    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(stream, U, V);
        U.swap(V);
    }

    save_to_file(buf); // No transfer.
}

```

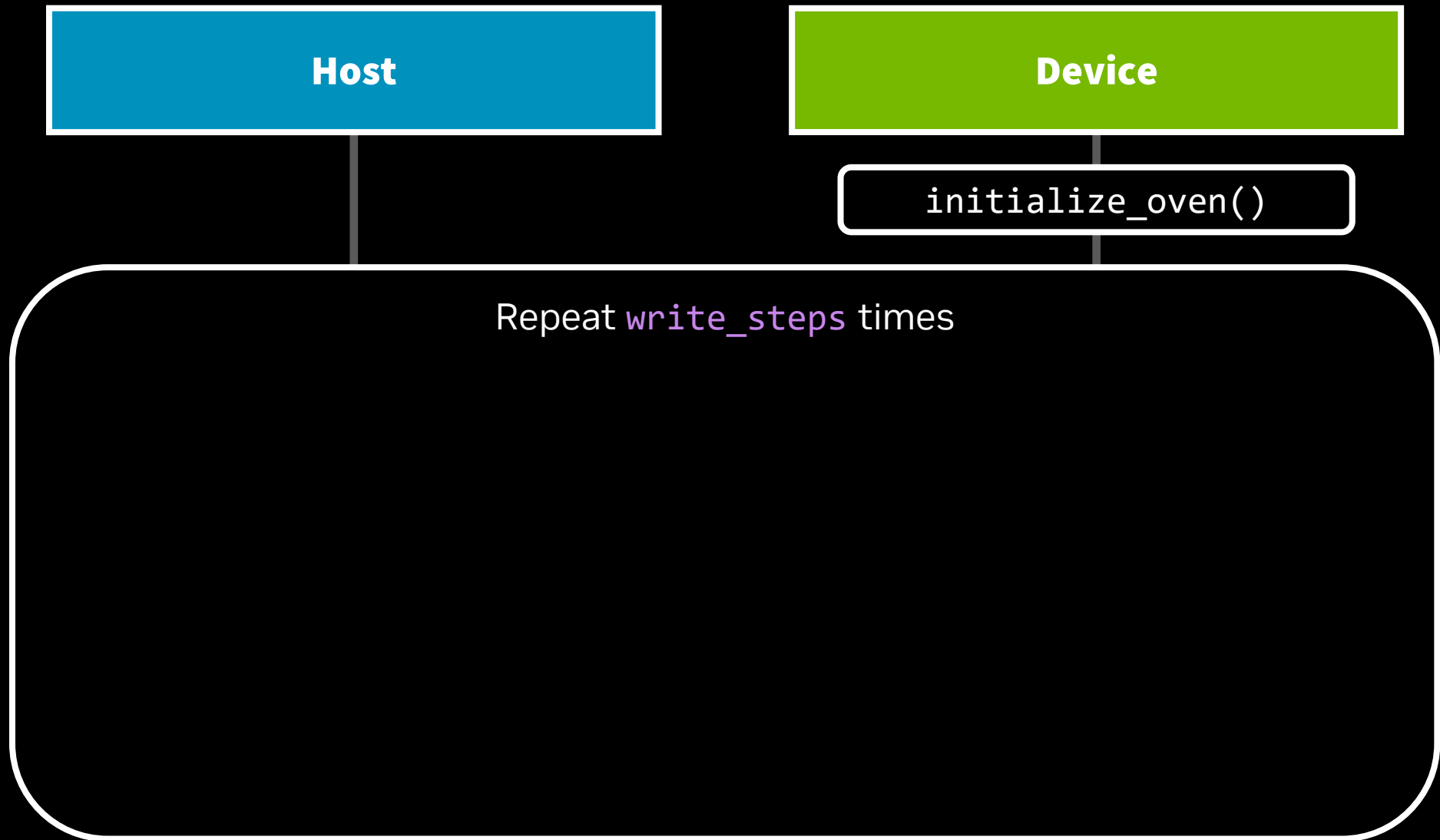


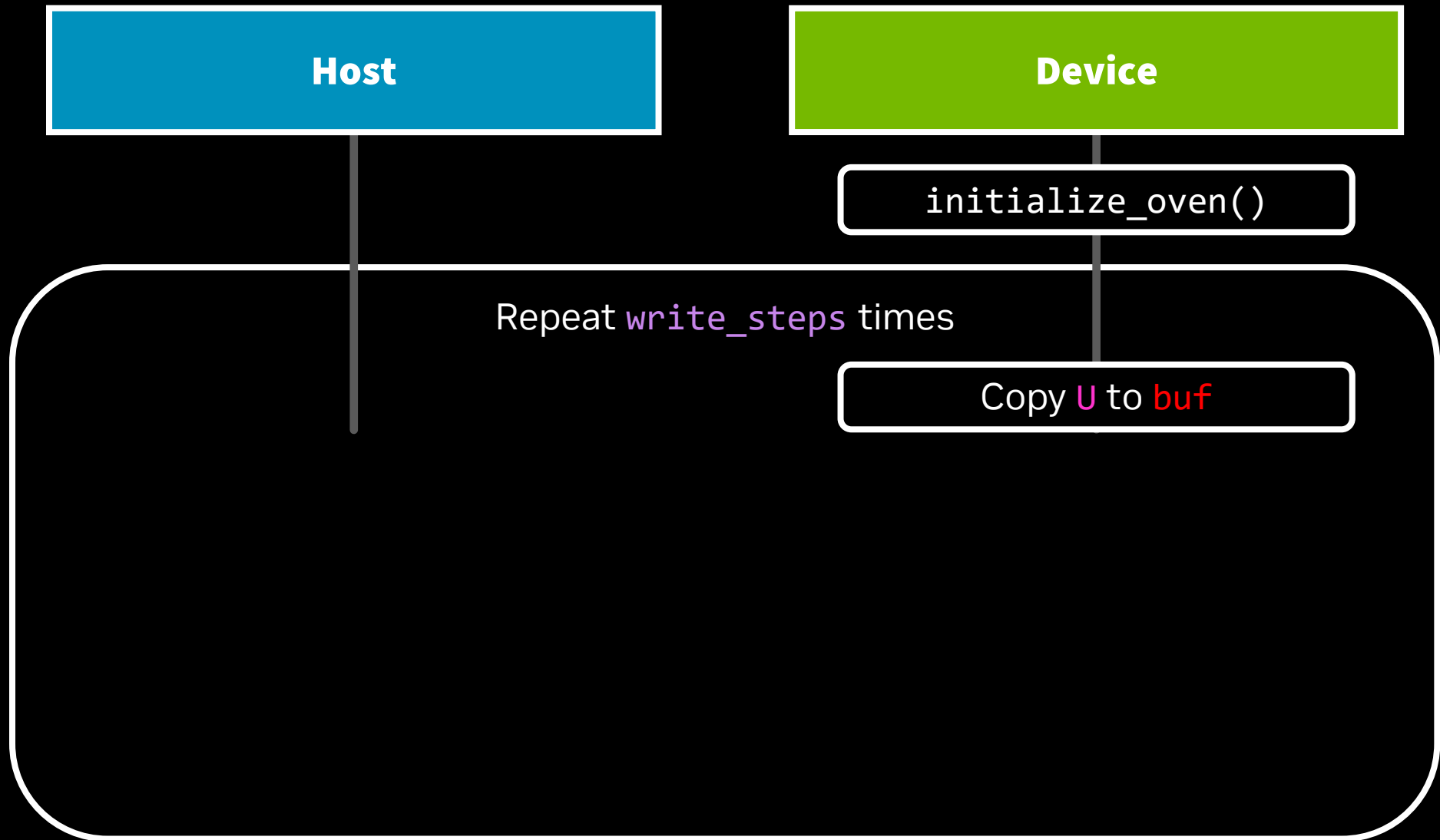
Host

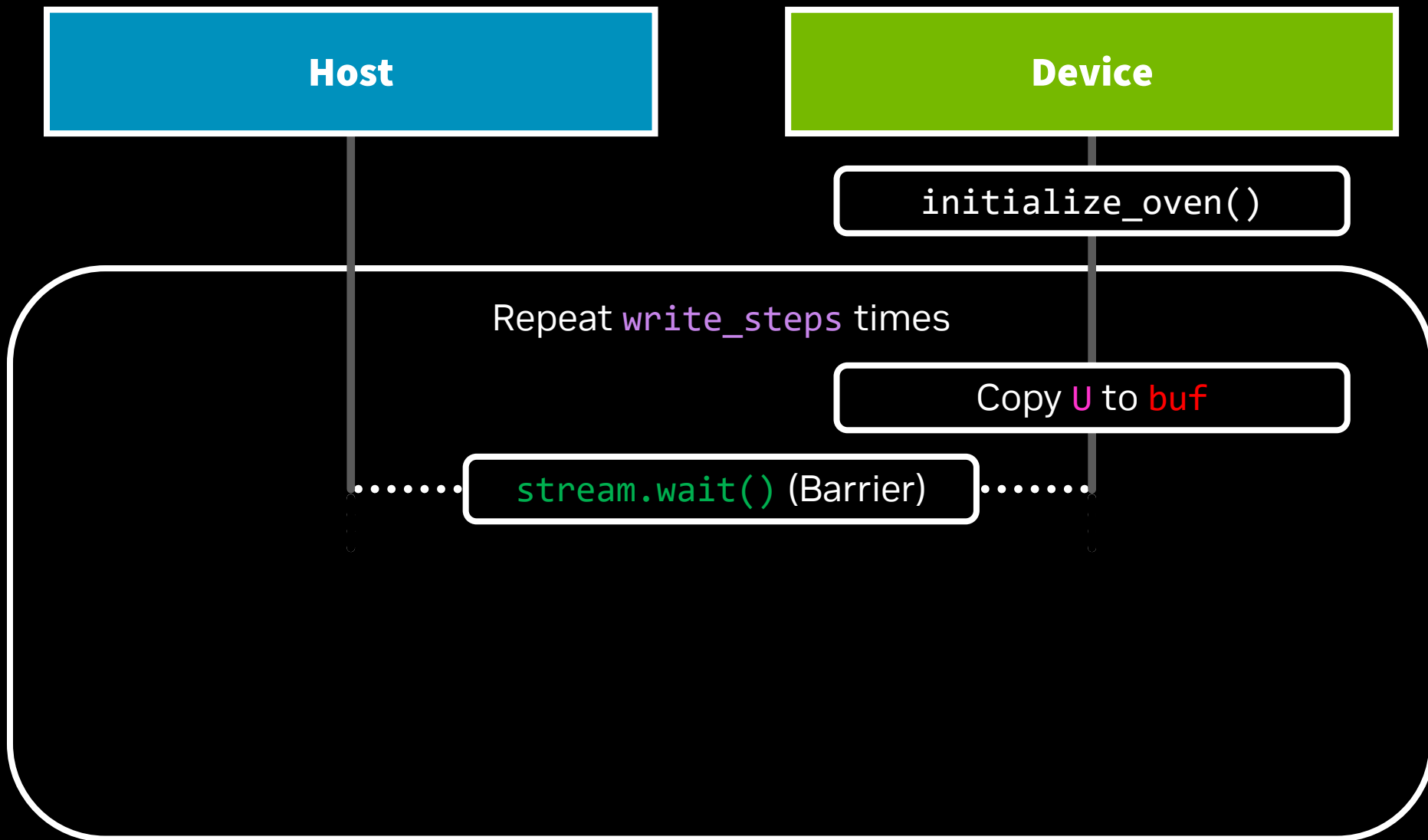


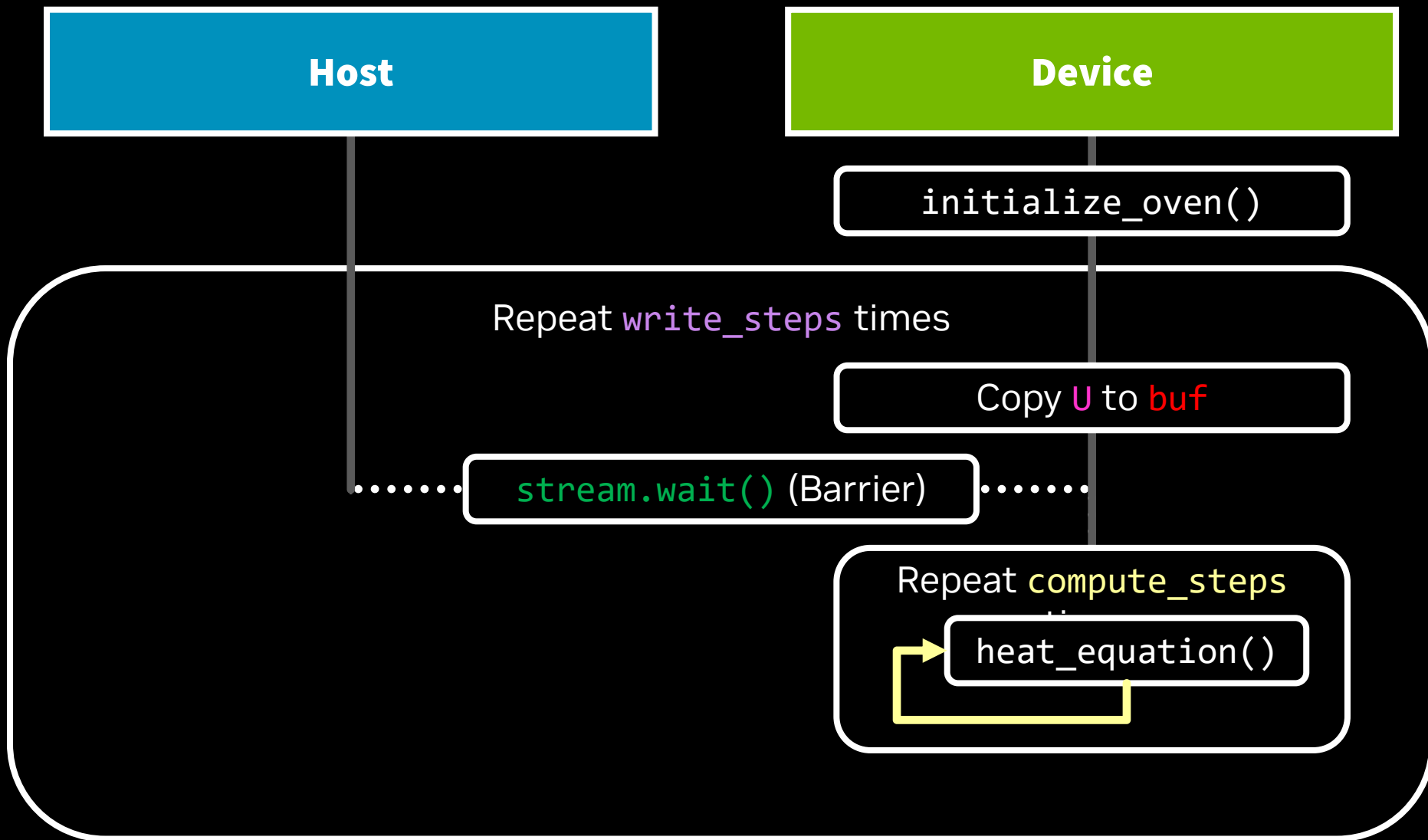
Device

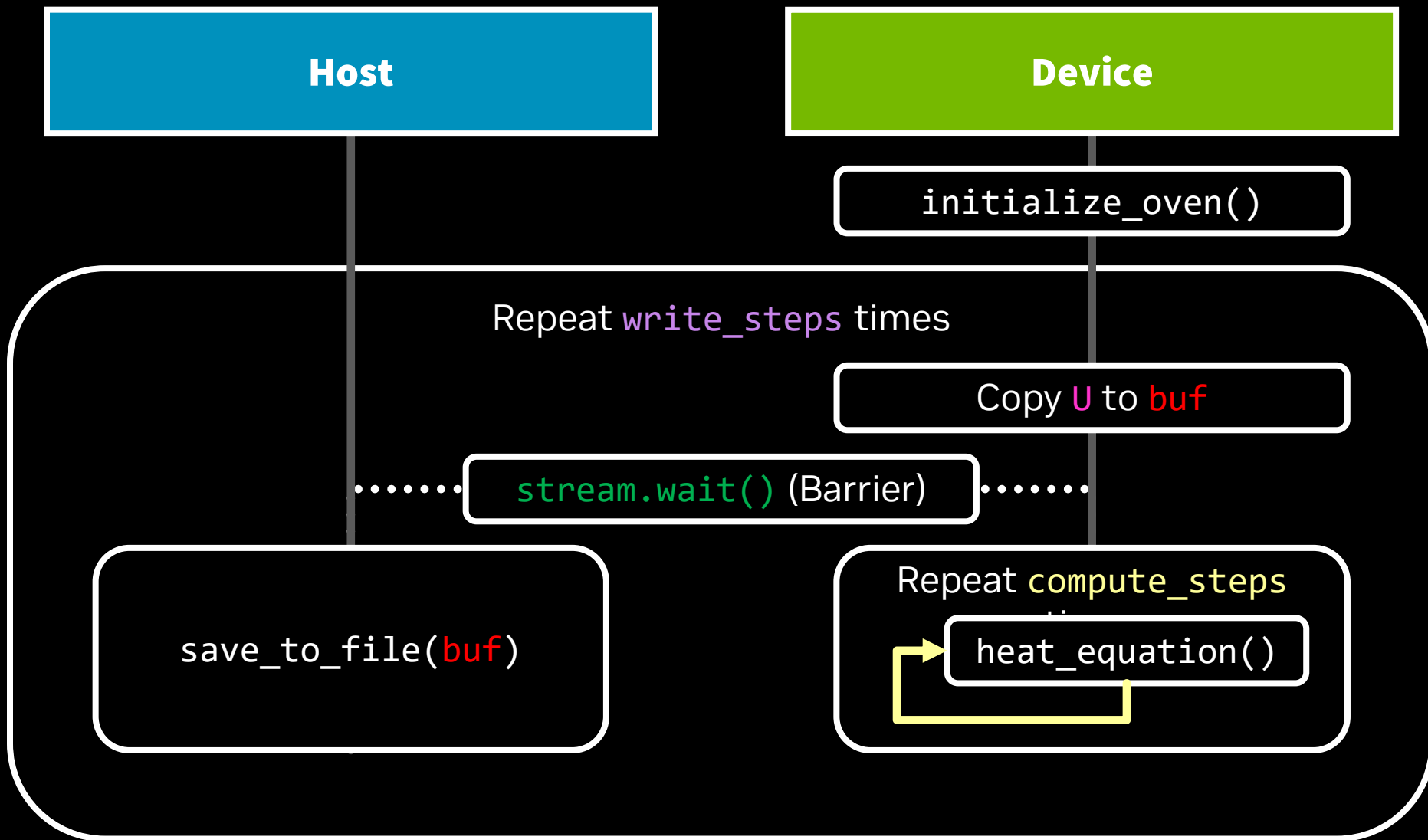
`initialize_oven()`

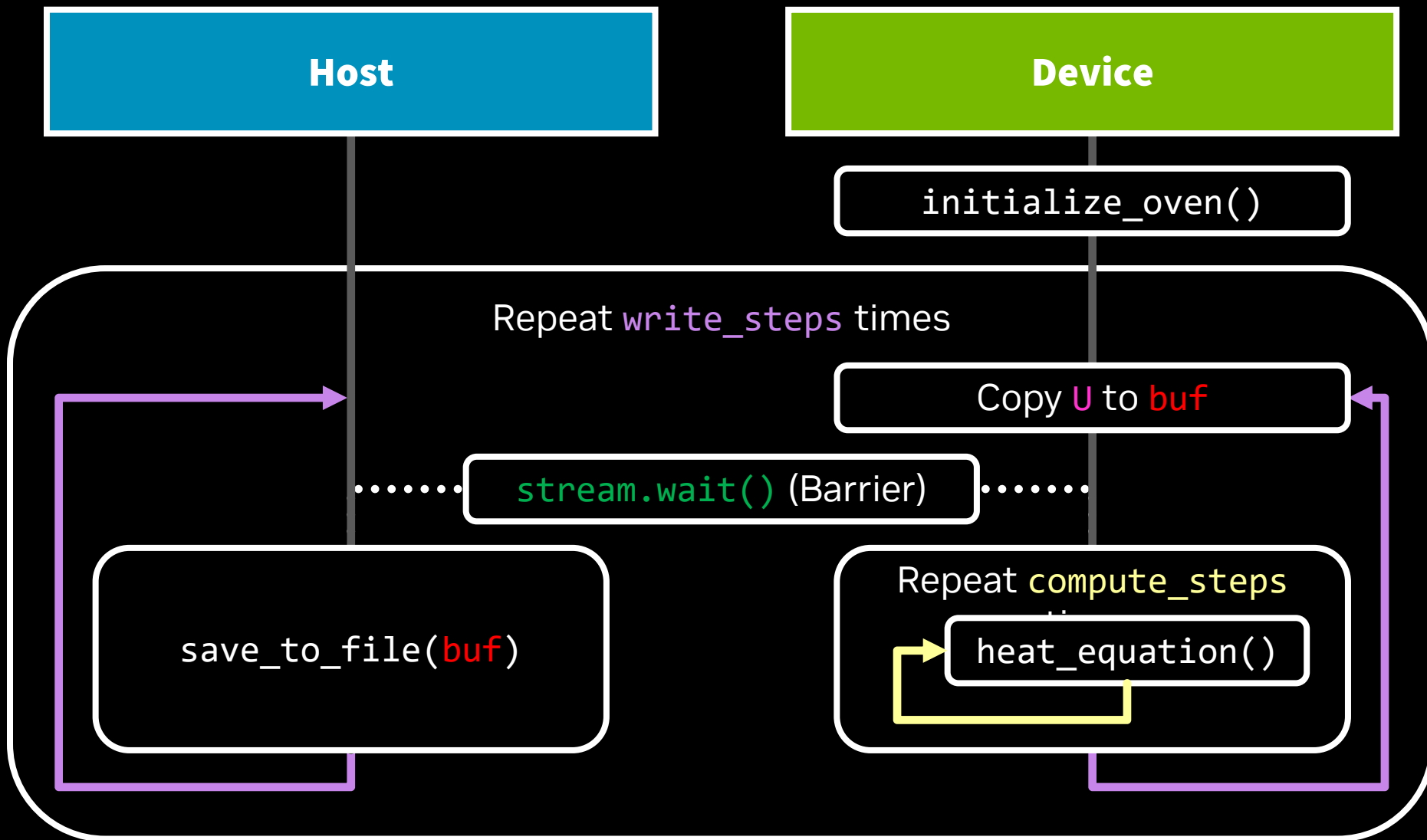












```

int nx(...), ny(...), write_steps(...), compute_steps(...);

cudax::stream stream;
auto policy(thrust::cuda::par_nosync.on(stream.get()));

thrust::universal_vector<float> U_data(nx * ny), V_data(nx * ny);
thrust::host_vector<float> buf_data(nx * ny);
cuda::std::mdspan U(...), V(...), buf(...);

initialize_oven(stream, U);

for (auto write_step : std::views::iota(0, write_steps)) {
    thrust::copy_n(policy, U.data(), U.size(), buf); // Device to host.
    stream.wait();

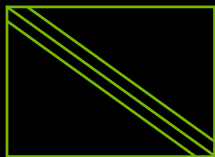
    for (auto compute_step : std::views::iota(0, compute_steps)) {
        heat_equation(stream, U, V);
        U.swap(V);
    }

    save_to_file(buf); // No transfer.
}

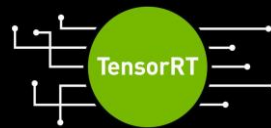
```



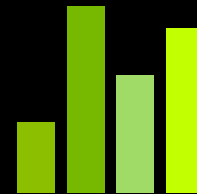
cuBLAS



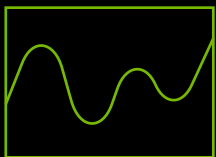
cuSPARSE



**cuFile
(GDS)**



NVBench



cuFFT

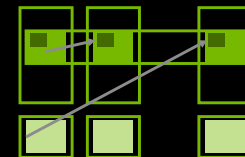


cuDSS

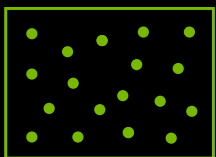
**CV
CUDA**

cuLitho

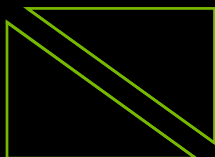
**Cooperative
Groups**



NVSHMEM



cuRAND



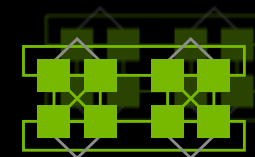
cuSOLVER

RAPIDS

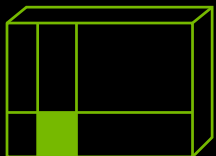


CUDA-Q

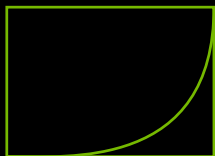
CUTLASS



NCCL



cuTENSOR



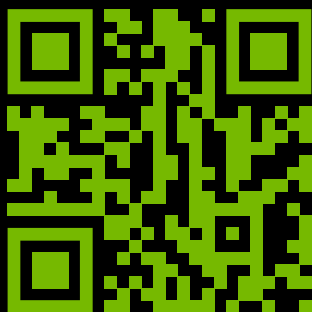
CUDA Math



**CUDA
Runtime**



GTC CUDA Developer Track
nvidia.ws/4gtfY1r



Accelerated Computing Hub
nvidia.ws/46b2Nik

GENERAL CUDA

- [S72571](#) What's CUDA All About Anyways?
- [S72897](#) How To Write A CUDA Program
- [S72527](#) Debugging & Optimizing CUDA with Intelligent Developer Tools

CUDA PYTHON

- [S72450](#) Accelerated Python: Tour of Community & Ecosystem
- [S72448](#) The CUDA Python Developer's Toolbox
- [S72449](#) 1001 Ways to Write CUDA Kernels in Python
- [S74639](#) Tensor Core Programming in Python with CUTLASS 4.0

CUDA C++

- [S72574](#) Building CUDA Software at the Speed-of-Light
- [S72572](#) The CUDA C++ Developer's Toolbox
- [S72575](#) How You Should Write a CUDA C++ Kernel

MULTI-GPU PROGRAMMING

- [S72576](#) Getting Started with Multi-GPU Scaling: Distributed Libraries
- [S72579](#) Going Deeper with Multi-GPU Scaling: Task-based Runtimes
- [S72578](#) Advanced Multi-GPU Scaling: Communication Libraries

PERFORMANCE OPTIMIZATION

- [S72683](#) CUDA Techniques to Maximize Memory Bandwidth & Hide Latency
- [S72685](#) CUDA Techniques to Maximize Compute & Instruction Throughput
- [S72686](#) CUDA Techniques to Maximize Concurrency & System Utilization
- [S72687](#) Get the Most Performance From Grace Hopper

NVIDIA DLI Courses

- Fundamentals of Accelerated Computing with CUDA Python
- Fundamental of Accelerated Computing with Modern CUDA C++
- Fundamentals of Accelerated Data Science
- Accelerate Data Science Workflows with Zero Code Changes
- Optimizing CUDA Machine Learning Codes With Nsight Profiling Tools

Scan the QR code to access the full course list and redeem one free training!

nvidia.com/training/42DdlEt





Algorithms +
Data
Structures =
Programs



```
constexpr std::array primes = {2, 3, 5, 7 // ... 97};

constexpr auto is_prime(int n) -> bool {
    return std::ranges::find(primes, n) != primes.end();
}

auto maximum_prime_difference(std::span<int const> nums) -> int {
    auto a = std::ranges::find_if(nums, is_prime);
    auto b = std::ranges::begin(std::ranges::find_last_if(nums, is_prime));
    return std::distance(a, b);
}
```



```
> [2,4,3] :: iota flat < prior : + * r . scan maxr add1
R      F < p δ + x r μ \ m '
[2,4,3] [1, 2], [1, 2, 3, 4], [1, 2, 3]
RF      [2,4,3] [1, 2, 1, 2, 3, 4, 1, 2, 3]
RF<     [2,4,3] [1, 1, 1, 2, 3, 4, 1, 2, 3]
RF<p    [2,4,3] [1, 0, 1, 1, 1, 0, 1, 1]
RF<pδ+  [2,4,3] [3, 4, 4, 1, 1, 0, 1, 1]
RF<pδ+x [2,4,3] [3, 0, 4, 2, 2, 0, 2, 2]
RF<pδ+xr [2,4,3] [6, 16, 12, 1, 1, 0, 1, 1]
RF<pδ+xrμ [2,4,3] [1, 0, 1, 2, 3, 0, 1, 2]
RF<pδ+xrμm [2,4,3] 3
RF<pδ+xrμm' [2,4,3] 4

This is a 1-1-2-q-s-2-2-2-s-q-1-1 monadic chain
      B h1 Φ1
      B h1
      B h1
      B
      B
      B
```

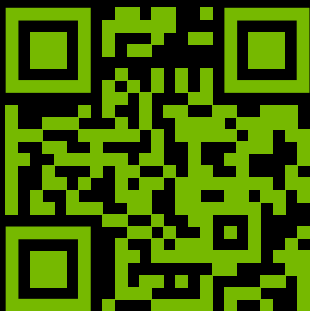




**GTC CUDA
Developer Track**
nvidia.ws/4gtfY1r



**Accelerated
Computing Hub**
nvidia.ws/46b2Nik



**NVIDIA
DLI Courses**
nvidia.ws/42DdIEt



**ADSP
Podcast**
adspthepodcast.com



THE CUDA C++ DEVELOPER'S TOOLBOX

Bryce Adelstein Lebach Principal Architect

✉ brycelelbach@gmail.com [@blelbach](https://twitter.com/blelbach)